ИНФОРМАТИКА

В. М. Котов
А. И. Лапо

7



В. М. Котов А. И. Лапо

ИНФОРМАТИКА

Учебное пособие для 7 класса
учреждений образования, реализующих образовательные
программы общего среднего образования,
с русским языком обучения и воспитания

Допущено
Министерством образования
Республики Беларусь

2-е издание, пересмотренное
и дополненное

Минск «Адукацыя і выхаванне» 2025

УДК 004(075.3=161.1)
ББК 32.81я721
К73

Рецензент

кафедра прикладной математики и информатики
учреждения образования «Брестский государственный университет
имени А. С. Пушкина» (кандидат физико-математических наук, доцент,
заведующий кафедрой *Д. В. Грицук*)

ISBN 978-985-34-0336-7

© Котов В. М., Лапо А. И., 2017
© Котов В. М., Лапо А. И., 2025,
с изменениями
© Оформление. Республиканское
унитарное предприятие «Издательство
“Адукацыя і выхаванне”», 2025

СОДЕРЖАНИЕ

Как работать с учебным пособием 6

Глава 1. Аппаратное и программное обеспечение компьютера

§ 1. Современные компьютерные устройства 8
1.1. Различные виды компьютерных устройств . . . . . —
1.2. Назначение устройств персонального компьютера 10
§ 2. Операционная система 14
2.1. Основные виды операционных систем . . . . . —
2.2. Элементы графического пользовательского интерфейса 16
2.3. Основные элементы файловой системы 19
2.4. Типовые операции с файлами и папками 21
§ 3. Локальная компьютерная сеть 25
§ 4. Архивация 28
4.1. Программы-архиваторы . . . . . —
4.2. Создание архивов и извлечение файлов из архива 30
§ 5. Программное обеспечение 33
5.1. Классификация программного обеспечения . . . . . —
5.2. Вредоносные программы и способы защиты от них 35

Глава 2. Представление о логике высказываний. Множества и операции над ними

§ 6. Логика высказываний 39
6.1. Понятие высказывания 40
6.2. Логическая операция **НЕ** 41
§ 7. Логические операции **И** и **ИЛИ** 45
7.1. Логическая операция **И** . . . . . —
7.2. Логическая операция **ИЛИ** 47
§ 8. Использование условий для исполнителей 50
8.1. Понятие условия . . . . . —
8.2. Условия для компьютерных исполнителей 53
§ 9. Множества. Операции над множествами 57
§ 10. Использование логических операций для построения поисковых запросов в Интернете 64

Глава 3. Основные алгоритмические конструкции

§ 11. Алгоритмы и исполнители 70
11.1. Понятие алгоритма . . . . . —
11.2. Исполнитель *Черепаха* 72
11.3. Алгоритмическая конструкция *следование* 73
11.4. Вспомогательные алгоритмы 74
§ 12. Исполнитель *Робот* 78
12.1. Роботы в жизни человека . . . . . —
12.2. Среда обитания и система команд компьютерного исполнителя *Робот* 80
12.3. Использование алгоритмической конструкции *следование* для исполнителя *Робот* 83
12.4. Вспомогательные алгоритмы для исполнителя *Робот* 86
§ 13. Алгоритмическая конструкция *повторение* 91
13.1. Алгоритмы с циклами . . . . . —
13.2. Использование команды *цикл с параметром* для исполнителя *Робот* 95
§ 14. Использование условий 99
14.1. Условия для исполнителя *Робот* . . . . . —
14.2. Команда *цикл с предусловием* 100
14.3. Использование команды *цикл с предусловием* для исполнителя *Робот* 103
§ 15. Алгоритмическая конструкция *ветвление* 106
15.1. Команда *ветвление* . . . . . —
15.2. Составные условия 109
15.3. Использование команды *ветвление* для исполнителя *Робот* 111
§ 16. Использование основных алгоритмических конструкций для исполнителя *Робот* 115
§ 17. Язык программирования Python 120
17.1. Команда вывода 121
17.2. Понятие типа данных 123
17.3. Оператор присваивания 125
17.4. Ввод данных 127
§ 18. Организация вычислений 129
18.1. Вычисление значения арифметического выражения 130
18.2. Использование языка программирования для решения задач 131

§ 19. Реализация алгоритмов работы с целочисленными данными 135
19.1. Целочисленный тип данных . . . . . —
19.2. Использование целочисленных данных для решения задач 136

Глава 4. Работа с векторной графикой

§ 20. Понятие векторной графики 141
§ 21. Интерфейс векторного графического редактора 146
§ 22. Создание и редактирование векторного изображения 149
22.1. Создание фигур 150
22.2. Редактирование фигур 154
22.3. Обводка и заливка 155
22.4. Работа с цветом 156
§ 23. Операции над объектами векторного изображения 163
23.1. Копирование, выравнивание и взаимное расположение объектов . . . . . —
23.2. Группировка. Операции объединения и пересечения объектов 166
§ 24. Работа с текстом 171

Глава 5. Информация и информационные процессы

§ 25. Информация в жизни человека 176
25.1. Виды информации . . . . . —
25.2. Носители информации 178
25.3. Информационные процессы 179
§ 26. Представление информации в компьютере 183
26.1. Кодирование информации . . . . . —
26.2. Единицы измерения объёма информации 186

Правообладатель Адукацыя і выхаванне

Как работать с учебным пособием

Дорогие семиклассники! Вы держите в руках учебное пособие по информатике — предмету, изучение которого позволит вам получить необходимые знания и умения в области информационных технологий, ставших неотъемлемой частью нашей жизни.

В настоящее время научные исследования и практические разработки в области информатики особенно востребованы и актуальны. Роль этой дисциплины в условиях современного мира всё более возрастает. Мы, авторы учебного пособия, постарались сделать так, чтобы изучение информатики было для вас интересным и увлекательным. Надеемся, что полученные знания вы сможете применить для решения практических задач из различных предметных областей.

Материал учебного пособия разделён на две колонки. Цвет фона определяет назначение размещённой на нём информации:

— основной материал, обязательный для изучения. Понятия, которые понадобятся вам для понимания и объяснения изучаемого материала, выделены полужирным шрифтом;

— примеры, иллюстрирующие основные материалы;

— определения основных понятий, обязательные для запоминания;

— дополнительный материал для любознательных.

В учебном пособии используются следующие условные обозначения:

 — вопросы и задания для проверки знаний;

 — раздел «Упражнения» содержит задания для выполнения в тетради;

 — раздел «Упражнения» содержит задания, при выполнении которых используется компьютер;

 — раздел «Упражнения» содержит задания, при выполнении которых может быть использована информация, размещённая на Едином информационно-образовательном ресурсе;

* — задание или пример для любознательных.

Зайти на сайт Единого информационно-образовательного ресурса можно с помощью QR-кода или по ссылке https://eior.by.





После перехода выберите в меню «7 класс», «Информатика»; в соответствующей теме нажмите кнопку «Дополнительные материалы» и скачайте нужный материал.

Имя файла для скачивания содержит номер параграфа и номер задания из упражнения после этого параграфа. Например, имя файла `upr4_3` означает, что файл относится к третьему упражнению после четвёртого параграфа. Также на портале размещены файлы, рассмотренные в некоторых примерах. Такие файлы имеют имя `pr20_1` (пример 1 из параграфа 20). Расширения в файлах примеров или файлах, предназначенных для выполнения упражнений, соответствуют тому программному средству, которое изучается в соответствующем параграфе.

Правообладатель Адукацыя і выхаванне

Глава 1
АППАРАТНОЕ И ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ КОМПЬЮТЕРА

§ 1. Современные компьютерные устройства

1.1. Различные виды компьютерных устройств

Развитие вычислительной техники привело к появлению большого разнообразия устройств. Современные компьютеры имеют различную конструкцию и внешний вид (пример 1.1).

Совокупность всех устройств компьютера называют его **аппаратным обеспечением.**

Настольный компьютер состоит из системного блока и подключённых к нему внешних устройств. Пользователь сам определяет, какие устройства будут подключаться к системному блоку.

В **мобильных компьютерах** все необходимые устройства находятся в одном корпусе. Такие компьютеры имеют возможность беспроводного подключения к внешним устройствам и сетям.

Основные разновидности мобильных компьютеров:

Пример 1.1. Разнообразие компьютерных устройств.



Настольный компьютер



Ноутбук



Планшет



Смартфон

1. Ноутбуки — полноценные компьютеры с клавиатурой, экраном, устройством памяти и возможностью использования широкого спектра программ.

2. Планшетные компьютеры (планшеты) имеют ограниченные возможности по сравнению с ноутбуками, виртуальную клавиатуру и операционную систему с набором команд.

3. Смартфоны — телефоны с возможностями компьютера. Современные смартфоны прекрасно справляются со многими задачами, не свойственными телефонам. Это работа с электронной почтой, создание и редактирование текстовых документов, просмотр фильмов, прослушивание музыки и многое другое.

Для решения наиболее сложных задач применяют **суперкомпьютеры.** Они обладают огромной вычислительной мощностью и превосходят по своим характеристикам большинство существующих в мире компьютеров. Среди областей их применения можно отметить метеорологию, сейсмологию, математическое моделирование, авиационную промышленность и др.

Пример 1.1. *Продолжение.*



Суперкомпьютер, работающий в Объединённом институте проблем информатики (ОИПИ) Национальной академии наук (НАН) Беларуси



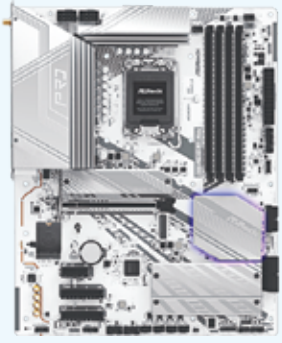
Офисный мобильный суперкомпьютер «СКИФ-ГЕО-Офис», созданный в ОИПИ НАН Беларуси в 2024 г. Он разрабатывался в рамках программы Союзного государства «СКИФ-НЕДРА» для повышения эффективности добычи полезных ископаемых, изучения недр земли



Один из возможных компьютеров будущего

Правообладатель Адукацыя і выхаванне

Пример 1.2. Устройства компьютера, расположенные в системном блоке.

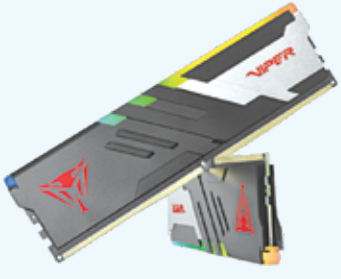


Материнская плата



Процессор

Оперативная память конструктивно представляет собой набор микросхем, размещённых на одной небольшой плате. Модули оперативной памяти вставляются в соответствующие разъёмы материнской платы.



Модули оперативной памяти

1.2. Назначение устройств персонального компьютера

Состав устройств (конфигурация) компьютера может изменяться в зависимости от решаемых задач.

Базовая конфигурация настольного компьютера содержит следующие функциональные блоки: системный блок, монитор, клавиатуру, мышь. В мобильных компьютерах эти устройства объединены в единое целое.

В **системном блоке** размещаются: материнская плата, блок питания, устройства памяти (пример 1.2), карты расширений (видеокарта, звуковая карта, сетевая карта).

Все компоненты компьютера связаны между собой посредством большой печатной платы. Эту плату называют **материнской платой.** На ней устанавливают процессор.

Процессор — важнейшее устройство компьютера, его мозг. Он обрабатывает данные, выполняя вычисления.

Устройства памяти предназначены для хранения информации. Память компьютера бывает внутренней и внешней.

Внутренняя память предназначена для хранения программ и их данных в процессе работы компьютера, подразделяется на оперативную и постоянную.

Оперативная память (RAM) служит для хранения программ и данных, с которыми компьютер работает в данный момент.

Обмен данными между процессором и оперативной памятью выполняется очень быстро. При выключении электропитания все данные из оперативной памяти исчезают.

Постоянная память (ROM) — это энергонезависимая память для хранения программ управления работой и тестирования устройств компьютера.

Постоянная память представляет собой микросхему, расположенную на материнской плате. Кроме программы первоначального тестирования компьютера в постоянной памяти хранится BIOS (базовая система ввода-вывода).

Внешняя память (пример 1.3) предназначена для долговременного и энергонезависимого

Пример 1.3. Устройства внешней памяти.



Жёсткий диск со снятой крышкой



Твердотельный накопитель



Твердотельный накопитель, который подключается к системному блоку через USB-порт



Устройства флеш-памяти (USB-накопители и флеш-карты)

Правообладатель Адукацыя і выхаванне

Пример 1.4. Периферийные устройства.



Видеопроектор



Оцифровка
с помощью документ-камеры



Веб-камера



Колонки

хранения программ и данных. К одному компьютеру можно подключить несколько таких устройств памяти.

К устройствам внешней памяти относятся:

1. HDD — жёсткий диск. Состоит из одного или нескольких магнитных дисков и считывающей головки, которая перемещается над вращающимися дисками и считывает/записывает данные.

2. SSD — твердотельный накопитель. По сравнению с жёстким диском более современный и высокотехнологичный вид памяти. Представляет собой плату с размещёнными на ней микросхемами.

3. Флеш-память. Носителем информации у такой памяти являются микросхемы. Устройства флеш-памяти подключаются к материнской плате через специальные порты.

Порт — разъём материнской платы или карты расширения, обычно выводится на заднюю панель системного блока.

HDD и SSD обычно находятся внутри системного блока и подключаются напрямую к материнской плате, но есть и такие,

которые подключаются к системному блоку через USB-порт.

Периферийные устройства — дополнительные устройства, которые подключаются к системному блоку для расширения функциональных возможностей компьютера.

С назначением клавиатуры, мыши, монитора, принтера и сканера вы познакомились в 6-м классе. Рассмотрим назначение других периферийных устройств (пример 1.4):

1. Видеопроектор — предназначен для проецирования изображения на большой экран.

2. Документ-камера позволяет получить цифровое изображение любых предметов.

3. Веб-камера — небольшая цифровая камера, способная в реальном времени фиксировать видеоданные, предназначенные для дальнейшей передачи по сети Интернет.

4. Для ввода звуковой информации используют **микрофон**, а для воспроизведения — **акустические системы** (звуковые колонки и наушники). Иногда микрофон и наушники объединяются в одно устройство — **гарнитуру**.

Пример 1.4. *Продолжение.*



Наушники



Микрофон



Гарнитура

Правообладатель Адукацыя і выхаванне

1. Что такое аппаратное обеспечение компьютера?
2. Что можно отнести к мобильным компьютерным устройствам?
3. Для чего предназначен процессор?
4. Какой бывает память компьютера?
5. Для чего предназначена оперативная память?
6. Какие устройства относят к внешней памяти?
7. Какие устройства предназначены для ввода информации?
8. С помощью каких устройств можно выводить информацию?

Упражнения

1 Определите объём устройств внешней памяти компьютера, к которым вы имеете доступ в школе или дома. Какой объём занимает информация, хранящаяся на них? Запишите данные в тетрадь.

2 Каким, на ваш взгляд, может быть компьютер будущего? Напишите сообщение на эту тему.

3 Создайте изображение компьютера будущего с помощью графического редактора Paint.

§ 2. Операционная система

Пример 2.1. Значки наиболее распространённых ОС.

	Windows 11
	MacOS Ventura
	Ubuntu — ОС на базе ядра Linux
	Android
	iOS, версия 17 — для смартфонов марки Apple

2.1. Основные виды операционных систем

Операционная система (ОС) — это комплекс программ, позволяющий пользователю общаться с компьютером, управлять устройствами компьютера, программами и информацией, хранящейся в памяти компьютера.

Компьютеры работают под управлением различных операционных систем. Иногда на одном

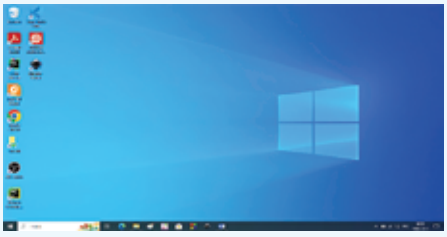
компьютере даже устанавливают несколько ОС. Наиболее распространённые ОС для настольных компьютеров и ноутбуков: Windows, MacOS, Linux. Операционные системы Android и iOS устанавливаются на смартфоны и планшеты (примеры 2.1 и 2.2).

При включении компьютера операционная система начинает автоматически загружаться с диска в оперативную память и остаётся в рабочем состоянии до выключения компьютера. Загрузка ОС из долговременной памяти в оперативную называется **загрузкой компьютера.**

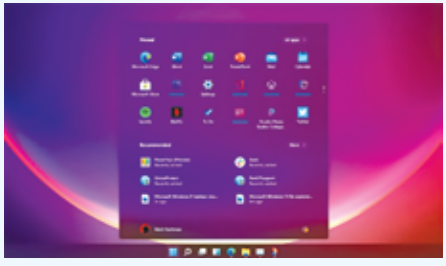
ОС выводит на экран монитора приглашение к работе в какой-либо форме. В ответ пользователь даёт команду на выполнение конкретного действия. Если такая команда знакома ОС и в данный момент времени команда может быть выполнена, то ОС её выполняет, если нет — пользователю выдаётся соответствующее сообщение. После этого ОС ожидает следующую команду пользователя. Такой режим работы называется **диалоговым режимом.**

У современных ОС диалоговый режим графический.

Пример 2.2. Интерфейс различных ОС.



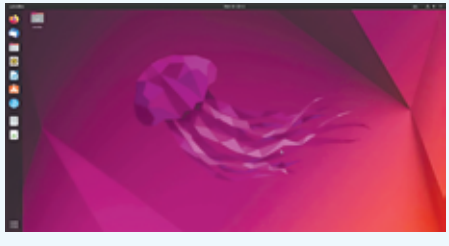
Windows 10



Windows 11



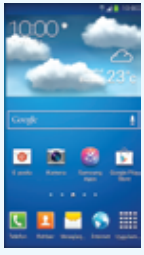
MacOS Ventura



Ubuntu «Jammy Jellyfish»
(Везучая медуза)

Правообладатель Адукацыя і выхаванне

Пример 2.2. *Продолжение.*



Android для смартфона



iOS для планшета

Пример 2.3. Значки **Рабочего стола.**

	Этот компьютер
	Корзина
	Сеть
	Проводник

В **графическом режиме** пользователь может давать команды ОС, выбирая их из различных меню. При такой организации диалога командам ОС соответствуют определённые значки — графические объекты.

Выбор команд часто осуществляют с помощью мыши. Так, например, организован диалог пользователя с операционными системами семейства Windows.

ОС позволяет пользователю работать с другими программами. Она находит программу в долговременной памяти, загружает её в оперативную память и передаёт управление процессору для выполнения команд данной программы. В оперативную память может быть загружено несколько программ. Такой режим работы ОС называется **многозадачным.** ОС также организует и контролирует работу всех устройств компьютера.

2.2. Элементы графического пользовательского интерфейса

Пользовательский интерфейс — это совокупность средств и способов взаимодействия человека и компьютера.

Современные ОС имеют графический пользовательский интерфейс, т. е. взаимодействие пользователя с компьютером осуществляется с помощью различных графических объектов. Основными элементами графического интерфейса являются **окна** и **меню.**

Рассмотрим пользовательский интерфейс ОС Windows 11. После загрузки ОС на экране компьютера появляется **Рабочий стол** с ярлыками и значками (пример 2.3).

Рабочий стол — основное окно графического интерфейса ОС, занимающее всё пространство экрана.

Внизу Рабочего стола находится **Панель задач.** На ней расположена кнопка **Пуск**, кнопки программ и открытых файлов, регулятор громкости и различные индикаторы (пример 2.4). Кнопка **Пуск** вызывает **Главное меню**, с помощью выбора пунктов которого пользователь может получить доступ ко многим программам, установленным на компьютере.

Ярлык представляет собой ссылку на объект (файл, папку) и всегда содержит стрелку . Двойной щелчок левой кнопкой мыши запускает программу, на

Пример 2.4. Элементы **Панели задач.**

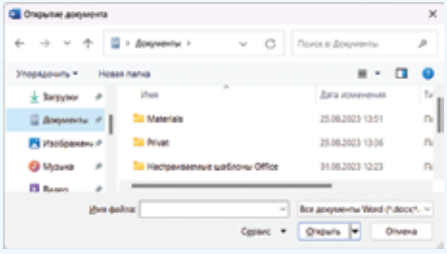
	Кнопка **Пуск**
	Значки программ и открытых файлов
10:50 18.12.2026	Индикатор даты и времени
	Индикаторы сетевых подключений
	Регулятор громкости звука
ENG	Индикатор языка

Пример 2.5. Значки программ и соответствующих документов.

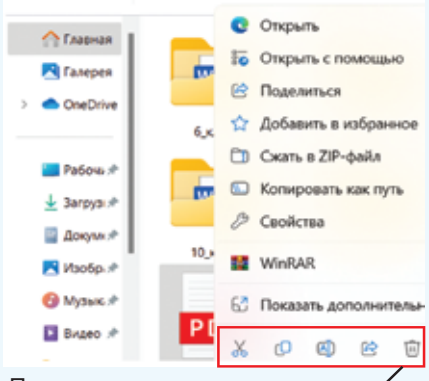
Значок программы	Значок документа
Текстовый редактор Блокнот	
	
PowerPoint	
	
Текстовый редактор Word	
	

Правообладатель Адукацыя і выхаванне

Пример 2.6. Диалоговое окно «Открытие документа».



Пример 2.7. Контекстное меню файла.



Панель задач контекстного меню

Пример 2.8. Клавиша вызова контекстного меню на клавиатуре.



В настоящее время одно из основных направлений в развитии операционных систем — интеграция с искусственным интеллектом.

которую ссылается ярлык. **Значок** без стрелки используется для обозначения самого объекта — папки или файла (вид значков некоторых программ и документов представлен в примере 2.5 на с. 17).

В процессе работы на Рабочем столе могут быть раскрыты диалоговые окна, окна папок, окна программ и др.

Диалоговые окна предназначены для организации диалога пользователя с компьютером (пример 2.6).

Окно программы появляется после запуска программы.

Меню — список команд, которые пользователь может выбрать тем или иным способом для выполнения какого-либо действия.

Доступ ко всем командам, возможным для данного объекта, можно получить с помощью **контекстного меню** (пример 2.7), которое можно вызвать одним из способов:

- щелчком правой кнопки мыши на объекте;
- специальной клавишей на клавиатуре (пример 2.8);
- сочетанием клавиш: Shift + + F10.

2.3. Основные элементы файловой системы

Нужная нам информация хранится в компьютере в виде файлов. Работа с ними производится с помощью **файловой системы.**

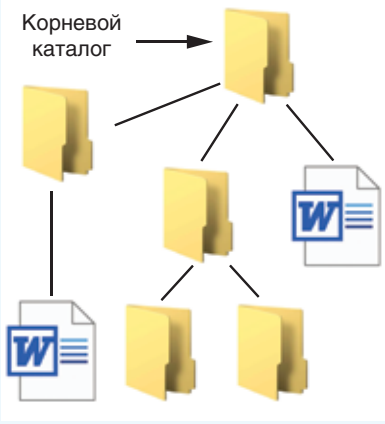
Файловая система предназначена для организации хранения файлов и папок (каталогов) и выполнения операций над ними.

Каждая ОС поддерживает определённые файловые системы. Объектами любой файловой системы являются файлы, папки и диски.

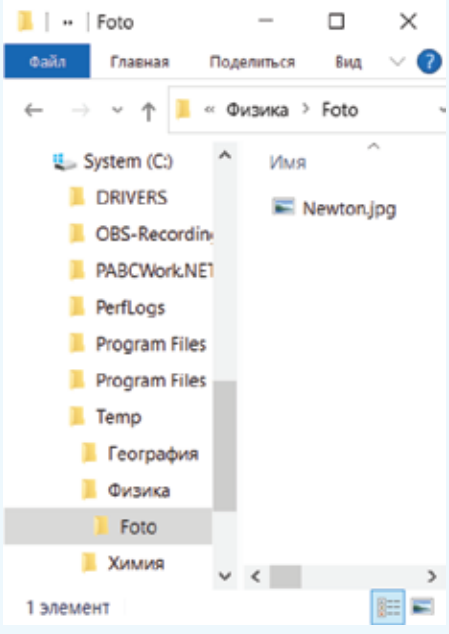
Структура файловой системы Windows представляет собой систему вложенных папок (пример 2.9). В каждой папке могут храниться другие папки и файлы. О папках или файлах, находящихся в другой папке, говорят, что они вложены в эту папку. Структуры, построенные на принципах вложенности (подчинения), называются **иерархическими.** Файловая система ОС Windows является иерархической.

Файловая система позволяет создавать, переименовывать и удалять

Пример 2.9. Структура файловой системы. Её можно представить как дерево с перевёрнутой кроной.



Пример 2.10. Определение пути к файлу `Newton.jpg`.



Правообладатель Адукацыя і выхаванне

Пример 2.10. *Продолжение.*

При переходе из папки `Foto` в папку `Физика` происходит подъём на один уровень иерархии. При обратном перемещении — спуск. Полное имя файла `C:\Temp\Физика\Foto\Newton.jpg` означает, что:

1. Файл `Newton.jpg` вложен в папку `Foto`.
2. Папка `Foto` вложена в папку `Физика`.
3. Папка `Физика` вложена в папку `Temp`, которая находится на диске `C`.

Путь к файлу следующий: `C:\Temp\Физика\Foto\`. Отображение пути к файлу `Newton.jpg` в адресной строке:



Пример 2.11. Работа с каталогом.

Семиклассник Вадим работал с каталогом `E:\Matem\Form\1_ch\`. Сначала он поднялся на один уровень вверх, затем ещё раз вверх, потом опустился в каталог `Prog`, в котором находился файл `z1.py`. Каков путь к этому файлу?

Вадим работал с каталогом `E:\Matem\Form\1_ch\`. Поднявшись на один уровень вверх, он оказался в каталоге `E:\Matem\Form\`. Поднявшись ещё раз вверх, он оказался в `E:\Matem\`. После, опустившись в каталог `Prog`, Вадим оказался в `E:\Matem\Prog\`. Это и есть путь к файлу `z1.py`.

файлы, переносить и копировать файлы с одного носителя на другой, выполнять поиск файлов, хранящихся на разных носителях.

Одни папки создаёт пользователь, другие, такие как **Этот компьютер** или **Корзина**, создаются автоматически при установке операционной системы. Для того чтобы найти файл в файловой структуре, нужно указать путь к файлу.

Путь к файлу — последовательность папок, начиная от самой верхней и заканчивая той, в которой непосредственно хранится файл. Путь к файлу вместе с именем файла называют **полным именем файла.**

Путь начинается с **корневой папки** (имени диска) и содержит последовательность имён папок, в которые вложен файл. Диски именуются заглавными буквами английского алфавита с двоеточием после буквы. Имена дисков начинаются с C:. После имени каждой папки ставится обратный слэш (примеры 2.10 и 2.11).

Для работы с файлами и папками используют программы, которые называют **файловыми менеджерами.**

Для каждой операционной системы созданы разные файловые менеджеры. Для ОС Windows это Проводник, Total Commander, Far Manager и др. (пример 2.12).

2.4. Типовые операции с файлами и папками

Рассмотрим работу с файловым менеджером **Проводник**. Способы его запуска представлены в примере 2.13, а способы изменения отображения информации в окне Проводника — в примере 2.14 на с. 22.

Проводник позволяет пользователю создавать и удалять объекты файловой структуры, копировать и переносить их с одного носителя на другой, а также переименовывать их. Действия по копированию, переносу и удалению файловых объектов аналогичны действиям по копированию, переносу и удалению текстовых или графических фрагментов.

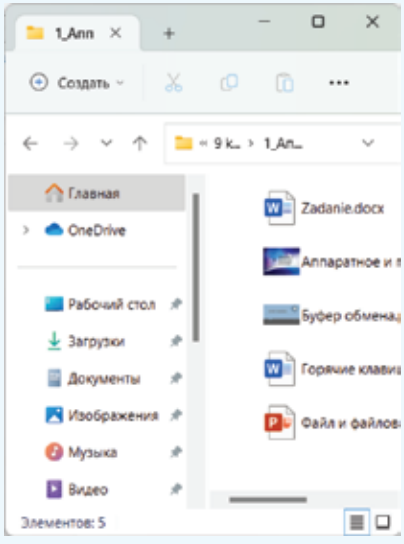
Перед выполнением операций с файловыми объектами их нужно выделить. Если объекты расположены рядом, то их выделяют так: щёлкают мышью на первом объекте и, удерживая клавишу Shift, щёлкают на последнем. Если выделяемые объекты не расположены рядом,

Пример 2.12. Значки файловых менеджеров для ОС Windows.

	Проводник
	Total Commander
	Far Manager

Пример 2.13. Способы запуска программы Проводник.

- Ярлык программы на Рабочем столе.
- Кнопка **Пуск (Служебные → → Проводник)**.
- Контекстное меню кнопки **Пуск**.

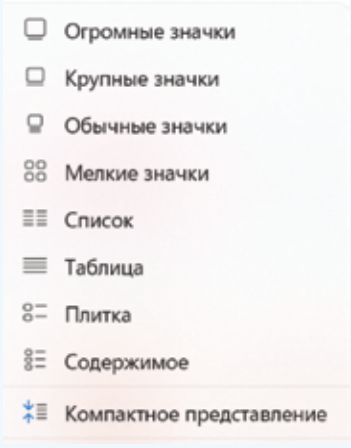


Окно файлового менеджера Проводник

Правообладатель Адукацыя і выхаванне

Пример 2.14. Способы изменения отображения информации в окне Проводника.

1. Выполнить щелчок по кнопке Просмотреть на панели инструментов и выбрать способ отображения из развернувшегося меню:



2. Прокрутить колесо мыши.

Пример 2.15. «Горячие клавиши».

Выделить всё	Ctrl + A
Копировать	Ctrl + C
Вырезать	Ctrl + X
Вставить	Ctrl + V
Переименовать	F2
Удалить без перемещения в Корзину	Shift + Delete
Создать папку	Ctrl+Shift+N
Открыть Проводник	Win + E

то их выделяют, удерживая клавишу Ctrl.

Результат копирования — это два одинаковых файловых объекта. Чтобы получить копию, нужно в папке-источнике выделить объект для копирования и в контекстном меню объекта выбрать команду **Правка → Копировать.** После этого следует зайти в папку-приёмник и в её контекстном меню выбрать команду **Правка →** **→ Вставить.** В качестве источника и приёмника может использоваться одна и та же папка.

Действия по перемещению файла (папки) аналогичны действиям по копированию. Для перемещения объекта из папки-источника в папку-приёмник используются команды **Вырезать** и **Вставить.**

Удалить выделенные файловые объекты можно нажав клавишу Delete или выполнив команду контекстного меню **Удалить.** После удаления объекты обычно помещаются в Корзину. Корзина предназначена для временного хранения удалённых объектов, которые можно восстановить после удаления. Удалённые из Корзины объекты восстановить с помощью операционной системы невозможно.

Для выполнения действий с файловыми объектами можно использовать функциональные клавиши и сочетания клавиш — «горячие клавиши». Это ускоряет выполнение действий. Таких сочетаний клавиш очень много, но достаточно запомнить только некоторые из них (пример 2.15).

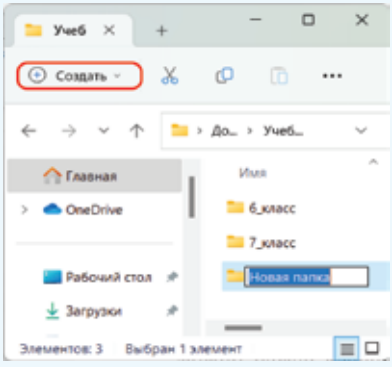
Чтобы создать в Проводнике новую папку, нужно выбрать на панели инструментов окна Проводника кнопку **Создать**, а затем в развернувшемся меню — пункт Папку. В рабочей области Проводника появится папка с именем «Новая папка» (пример 2.16). Это имя можно поменять на иное.

Чтобы переименовать файл или папку, нужно щёлкнуть по кнопке на панели инструментов контекстного меню файлового объекта или нажать клавишу F2.

Файловый менеджер Проводник встроен в Windows и является одним из самых используемых. Но кроме него существуют и другие файловые менеджеры, которые обладают расширенным функционалом. Например, файловый менеджер Total Commander.

Процесс создания новой папки в файловом менеджере Total Commander показан в примере 2.17.

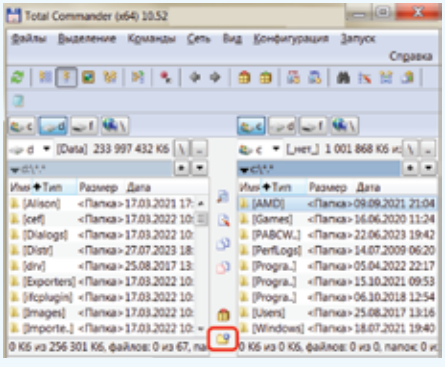
Пример 2.16. Создание папки в окне файлового менеджера Проводник.



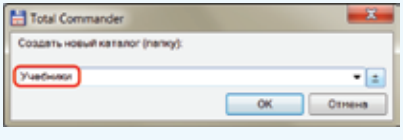
Пример 2.17. Создание папки в Total Commander.

1. Открыть папку, внутри которой будет находиться новая папка.

2. Нажать кнопку **Создать каталог** или клавишу F7.



3. В открывшемся окне **Создать новый каталог (папку)** ввести имя новой папки.



Правообладатель Адукацыя і выхаванне

1. Что такое операционная система?
2. Какие основные функции выполняет ОС?
3. Что представляют собой значки и ярлыки?
4. Для чего используется кнопка **Пуск**?
5. Что такое файловая система?
6. По какому принципу построена структура файловой системы?
7. Какие операции позволяет выполнять файловая система?
8. Какие программы называют файловыми менеджерами?

Упражнения

1 Учитель работал в папке `E:\Уроки\7 класс\Задания\`. Потом перешёл на уровень выше, вошёл в папку `Тема 4` и удалил из неё файл `Обобщение.docx`. Каково полное имя файла, удалённого учителем?

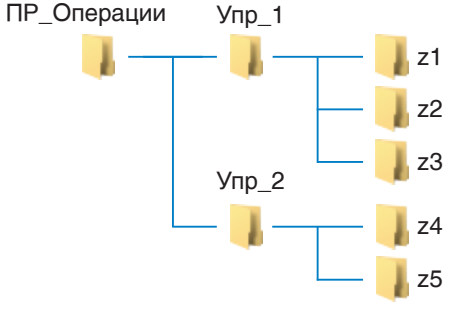
2 Измените фон Рабочего стола. Для этого:

1. В контекстном меню Рабочего стола выберите **Персонализация.**
2. В открывшемся окне выберите одну из тем.

3 Используя кнопку **Пуск**, загрузите следующие программы из подменю Стандартные: Блокнот, Калькулятор, Paint, Word. Расположите окна программ каскадом, стопкой, рядом, используя соответствующие команды контекстного меню Панели задач. Покажите учителю свою работу, после чего закройте все окна.

4 Разнесите значки Рабочего стола по разным углам экрана. Выровняйте значки. Используя команды контекстного меню Рабочего стола, отсортируйте значки по типу, дате, размеру. После каждой сортировки сделайте скриншоты экрана (с помощью клавиши PrScr или SysRq).

5 Создайте дерево папок со следующей структурой:



1. В папке z1 создайте файл — документ Word с именем text1.docx, в котором будет содержаться ответ на вопрос 8 этого параграфа.



2. В графическом редакторе Paint создайте изображение смайла. Сохраните его в папке z2 под именем ris1.png.

3. Скопируйте папку Упр_1 в папку z5. Переименуйте папку z5 в папку с именем Copy. Скопированные папки z1, z2, z3 переименуйте в Copy1, Copy2, Copy3 соответственно. Удалите папку Copy3.

§ 3. Локальная компьютерная сеть

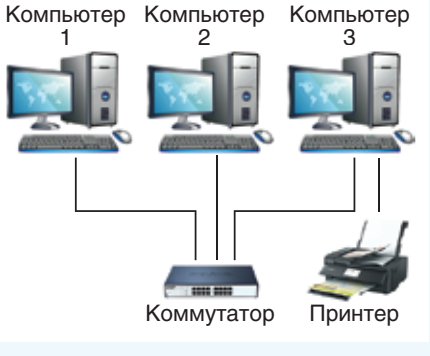
Желание передавать информацию от одного компьютера к другому, обеспечивать пользователям совместный доступ к техническим устройствам, программам и информационным ресурсам компьютеров вызвало необходимость объединения компьютеров в единую сеть.

Компьютерная сеть — это объединение компьютеров, обеспечивающее совместное использование сетевых ресурсов.

Компьютеры, расположенные на небольших расстояниях друг от друга, могут быть объединены в **локальную сеть.** Это, как правило, сеть одной организации, фирмы, предприятия, учебного заведения и др. (пример 3.1).

Пример 3.1. Схема локальной сети.

Примером локальной компьютерной сети является сеть в кабинете информатики. Она существует для того, чтобы учащиеся могли работать с одними и теми же информационными ресурсами и использовать общий принтер. В единую сеть несколько компьютеров соединяются через сетевой коммутатор (свитч, роутер).



Правообладатель Адукацыя і выхаванне

Пример 3.2. Домашняя локальная сеть с беспроводным подключением.



Пример 3.3. Аппаратное обеспечение локальной сети.



Внешняя сетевая карта

Сетевые карты предназначены для приёма и передачи данных в сети. Они могут быть встроены в конструкцию материнской платы.



Сетевая USB-карта для обеспечения беспроводного подключения



Сетевой коммутатор



Сетевой кабель

По способу организации локальные компьютерные сети делятся на **одноранговые** и **с выделенным сервером.**

В одноранговых сетях все компьютеры равноправны. Сеть с выделенным сервером имеет один высокопроизводительный компьютер, управляющий работой этой сети. Этот компьютер называется **сервером.** Он предоставляет свои ресурсы для совместного использования остальным компьютерам сети, называемым **клиентами,** и может управлять их работой.

По способу подключения компьютерные сети могут быть **проводными** и **беспроводными** (пример 3.2).

Для организации работы компьютеров, объединённых в локальную сеть, необходимо соответствующее аппаратное (пример 3.3) и программное обеспечение.

Программную поддержку работы компьютеров в локальной сети выполняет операционная система.

Сетевыми ресурсами (ресурсами сети) компьютеров могут являться:

1) технические устройства (модемы, принтеры, дисководы и др.);

2) программное обеспечение (операционная система, различные редакторы и др.);

3) информационные ресурсы (файлы с данными).

Для доступа к сетевым ресурсам часто бывает нужно указать имя пользователя и его пароль.

Пользователь, на компьютере которого находится ресурс (файл, диск, папка или устройство), является его владельцем и имеет полный доступ к этому ресурсу. Владелец ресурса может разрешить другим пользователям сети доступ к своему диску, папке, файлу, устройству.

В Windows 11 сетевые ресурсы не отображаются в сетевом окружении проводника (папка Сеть) изначально. Доступ к сетевым ресурсам может быть получен только после предварительных настроек.

Важнейшей характеристикой работы локальной сети является скорость передачи данных в ней — количество информации, передаваемое за единицу времени. Скорость передачи данных по сети обычно измеряется в битах в секунду (бит/с). Рассмотрите решение задачи в примере 3.4.

Общепринятых правил поведения пользователей в локальной сети не существует. Тем не менее отметим некоторые общие требования:

- не передавайте другим пользователям ваше имя и пароль для входа в сеть;
- для сохранения информации используйте личное дисковое пространство, а не диски общего пользования.

Пример 3.4. Определите объём файла компьютерной презентации, если передача его по сети длилась 5 с при скорости 1 024 000 бит/с. Запишите полученный результат в килобайтах.

Решение

$1\ 024\ 000$ бит/с $\cdot$ 5 с $=$

$= (2^{10} \cdot 10^3 \cdot 5)$ бит $=$

$= 2^{10} \cdot (2^3 \cdot 5^4)$ бит $= 2^{13} \cdot 5^4$ бит.

Переведём биты в килобайты:

1 байт = 8 бит, или 2^3 бит;

1 Кбайт = 1024 байт, или 2^{10} байт;

1 Кбайт $= 2^3 \cdot 2^{10} = 2^{13}$ бит.

Тогда: $(2^{13} \cdot 5^4) / 2^{13} = 5^4 =$

$= 625$ Кбайт.

Беспроводные локальные сети (WLAN), которые не требуют физических кабелей для соединения устройств, появились в двухтысячных годах.

Правообладатель Адукацыя і выхаванне

1. Для чего компьютеры объединяют в сети?
2. Какая сеть называется локальной?
3. С помощью чего компьютеры объединяются в локальную сеть?
4. Какие бывают локальные сети?
5. Назовите сетевые ресурсы.

Упражнения

1 Определите объём видеофайла, если передача его по сети длилась 1 мин 20 с при скорости 80–1020 бит/с. Запишите полученный результат в мегабайтах.

2 Определите объём звукового файла, если передача его по сети длилась 0,5 с при скорости 155 Мбит/с. Запишите полученный результат в байтах.

3 Определите скорость передачи данных по сети, если архивный файл объёмом 1,5 Гбайт передавался 2 мин.

4 Определите скорость передачи данных по сети, если файл с компьютерной игрой объёмом 3,2 Гбайт передавался 3 мин 45 с.

§ 4. Архивация

В 40-х гг. XX в. учёные, работавшие в области информационных технологий, пришли к выводу, что можно разработать такой способ хранения данных, при котором компьютерная память будет расходоваться более экономно. Одними из первых алгоритмов по сжатию данных являются алгоритмы Шеннона — Фано и Хаффмана.

Клод Шеннон является основателем теории информации. Он разработал большинство базовых понятий теории сжатия информации.

4.1. Программы-архиваторы

Для рационального хранения данных на компьютерных носителях используются программы-архиваторы, позволяющие представить данные в упакованном виде. Архивы создают в следующих случаях:

- необходимо создать резервные копии наиболее важных файлов;
- требуется освободить место на диске;
- следует передать файлы по электронной почте;

- планируется перенести большое количество файлов на другой носитель;
- нужно защитить данные от несанкционированного доступа.

При помощи архивации мы можем уменьшить объём данных в несколько раз. Но значительное уменьшение возможно не всегда. Если попробовать заархивировать музыку, видео, фотографии, то размер практически не изменится. Такой тип данных сжимается другим способом.

Упаковывать файлы и размещать их в специальных архивах позволяют **программы-архиваторы. Архивный файл (архив)** хранит в упакованном виде другие файлы (один или несколько), которые при необходимости могут быть извлечены из архива в первоначальной форме. Основные функции программ-архиваторов:

- помещение исходных файлов в архив;
- извлечение файлов из архива;
- удаление файлов из архива;
- просмотр оглавления архива;
- проверка архива.

Для преобразования данных программы-архиваторы используют разные алгоритмы, поэтому размеры архивов, содержащих

Роберт Фано независимо от Клода Шеннона изобрёл алгоритм сжатия информации.

Дэвид Хаффман разработал метод построения минимально-избыточных кодов.

Большие архивы данных хранят в специальных хранилищах информации — дата-центрах. Одним из крупнейших хранилищ данных Беларуси является Республиканский ЦОД (центр обработки данных), созданный компанией beCloud.



Сегодня алгоритмы сжатия данных основываются на базе искусственного интеллекта (ИИ). Они могут автоматически выявлять шаблоны и избыточность в данных и сжимать их, минимизируя размер архива и сохраняя высокую производительность и качество.



Правообладатель Адукацыя і выхаванне

Первые программы-архиваторы появились в 1980-х гг. В 1985 г. был создан архиватор ARC для операционной системы MS-DOS. Он умел не только сжимать файлы, но и объединять их в пакеты (как, собственно, это работает и сейчас).

Архиватор WinRAR был создан в 1995 г. российским программистом Евгением Рошалем. Правообладателем архиватора WinZip является компания Corel Corporation.

7-Zip — бесплатный архиватор с открытым исходным кодом, был создан в 1999 г. Автором архиватора 7-Zip является российский программист Игорь Павлов.

Пример 4.1. Значки программ-архиваторов.

	WinRar
	7-Zip
	WinZip
	Bandizip
	HaoZip

одни и те же файлы, но созданных с помощью различных архиваторов, могут отличаться. Информация в архиве хранится в закодированном виде, поэтому для просмотра и работы с содержимым архивного файла нужно воспользоваться программой-архиватором. Чтобы работать с файлом, его следует извлечь из архива. Делают это, используя ту же программу, с помощью которой создавался архив, или другую программу, распознающую данный тип архива.

4.2. Создание архивов и извлечение файлов из архива

Сегодня существует большое количество программ-архиваторов: WinRar, 7-Zip, WinZip и др. Архивные файлы имеют расширения, соответствующие программам, с помощью которых их создавали: .rar, .7z, .zip. При просмотре списка файлов в Проводнике архивные файлы помечаются соответствующими значками (пример 4.1).

В операционной системе Windows 11 есть встроенный архиватор, с помощью которого можно создать архивную ZIP-папку. Но

пока его возможности уступают другим программам-архиваторам.

После установки программ-архиваторов их значки и наименование добавляются в контекстное меню любого объекта файловой системы (пример 4.2).

Алгоритм создания архива рассмотрен в примере 4.3.

Архив с предложенным именем создаётся в текущей папке. Если выбрана команда **Добавить в архив**, то пользователю нужно задать имя архива и указать папку, в которой он будет храниться. Кроме того, WinRAR предоставляет возможность отправить архивный файл по электронной почте.

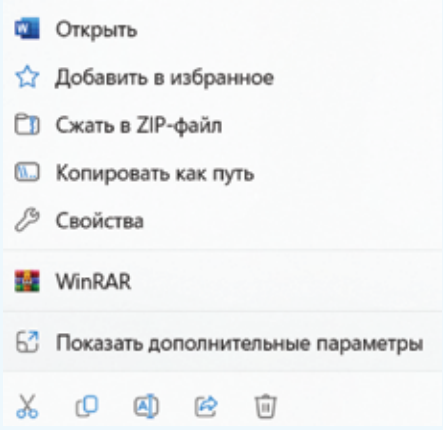
Для извлечения файлов из архива нужно:

1. В контекстном меню архивного файла выбрать WinRAR.
2. В развернувшемся меню выбрать одну из команд:
   - Открыть в WinRAR.
   - Извлечь файлы... .
   - Извлечь в (имя папки\).
   - Извлечь в текущую папку.

Рассмотрите пример 4.4.

После выбора команды **Открыть в WinRAR** открывается окно архиватора, в котором можно не только просмотреть содержимое архива, но и выполнить

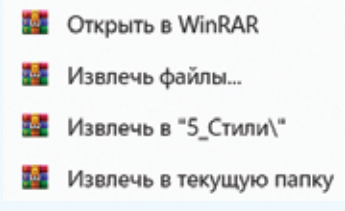
Пример 4.2. Контекстное меню с выбором архиватора.



Пример 4.3. Создание архива с помощью архиватора WinRAR.

1. Выделить файлы, предназначенные для архивации.
2. В контекстном меню выделенных файлов выбрать WinRAR.
3. Выбрать одну из команд:
   - Добавить в архив.
   - Добавить в архив с предложенным именем.

Пример 4.4. Команды меню архиватора WinRAR по извлечению файлов из архива.



Правообладатель Адукацыя і выхаванне

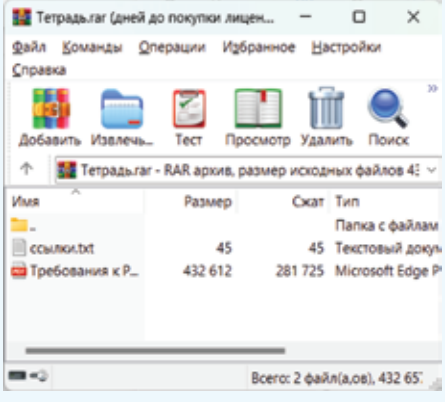
Пример 4.4. *Продолжение.*

Для извлечения файлов из ZIP-папки нужно в контекстном меню архива выбрать Извлечь все... . После этого откроется окно **Распаковка сжатых ZIP-папок**, в котором можно:

1. Для извлечения файлов в текущую папку нажать кнопку Извлечь .

2. Для извлечения файлов в другую папку нажать кнопку Обзор... и в открывшемся окне выбрать папку, куда будут помещены файлы из архива.

Пример 4.5. Окно архиватора WinRAR.



другие действия с архивом (пример 4.5).

При выборе команды **Извлечь файлы...** пользователь должен указать имя папки, в которую будут извлечены файлы. При выборе **Извлечь в** (с указанием имени папки) файлы будут извлечены в папку, которая будет расположена в текущей папке. При выборе команды **Извлечь в текущую папку** файлы из архива будут помещены в ту же папку, в которой находится архив.

Для настройки конфиденциальности при создании архива в WinRAR нужно нажать кнопку **Установить пароль**, ввести символы пароля, повторить их для подтверждения и после нажать ОК. Будет создан зашифрованный архив. Кроме того, чтобы защитить документы, можно установить галочку в пункте **Шифровать имена файлов и папок**. В этом случае названия файлов и папок нельзя будет просмотреть.

1. Для чего нужно создавать архивы?
2. Какой файл называют архивным?
3. Для чего предназначены программы-архиваторы?
4. Какие программы-архиваторы вам известны?
5. Как заархивировать файл(-ы)?
6. Как извлечь файл(-ы) из архива?

Упражнения

1 Создайте рисунок в графическом редакторе Paint. Сохраните файл на диске. Заархивируйте этот файл. Сравните размеры исходного и архивного файлов.

2 Заархивируйте графический файл разными архиваторами. Сравните размеры полученных архивов.

3 Заархивируйте файлы разных типов: рисунки, тексты, программы. Сравните размеры исходных файлов и их архивов разных типов. Какие файлы сжимаются лучше?

4 Извлеките файлы из архива, указанного учителем, в свою папку.

§ 5. Программное обеспечение

5.1. Классификация программного обеспечения

Программное обеспечение (ПО) — совокупность всех компьютерных программ.

Компьютер рассматривают как единую систему, состоящую из аппаратного обеспечения, программного обеспечения и информационных ресурсов. ПО компьютера постоянно изменяется, совершенствуется, дополняется.

Программное обеспечение компьютера по назначению бывает:

1) системным;
2) прикладным;
3) инструментальным.

Системное ПО — программы для обеспечения работы

Пример 5.1. Прикладные программы.

Редакторы позволяют обрабатывать информацию, представленную в текстовой, графической, звуковой, числовой форме. Например:

	Текстовый редактор Word
	Графический редактор Paint
	Графический редактор Inkscape

Мультимедийные программы сочетают возможность работы с видеофрагментами, звуком, анимацией, статическими картинками и гипертекстом.

Правообладатель Адукацыя і выхаванне

Пример 5.1. *Продолжение.*

Например:

	Программа для создания презентаций PowerPoint

Коммуникационные программы предназначены для поддержки пользовательского интерфейса при работе в Сети. Например, браузеры:

	Microsoft Edge
	Firefox
	Chrome

Наряду с онлайн-сервисами, такими как Google, Yandex и др., которые могут переводить информацию с одного естественного языка на другой, существуют программы-переводчики. Например:

	Программа-словарь для перевода отдельных слов ABBYY Lingvo
	Программа для перевода текстов Promt

компьютера и компьютерных сетей. Системное ПО позволяет пользователю осуществлять руководство и контроль над работой компьютера и компьютерной сети, а также обеспечивает возможность выполнения других программ.

К системному программному обеспечению относятся уже знакомые вам операционные системы, файловые менеджеры, архиваторы.

Прикладное ПО — программы для решения задач определённого класса предметной области. Прикладное ПО самое многочисленное (пример 5.1 на с. 33). Для наименования прикладных программ часто используют термин **приложения**. К прикладным программам относятся:

- программы общего назначения (требуются практически каждому пользователю);
- программы специального назначения (предназначены для профессионального использования в различных сферах деятельности);
- компьютерные игры.

Инструментальное ПО предназначено для создания другого ПО (пример 5.2). С инструментальным ПО работают программисты.

5.2. Вредоносные программы и способы защиты от них

Вредоносные программы — специально написанные программы, способные нанести ущерб информации, хранящейся на компьютере, или вывести компьютер из строя.

По способу распространения вредоносные программы делятся на компьютерные вирусы, сетевые черви и троянские программы.

Компьютерные вирусы могут распространяться самостоятельно, добавляя свой код к другим файлам.

Сетевые черви не изменяют файлы на дисках, а распространяются в компьютерной сети, проникают в операционную систему компьютера, находят адреса других компьютеров или пользователей и рассылают по этим адресам свои копии.

Троянские программы — это вредоносные программы, которые сами не распространяются, а, маскируясь под популярную программу, побуждают пользователя переписать и установить вредителя на свой компьютер самостоятельно.

Программы по их правовому статусу можно разделить на следующие группы:

- платные;
- свободно распространяемые;
- условно-бесплатные;
- пробные (оценочные);
- демонстрационные.

Примеры свободно распространяемого ПО: ОС Linux, графический редактор Inkscape, антивирусные программы AVAST и AVG, среда программирования IDLE.

Пример 5.2. Инструментальное ПО.



Среда IDLE для языка Python

Прообразом современных вирусов можно считать программу «Дарвин», появившуюся ещё в 1962 г. Именно тогда инженеры из американской компании создали игру под этим названием. Смысл игры заключался в удалении всех копий программы противника и захвате поля битвы. Программы-вирусы, являющиеся проблемой для пользователей, возникли двадцать с лишним лет спустя.

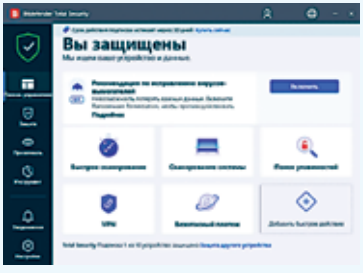
Регулярное архивирование и резервное копирование файлов позволяют минимизировать ущерб от вирусной атаки.

Правообладатель Адукацыя і выхаванне

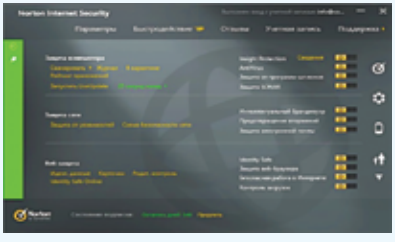
Пример 5.3. Интерфейсы популярных антивирусных программ.



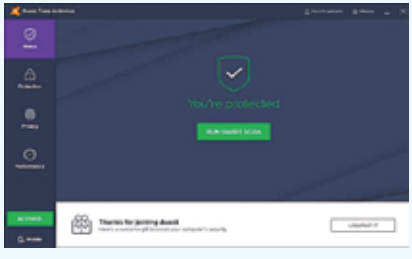
Kaspersky Endpoint Security для Windows 11



Bitdefender Total Security



Norton Internet Security



Avast Free Antivirus

Большинство вирусов разрабатывается для причинения вреда пользователям, работающим с операционными системами семейства Windows.

При заражении компьютера вирусом очень важно его обнаружить.

Признаки заражения:

1) медленная работа компьютера;

2) зависания и сбои в работе компьютера;

3) изменение размеров файлов;

4) уменьшение размера свободной оперативной памяти;

5) значительное увеличение количества файлов на диске;

6) исчезновение файлов и папок или искажение их содержимого.

Для борьбы с вредоносными программами используют программные средства антивирусной защиты: Kaspersky, Bitdefender, Norton, Avast, Avira и др. (примеры 5.3 и 5.4).

Сканирование компьютера в поисках вредоносных программ обычно выполняется автоматически при каждом включении. При сканировании антивирусная программа ищет вирус путём сравнения кода программ с кодами из-

вестных ей вирусов, хранящихся в базе данных. Одним из основных способов борьбы с вредоносными программами является профилактика (предотвращение заражения). Чтобы предотвратить заражение компьютера, необходимо соблюдать следующие рекомендации:

1. Установите антивирусную программу и регулярно пользуйтесь ею для проверки компьютеров.

2. Не запускайте программы, полученные из Интернета, без проверки на наличие в них вируса.

3. Проверяйте все внешние носители на наличие вирусов, прежде чем копировать или открывать содержащиеся на них файлы.

Тенденции в разработке современных антивирусных программ — использование нейросетевых технологий и алгоритмов машинного обучения для нейтрализации вредоносного ПО. В такие антивирусные программы встроены алгоритмы, позволяющие искусственному интеллекту (ИИ) обнаруживать вредоносное ПО. Нейросеть анализирует поведение программ на компьютерном устройстве и определяет, есть ли на нём вирусы.

Пример 5.4. Значки популярных антивирусных программ.

	Kaspersky
	Bitdefender
	Norton Internet Security
	Avast Free Antivirus

Самым разрушительным вирусом за всю историю их существования считают вирус ILOVEYOU. Он был разослан на почтовые ящики с Филиппин в 2000 г. В теме письма содержалась строка «ILoveYou», а к письму был приложен скрипт «LOVE-LETTER-FOR-YOU.TXT.vbs». При открытии вложения вирус рассылал копию самого себя всем контактам в адресной книге Windows, а также на адрес, указанный как адрес отправителя. Вирус поразил более 3 млн компьютеров по всему миру. Ущерб, который вирус нанёс мировой экономике, был настолько велик, что вирус вошёл в Книгу рекордов Гиннесса.

Правообладатель Адукацыя і выхаванне

1. Что такое программное обеспечение?
2. На какие классы можно разделить программное обеспечение в зависимости от назначения?
3. Какие программы называются вредоносными?
4. Какие виды вредоносных программ существуют?
5. Какие признаки указывают на то, что компьютер заражён?
6. Что необходимо делать, чтобы предотвратить заражение компьютера?
7. Назовите программы антивирусной защиты.

Упражнения

1 Перечислите названия известных вам программ.

1. Платные.
2. Свободно распространяемые.

2 Запишите в тетради названия антивирусных программ, которые установлены у вас дома, в школьном компьютерном кабинете, у ваших друзей.

3 Определите, к какому классу программного обеспечения относятся программы, значки которых представлены на рисунках.

а) 	б) 	в) 	г) 
д) 	е) 	ж) 	з) 
и) 	к) 	л) 	м) 

Глава 2
ПРЕДСТАВЛЕНИЕ О ЛОГИКЕ ВЫСКАЗЫВАНИЙ. МНОЖЕСТВА И ОПЕРАЦИИ НАД НИМИ

§ 6. Логика высказываний

Возможности компьютера велики. Он может помочь врачу поставить правильный диагноз пациенту, пассажиру — выбрать билет на нужный поезд. Компьютер управляет автомобилем, составляет прогнозы погоды и умеет делать многое другое.

Для того чтобы выяснить, как «рассуждает» компьютер, сначала нужно попытаться понять, как думает человек. Ведь именно человек создал компьютер, и компьютер выполняет только те действия, которым его научил человек. Изучение логики высказываний поможет понять, как «рассуждает» компьютер.

Наши знания об окружающем мире мы выражаем в повествовательных предложениях. Такие предложения могут отражать действительность верно или неверно. Думая, человек строит свои рассуждения, основываясь на собственных знаниях.

Ещё Аристотель заметил, что правильность рассуждений не

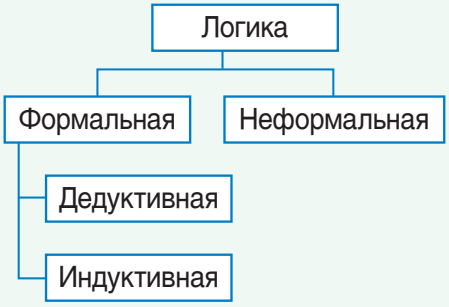


Древнегреческий философ Аристотель (384–322 гг. до н. э.) первым систематизировал формы и правила мышления, разработал теорию умозаключений и доказательств, описал логические операции. Аристотелю принадлежат формулировки основных законов мышления.

У истоков современной логики стоял немецкий математик Готфрид Вильгельм Лейбниц (1646–1716 гг.). Учёный предложил идею представления логических рассуждений как вычислений, подобных вычислениям в математике.

Английский математик Джордж Буль (1815–1864 гг.) перенёс на логику законы и правила математических (алгебраических) действий, создав тем самым алгебру логики.

Правообладатель Адукацыя і выхаванне



Формальная логика использует строгие правила и математические конструкции, применяется в точных науках.

Неформальная логика используется в естественном языке и в дискуссиях и риторике, анализирует повседневные рассуждения.

В **дедуктивных** рассуждениях из истинных посылок следует истинное заключение, в **индуктивных** рассуждениях даже при верных посылках вывод может оказаться как истинным, так и ложным.

Пример 6.1. Предложения, являющиеся высказываниями.

1. Атом водорода самый лёгкий (истинно).

2. Клетка — центральная часть атома (ложно).

3. Кирилл Туровский — известный английский писатель и оратор, живший во второй половине XII в. (ложно).

4. Частное какого-то числа и 1 равно этому числу (истинно).

зависит от содержания, а определяется формой.

Наука, изучающая формы рассуждений, называется **формальной логикой.**

Математическая логика использует математические методы для исследования способов построения рассуждений, доказательств, выводов.

Одним из разделов современной математической логики является **логика высказываний.**

На правилах математической логики построены процессы «рассуждений» компьютера. Изучение логики высказываний поможет понять, как можно научить компьютер «думать».

6.1. Понятие высказывания

Высказывание — повествовательное предложение (утверждение), о котором в настоящее время можно сказать, истинно оно или ложно (пример 6.1).

Об истинности высказывания можно говорить только в настоящем времени: высказывание «Идёт дождь» может быть истинным сейчас и ложным через час.

Как правило, высказывания обозначают заглавными латинскими буквами. Если высказывание A истинно, обычно пишут $A = 1$, если ложно — $A = 0$ (пример 6.2). Часто используют такие обозначения: A = true (истина) или A = false (ложь).

6.2. Логическая операция НЕ

С высказываниями можно производить различные операции, подобно тому, как в математике — с числами (сложение, умножение, вычитание и др.).

Логическая операция **НЕ** (отрицание) меняет значение высказывания на противоположное: истинно на ложно, а ложно на истинно.

Логическое отрицание получается из высказывания добавлением частицы «не» к сказуемому или использованием оборота «неверно, что...» (пример 6.3). Иногда при построении отрицаний некоторые слова заменяют их антонимами, если это возможно.

Если высказывание содержит слова «все», «всякий», «любой», то отрицание такого высказывания строится с использованием слов «некоторые», «хотя бы один» (пример 6.4). И наоборот,

Пример 6.2. Обозначение истинности высказывания.

$A = a^0$ равно 1, если $a \neq 0$;

B = Масса измеряется в литрах.

Для приведённого примера $A = 1$, $B = 0$.

Пример 6.3. Построение отрицания высказываний.

Высказывания:

1. У цветковых растений развивается плод.
2. Фреска — это живопись водяными красками по свежей штукатурке.

Отрицание высказываний:

1. У цветковых растений **не** развивается плод.
2. **Неверно**, что фреска — это живопись водяными красками по свежей штукатурке.

Пример 6.4. Построение отрицания высказываний.

Высказывания:

1. Все учащиеся занимаются спортом.
2. Некоторые птицы умеют плавать.
3. Любой цветок имеет запах.
4. Иногда у мамы бывает плохое настроение.

Отрицание высказываний:

1. **Некоторые** учащиеся **не** занимаются спортом.
2. **Все** птицы **не** умеют плавать.
3. **Хотя бы один** цветок не имеет запаха.
4. У мамы **всегда** бывает **хорошее** настроение.

Правообладатель Адукацыя і выхаванне

Пример 6.5. Определение истинности высказываний с отрицаниями.

1. Ель — это дерево (истинное высказывание). Ель — это не дерево (ложное высказывание).

$A = 1$, **НЕ** $A = 0$.

2. Число -7 является положительным числом (ложное высказывание). Число -7 не является положительным числом (истинное высказывание).

$A = 0$, **НЕ** $A = 1$.

3. Все вещества — металлы (ложное высказывание). Некоторые вещества — неметаллы (истинное высказывание).

$A = 0$, **НЕ** $A = 1$.

4. Все составляющие воздуха являются газами (истинное высказывание). Некоторые составляющие воздуха не являются газами (ложное высказывание).

$A = 1$, **НЕ** $A = 0$.

5. Длительность суток не зависит от скорости вращения планеты (ложное высказывание). Длительность суток зависит от скорости вращения планеты (истинное высказывание).

$A = 0$, **НЕ** $A = 1$.

6. Дома на левой стороне улицы имеют чётные номера (ложное высказывание). Неверно, что дома на левой стороне улицы имеют чётные номера (истинное высказывание).

$A = 0$, **НЕ** $A = 1$.

для высказываний со словами «некоторые», «хотя бы один» отрицание будет содержать слова «все», «всякий», «любой».

Любую операцию над числами в математике обозначают каким-либо знаком: «+», «−», «·», «:». Для логических операций также определены свои обозначения. Если операцию отрицания применяют к высказыванию A, то это можно записать так: **НЕ** A. Можно встретить и другие обозначения для логической операции отрицания: **Not** A, $\neg A$, $\overline{A}$, $\sim A$.

Истинность высказывания **НЕ** A, независимо от содержания высказывания, можно определить по **таблице истинности:**

A	**НЕ** A
1	0
0	1

Из таблицы истинности следует, что отрицанием истинного высказывания будет ложное, а отрицанием ложного — истинное (пример 6.5). Высказывание и его отрицание никогда не могут быть истинными или ложными одновременно.

Например, отрицанием высказывания «У меня есть компью-

тер» будет высказывание «У меня нет компьютера» (или высказывание «Неверно, что у меня есть компьютер»). Истинность этих высказываний зависит от конкретного человека. Для одних будет истинным первое высказывание, а для других — второе. Но эти два высказывания не могут быть истинными или ложными одновременно для одного и того же человека.

Часто трудно установить истинность высказывания. Высказывание «Площадь озера Нарочь 79,6 км^2» в одной ситуации можно посчитать ложным, а в другой — истинным. Указанное значение не является постоянным. Поэтому рассматривать его как истинное можно только при некотором приближении, приемлемом на практике.

1. Что такое высказывание?
2. Какие значения могут иметь высказывания?
3. Что делает логическая операция *отрицание*?
4. Как построить отрицание высказывания?

Упражнения

1 Определите, какие из предложений являются высказываниями.

1. Включи монитор.
2. Кислород — это газ.
3. Шишка — это цветок.
4. Сколько воды утекло?
5. Все дети — учащиеся.
6. Хотя бы один пароль будет верным.

2 Постройте отрицания высказываний.

1. Миша не может пойти в кино.
2. Соня любит рисовать.
3. Все планеты не имеют атмосферы.
4. В сентябре не бывает дождей.
5. Солнце светит ярко.
6. Некоторые птицы улетают на юг.

Правообладатель Адукацыя і выхаванне

3 Откройте файл с предложениями и отредактируйте их, удалив или вставив частицу «не» так, чтобы предложения стали истинными высказываниями.

Озеро Нарочь не является крупнейшим озером Беларуси.
Все горы являются вулканами.
Дуб — хвойное дерево.
Число 27 является простым числом.
Термометр не позволяет определить температуру тела.
Число 2028 не делится на 3.
Треугольник не является геометрической фигурой.

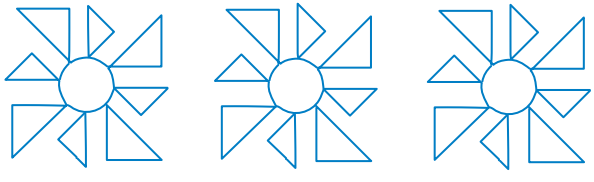
4 Определите, какие утверждения о животных, представленных на рисунках, истинные, а какие — ложные.





1. Некоторые животные умеют лазить по деревьям.
2. Все животные обитают в лесах.
3. Ни одно из животных не является домашним.
4. Каждое животное можно погладить.
5. Все люди любят мышей.
6. Ни одно из животных не умеет плавать.

5 Откройте файл с рисунком трёх цветков. Раскрасьте их так, чтобы каждое из следующих высказываний было истинным.



1. Все цветки имеют белый круг в середине.
2. На рисунке есть цветок с синими лепестками.
3. На рисунке нет цветка с красными лепестками.

4. Неверно, что цвет круга в середине цветка совпадает с цветом лепестков.

5. Хотя бы у одного цветка лепестки разных цветов.

6* Решите задачу-стихотворение.

Собаки с рыжими хвостами
Себе овсянку варят сами.
Тем, чьи хвосты стального цвета,
Не позволяют делать это.
Кто варит сам себе овсянку,
Гулять выходит спозаранку.
Все, кто гулять выходят рано,
Не терпят фальши и обмана.
Вид добродушный у Барбоса,
Но на сорок он смотрит косо.
Он видит: норовят сороки
У воробьёв списать уроки!
Скажите — проще нет вопроса! —
Какого цвета хвост Барбоса?[1]

§ 7. Логические операции И и ИЛИ

Логика высказываний позволяет строить **составные высказывания.** Они создаются из нескольких простых высказываний путём соединения их друг с другом с помощью логических операций **НЕ**, **И**, **ИЛИ** и др.

7.1. Логическая операция И

Определение истинности или ложности составного высказывания зависит от того, являются ли истинными или ложными простые высказывания, входящие

В 1936–1938 гг. Клод Шеннон, инженер и математик, нашёл применение булевой логике при конструировании схем из реле и переключателей.

[1] Разговоров Н. В. Собаки с рыжими хвостами... [Электронный ресурс] // Ушедшие поэты. URL: http://po.m-necropol.ru/razgovorov-nikita.html (дата обращения: 19.09.2025).

Правообладатель Адукацыя і выхаванне

В дальнейшем это открытие послужило основанием для построения логических элементов, на которых работает компьютерная техника. Состояние элементов компьютера соответствует логическим значениям:

- 0 — сигнал отсутствует;
- 1 — сигнал присутствует.

Пример 7.1. Анализ высказывания «Число 456 трёхзначное и чётное».

Данное высказывание является составным, поскольку оно содержит два простых высказывания:

A = Число 456 трёхзначное.

B = Число 456 чётное.

Высказывания A и B соединены логической операцией **И**, поэтому получилось составное высказывание A **И** B. Высказывание A — истинно, высказывание B — истинно. Поэтому высказывание A **И** B — истинно: $(A \text{ И } B) = 1$.

Пример 7.2. Анализ высказывания «Геракл — герой древнерусской мифологии и сын бога Зевса».

Высказывание A = Геракл — герой древнерусской мифологии. Ложно, $A = 0$.

Высказывание B = Геракл — сын бога Зевса. Истинно, $B = 1$.



в его состав, а также от той логической операции, которая их связывает.

Составное высказывание A **И** B, образованное в результате объединения двух простых высказываний A и B логической операцией **И**, истинно тогда и только тогда, когда A и B одновременно истинны.

Если хотя бы одно из простых высказываний, связанных операцией **И**, будет ложным, то и составное высказывание будет ложным. В примерах 7.1 и 7.2 показано, как определять истинность составного высказывания, содержащего логическую операцию **И**.

Представим таблицу истинности для логической операции **И**:

A	B	A **И** B
1	1	1
0	1	0
1	0	0
0	0	0

Операцию **И** называют **логическим умножением.** Равенства $1 \cdot 1 = 1$, $1 \cdot 0 = 0$, $0 \cdot 1 = 0$, $0 \cdot 0 = 0$ верные как для обычного, так и для логического умножения.

Для записи логической операции **И** используют следующие обозначения: A **И** B, A **AND** B, $A \cdot B$, $A * B$, $A \wedge B$, A & B.

Пример 7.2. *Продолжение.*

Высказывание A **И** B = Геракл — герой древнерусской мифологии **И** сын бога Зевса ложно: (A **И** B) = 0.

7.2. Логическая операция ИЛИ

Составное высказывание A **ИЛИ** B, образованное в результате объединения двух простых высказываний A и B логической операцией **ИЛИ**, ложно тогда и только тогда, когда A и B одновременно ложны.

Если хотя бы одно из простых высказываний, связанных операцией **ИЛИ**, будет истинным, то и составное высказывание будет истинным (примеры 7.3 и 7.4).

Таблица истинности для логической операции **ИЛИ**:

A	B	A **ИЛИ** B
1	1	1
0	1	1
1	0	1
0	0	0

Запись логической операции **ИЛИ** можно встретить в следующих вариантах: A **ИЛИ** B, A **OR** B, $A + B$, $A \vee B$, $A \mid B$.

Если в логическом выражении присутствует несколько логических

Пример 7.3. Анализ высказывания «Семиклассники изучают философию или астрономию».

Данное составное высказывание образовано из двух простых:

A = Семиклассники изучают философию.

B = Семиклассники изучают астрономию.

Высказывания связаны логической операцией **ИЛИ**. В результате получили составное высказывание A **ИЛИ** B. Высказывание A ложно, высказывание B ложно. Поэтому высказывание A **ИЛИ** B ложно: (A **ИЛИ** B) = 0.

Пример 7.4. Анализ высказывания «Франциск Скорина — белорусский первопечатник или Стефан Баторий — турецкий султан».

A = Франциск Скорина — белорусский первопечатник. Истинно, A = 1.

B = Стефан Баторий — турецкий султан. Ложно, B = 0.

Составное высказывание будет истинным: (A **ИЛИ** B) = 1.

Правообладатель Адукацыя і выхаванне

Пример 7.5*. Определение значения логического выражения *A* **ИЛИ** *B* **И НЕ** *C*.

Распишем по действиям вычисление значения выражения:

1) *D* = **НЕ** *C*;

2) *E* = *B* **И** *D*;

3) *F* = *A* **ИЛИ** *E*.

Значение высказывания *F*, полученное в 3-м действии, определит значение исходного логического выражения.

Пример 7.6*. Определение значения логического выражения *A* **ИЛИ** *B* **И НЕ** *C* при *A* = 1, *B* = 0, *C* = 0.

Распишем по действиям вычисление значения выражения:

1. *D* = **НЕ** *C* = 1;

2. *E* = *B* **И** *D* = 0 **И** 1 = 0;

3. *F* = *A* **ИЛИ** *E* = 1 **ИЛИ** 0 = 1.

Результат: значение логического выражения *A* **ИЛИ** *B* **И НЕ** *C* истинно.

Пример 7.7. Определение значения составного высказывания.

Даны простые высказывания:

A = Прогноз погоды обещает дожди.

B = Сейчас на улице идёт дождь.

Высказывание *A* **И** *B* будет ложным, если мы увидим, что на улице нет дождя (независимо от того, что обещал прогноз погоды).

Высказывание *A* **ИЛИ** *B* будет истинным, если прогноз погоды обещал дождь (независимо от того, какую погоду мы наблюдаем сейчас).

операций, то важно определить порядок их выполнения. Наивысшим приоритетом обладает операция **НЕ**. Логическая операция **И** — логическое умножение — выполняется раньше операции **ИЛИ** — логического сложения (примеры 7.5* и 7.6*).

Для изменения порядка выполнения логических операций используют скобки: в этом случае сначала выполняются операции в скобках, а затем все остальные.

Логические операции **И** и **ИЛИ** подчиняются переместительному закону:

A **И** *B* = *B* **И** *A*;

A **ИЛИ** *B* = *B* **ИЛИ** *A*.

Часто для того чтобы определить значение составного логического выражения, достаточно знать значение только одного простого высказывания. Так, если в составном высказывании с операцией **И** значение хотя бы одного простого высказывания будет ложным, то значение составного высказывания будет ложным. Если в составном высказывании с операцией **ИЛИ** хотя бы одно простое высказывание будет истинным, то значение составного высказывания будет истинным (пример 7.7).

1. Какие высказывания являются составными?
2. Как называются известные вам логические операции?
3. В каких случаях составное высказывание *A* **И** *B* может быть истинным?
4. В каких случаях составное высказывание *A* **ИЛИ** *B* может быть ложным?

Упражнения

1 Определите, истинными или ложными являются составные высказывания.

1. Мяч круглый **ИЛИ** Земля плоская.
2. Кролики — домашние животные **И** баобаб растёт в Беловежской пуще.
3. Клавиатура — устройство ввода информации **ИЛИ** мышь — устройство вывода информации.
4. И. А. Крылов написал басню «Квартет» **И** М. Ю. Лермонтов написал стихотворение «Парус».
5. Сосна — хвойное дерево **И** кедр — не хвойное дерево.
6. Монитор — устройство ввода информации **ИЛИ** сканер — не устройство вывода информации.
7*. Континенты **И** острова — это большие участки суши.

2 Подготовьте к каждому из высказываний рисунки, учитывая, что все высказывания о том, как прошли летние каникулы, являются истинными.

1. Я была у бабушки в деревне, и рядом с деревней находилось озеро.
2. По озеру плавала лодка или утка.
3. Мы с бабушкой насобирали малины и смородины.
4. Я составила букет из цветов. В нём были ромашки или гвоздики.

Правообладатель Адукацыя і выхаванне

3 Откройте файл с рисунком и разложите грибы по корзинкам так, чтобы было истинным высказывание «В большой корзине все грибы съедобные **И** в маленькой корзине все грибы несъедобные».



4* Найдите значения логических выражений, если $A = 1$, $B = 1$, $C = 0$, $D = 0$.

1. A **ИЛИ** B **И НЕ** C.
2. A **И НЕ** B **ИЛИ** C.
3. A **ИЛИ** B **И НЕ** (C **И** D).
4. (A **И** B) **ИЛИ НЕ** C **И** (A **ИЛИ** B) **ИЛИ НЕ** D.

§ 8. Использование условий для исполнителей

8.1. Понятие условия

Пример 8.1. Условия в правилах дорожного движения: если горит зелёный свет, можно переходить улицу.



Принятие решений зачастую зависит от различных условий. Если на улице идёт дождь, то нужно взять зонт; если хорошо подготовился к уроку, то получишь высокую отметку, иначе низкую и др. Рассмотрите примеры 8.1–8.3, показывающие использование условий в жизни человека.

Человек способен понимать условия, сформулированные в произвольной форме. Но для того чтобы другой исполнитель мог принимать решения, нужно «на-

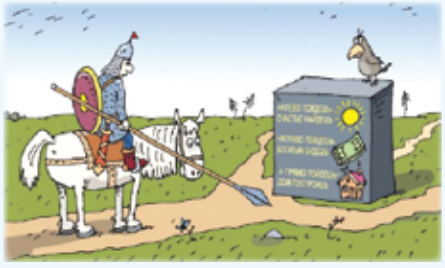
учить» его «понимать» условия. Условия для исполнителя формулируются на формальном языке, понятном данному исполнителю.

Условием для исполнителя является понятное исполнителю высказывание, которое может быть истинным (соблюдаться) либо ложным (не соблюдаться).

Условия могут быть **простыми** и **составными. Простым** условием называется выражение, которое включает в себя одно высказывание. **Составное условие** — условие, которое включает в себя несколько простых высказываний, связанных между собой логическими операциями.

В примере 8.4 приведены простые условия, в примере 8.5 — составные условия. Условия под одинаковыми номерами относятся к одному исполнителю. Исполнителем для первого условия может быть текстовый редактор. Второе условие может быть понятно роботу, двигающемуся по лабиринту, или компьютерному исполнителю *Робот*, средой обитания которого является

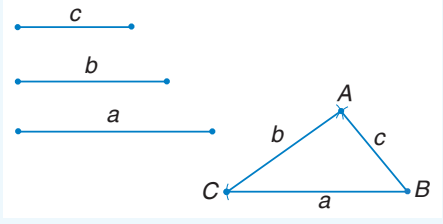
Пример 8.2. Условия в фольклоре: выбор пути сказочными героями.



Пример 8.3. Применение условия в математике.

Треугольник существует, если для большей стороны a выполняется неравенство:

$$a < b + c.$$



Пример 8.4. Простые условия.

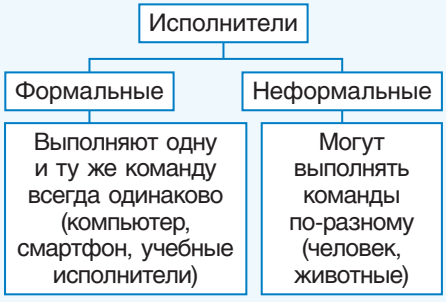
1. Символ скопирован.
2. Клетка закрашена.
3. $a > b$.

Пример 8.5. Составные условия.

1. (Символ скопирован) **ИЛИ** (Символ вырезан).
2. (Клетка закрашена) **И НЕ** (Справа стена).
3. **not** $(a > b)$ **and** $(a > c)$.

Правообладатель Адукацыя і выхаванне

Пример 8.6. Характеристики формальных и неформальных исполнителей.



Пример 8.7. Алгоритм для исполнителя *Редактор*.

1. Вычисли длину цепочки.
2. Если длина цепочки символов нечётна, то допиши в конце 0, иначе цепочка не изменяется.
3. Поменяй символы местами.
4. Допиши в конце 1.

Пусть исходная цепочка символов — ABC. Длина цепочки — 3.

1. Условие истинно, поэтому преобразовываем цепочку символов: ABC0.
2. Попарно меняем символы местами: BA0C.
3. Дописываем справа цифру 1: BA0C1.

Рассмотрим другую цепочку символов: ABCD, длина цепочки — 4.

1. Условие ложно, поэтому цепочку не преобразовываем.
2. Попарно меняем символы местами: BADC.
3. Дописываем справа цифру 1: BADC1.

клеточное поле. Исполнителем для третьего условия может быть человек, понимающий математику и английский язык, или компьютер, выполняющий программу на языке Python.

Использование условий в алгоритмах позволяет исполнителю выполнять различные действия в зависимости от истинности или ложности условия. Составлять условия проще для формальных исполнителей (пример 8.6), которые, в отличие от человека, выполняют действия всегда одинаково. Такие исполнители имеют ограниченный набор выполняемых команд и умеют проверять несколько условий.

Рассмотрите алгоритм с выбором действий в зависимости от значения условия (пример 8.7).

Исполнитель *Редактор* может выполнять следующие команды:

- вычисли длину цепочки символов;
- допиши символы в конце цепочки;
- поменяй местами соседние символы (первый — со вторым, третий — с четвёртым, пятый — с шестым и т. д.);

• проверь истинность условия «длина цепочки символов нечётная».

От значения условия зависит, будет повторяться выполнение действий или нет: действия будут выполняться пока условие истинно (ложно).

В примере 8.8 приведён алгоритм с повторением действий, пока истинно условие.

Исполнитель *Вычислитель* может выполнять следующие команды:

• раздели натуральное число нацело на 10;

• получи остаток от деления числа на 10;

• выдай ответ: запись цифр исходного числа в обратном порядке;

• проверь истинность условия «число не равно 0».

8.2. Условия для компьютерных исполнителей

Многие из вас неоднократно имели дело с различными компьютерными играми. Для достижения цели игры необходимо управлять компьютерным объектом в его среде обитания. Подавая отдельные команды объекту-исполнителю, вы предписываете

Пример 8.8. Алгоритм для исполнителя *Вычислитель*.

1. Если число не равно нулю, перейди к шагу 2, иначе перейди к шагу 5.
2. Раздели число на 10.
3. Получи остаток от деления на 10.
4. Перейди к шагу 1.
5. Запиши ответ.

Работа алгоритма для числа 1024

№	Условие	Число	Остаток
0		1024	
1	Истинно	102	4
2	Истинно	10	2
3	Истинно	1	0
4	Ложно	0	1

Ответ: 4201.

Алгоритм работал, пока условие было истинным. Условие стало ложным после того, как команды алгоритма (1–4) выполнились четыре раза.

Работа алгоритма для числа 0

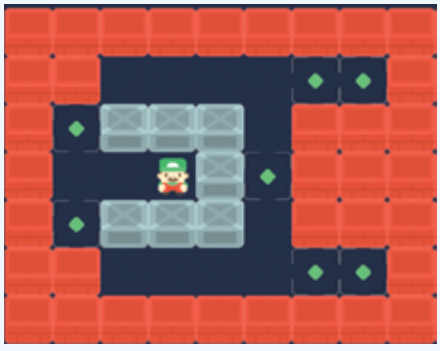
№	Условие	Число	Остаток
0		0	
1	Ложно	0	1

Ответ: 0.

Алгоритм прекратил работу сразу после проверки условия, поскольку оно являлось ложным. Команды 2–4 не выполнились ни разу.

Правообладатель Адукацыя і выхаванне

Пример 8.9. Игра «Сокобан»[1].



Исполнитель *Кладовщик* должен расставить ящики на места, помеченные точками. Исполнитель может сдвинуть только один ящик и не может двигаться сквозь стены. В системе условий исполнителя *Кладовщик* могут быть высказывания: «впереди один ящик», «впереди есть стена».

Пример 8.10. Исполнитель *Черепаха*.

В систему условий, истинность которых может проверять *Черепаха*, входят следующие:

Условие	Значение
`isdown()`	Условие истинно, если перо *Черепахи* опущено
`filling()`	Условие истинно, если была установлена заливка `begin_fill()`
`isvisible()`	Условие истинно, если *Черепаха* невидима

ему выполнить то или иное действие. Некоторые действия могли выполняться только при определённых условиях, т. е. исполнитель должен был уметь проверять условия (пример 8.9).

Компьютерные исполнители и среды их обитания располагаются в памяти компьютера и поэтому являются виртуальными. Прообразами компьютерных исполнителей часто являются реальные объекты, поэтому системы команд и системы условий компьютерных исполнителей могут быть близкими и понятными для человека.

Многие компьютерные исполнители используются для учебных целей. В прошлом году вы познакомились с компьютерным исполнителем *Черепаха*. Условия для этого исполнителя приведены в примере 8.10. Но есть ещё и другие учебные компьютерные исполнители: *Чертёжник*, *Робот*, *Водолей*, *Кузнечик*, *Кенгурёнок* и др. Условия могут входить в систему команд исполнителя как логические высказывания. Выполняя такую команду, исполни-

[1] https://yandex.ru/games/app/223433.

тель проверяет, истинно условие или ложно (пример 8.11).

Все компьютерные исполнители являются формальными исполнителями. Они всегда выполняют одну и ту же команду одинаково и не вникают в суть отдаваемых команд. За действия формального исполнителя всегда отвечает тот, кто им управляет. Формальные исполнители характеризуются:

- кругом решаемых задач;
- средой исполнителя;
- системой команд;
- режимами работы (непосредственный и программный).

В общем случае любые алгоритмы должны содержать предписания двух видов:

1) **функциональные** — предписания, направленные непосредственно на преобразование информации;

2) **логические** — предписания, определяющие дальнейшее направление действий.

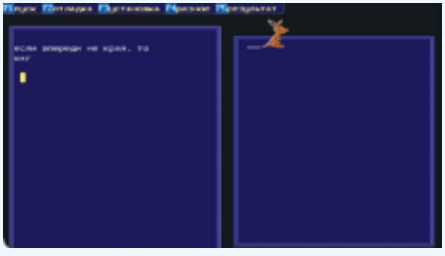
Это утверждение, разработанное выдающимися учёными современности в рамках общей теории алгоритмов, было доказано А. А. Марковым (младшим), являющимся создателем теории нормальных алгоритмов.

Также для *Черепахи* возможна проверка условия нахождения в начале координат:

```
position() == home()
```

Условие можно прочитать так: текущая позиция *Черепахи* совпадает с её «домом» — началом координат.

Пример 8.11. Исполнитель *Кенгурёнок*.



Исполнитель *Кенгурёнок* — простой компьютерный исполнитель, создающий рисунки из линий на прямоугольном поле. Система команд: «шаг», «прыжок», «поворот», «впереди край», «впереди не край». Последние две команды — условия, принимающие значения «истина» или «ложь».

Российский академик Андрей Андреевич Марков (младший) (1903–1979 гг.) — исследователь в области математики и теории алгоритмов.



Правообладатель Адукацыя і выхаванне

1. Что понимают под условием?
2. Какими могут быть условия?
3. Чем составные условия отличаются от простых?
4. Для чего могут использоваться условия в алгоритмах?

Упражнения

1 Наберите предложения в текстовом редакторе и выделите в них условия цветом (жёлтым — простые, зелёным — составные). Определите, все ли высказывания будут истинными.

1. Если ты хочешь покататься на велосипеде или на самокате, то нужно сделать уроки.
2. Я сегодня пойду на тренировку, если не будет дождя.
3. Деньги служат тебе, если умеешь ими пользоваться, если же не умеешь, властвуют над тобой.
4. Я помогу ей с задачей по математике, если она попросит меня об этом и не будет со мной спорить.

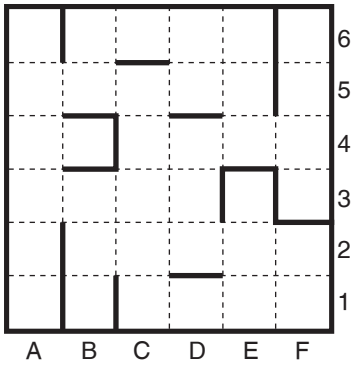
2 Откройте файл. Дополните условиями предложения в файле.

1. Бактерии быстро размножаются и образуют группы (колонии), ...
2. Будешь уважать традиции своих предков и гордиться их поступками и достижениями, ...
3. Вероятность ошибок при письме снижается, ...
4. В корне *жог/жёг* пишется буква *о*, ...

3 Сформулируйте и запишите в тетрадь условия, при которых можно выполнить следующие действия.

1. Начать набор нового абзаца текста в текстовом редакторе.
2. Увидеть результат работы программы для исполнителя *Черепаха*.
3. Определить, какая из двух точек, заданных координатами, находится ближе к центру координат.
4. Определить, есть ли среди цифр натурального трёхзначного числа одинаковые.

4 Исполнитель *Попрыгунчик* обитает в следующей обстановке.



Попрыгунчик может прыгать из той клетки, где он находится, в одну из четырёх соседних, но не может перепрыгивать через стены. Запишите в тетрадь:

1. Номера клеток, в которых может находиться *Попрыгунчик*, чтобы совершить один прыжок в определённом направлении. Например, вверх — A1, B1, ...

2. Условия, при которых *Попрыгунчик* может прыгать в определённом направлении.

3. Номера клеток начальных положений *Попрыгунчика* и условие, при котором *Попрыгунчик*, находясь в этих клетках, сможет совершить три прыжка влево.

5* Поле шахматной доски может быть указано парой натуральных чисел, каждое из которых не превосходит 8: первое число — это номер по вертикали, второе — по горизонтали. Даны натуральные числа k, l, m, n. Запишите простое условие, которое будет истинным тогда и только тогда, когда поля (k, l) и (m, n) являются полями одного цвета.

§ 9. Множества. Операции над множествами

Рассмотрим высказывание «Все учащиеся нашего класса имеют дома компьютер». Истинно оно или ложно? Для ответа на этот вопрос нужно у каждого из своих одноклассников уточнить: «У тебя дома есть компьютер?» Если все учащиеся класса ответят утвердительно, то высказывание истинно, если хотя бы один из учащихся ответит «нет»,

В конце XIX в. немецкий математик Георг Кантор (1845–1918 гг.) определил множество как «единое имя для совокупности всех объектов, обладающих данным свойством».

Множество всех объектов, обладающих свойством $A(x)$, Кантор обозначил $\{x \mid A(x)\}$, а само $A(x)$ назвал характеристическим свойством множества.

Правообладатель Адукацыя і выхаванне

то высказывание будет ложным. Для разных классов это высказывание будет иметь различные значения, потому что различными будут **множества** учащихся класса.

Множество — совокупность каких-либо объектов, обладающих общим свойством. Эти объекты называют **элементами множества.**

С элементами теории множеств вы познакомились в прошлом году на уроках математики. Вспомним основные понятия и обозначения в теории множеств.

Множества обычно обозначают прописными латинскими буквами, а элементы множества — строчными. Если множество M состоит из элементов a, b, c, то его записывают так: $M = \{a, b, c\}$. Чтобы задать конечное множество, можно перечислить его элементы (пример 9.1). В тех случаях, когда множество нельзя задать при помощи списка элементов, его задают путём указания некоторого общего свойства элементов множества (пример 9.2).

Пример 9.1. Задание множества перечислением.

Множество M некоторых школьных предметов, состоящее из элементов *математика*, *информатика*, *английский язык*, можно записать так:

M = {математика, информатика, английский язык};

информатика $\in M$;

литература $\notin M$.

Пример 9.2. Задание множества через общее свойство его элементов.

Пусть множество M — множество натуральных чисел. Тогда:

$8 \in M$;

$-3 \notin M$;

$3{,}14 \notin M$.

Некоторые множества могут быть описаны обоими способами.

Путём перечисления элементов задали следующее множество:

$A = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

Это же множество может быть задано путём указания общего свойства элементов множества: «Множество однозначных натуральных чисел».

Если использовать обозначение, введённое Кантором, то это множество можно описать так:

$A = \{a \mid a \in N,\ 1 \leqslant a < 10\}$.

Для обозначения принадлежности элемента множеству используют специальные знаки: $a \in M$ (элемент a принадлежит множеству M), $a \notin M$ (элемент a не принадлежит множеству M).

Мощность множества — количество элементов данного множества (пример 9.3).

Подмножеством множества M называется любое множество A, каждый элемент которого является элементом множества M. Если A — подмножество множества M, то это записывают так: $A \subset M$. Запись $A \not\subset M$ обозначает, что множество A не является подмножеством множества M.

Пустое множество — множество, не содержащее ни одного элемента (обозначается знаком $\varnothing$).

Для наглядной геометрической иллюстрации множеств и отношений между ними используют **круги Эйлера** (пример 9.4).

Для множеств, как и для высказываний, определены свои операции. Такими операциями являются операции пересечения и объединения множеств (пример 9.5).

Пересечением множеств A и B называется множество, в которое входят только те элементы, которые принадлежат как множеству A,

Пример 9.3. Мощность множества.

Мощность множества $M = \{$математика, информатика, английский язык$\}$ равна трём.

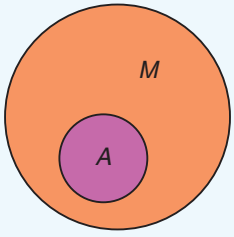
Обозначение: $|M| = 3$.

Мощность пустого множества равна нулю.

Пример 9.4. Круги Эйлера.

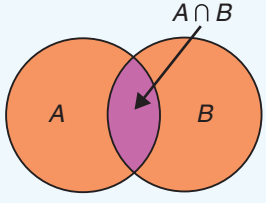
M — множество всех хищников;

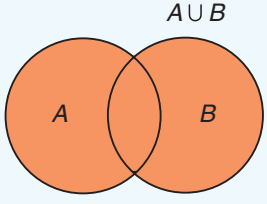
A — множество всех рысей.



$A \subset M$

Пример 9.5. Пересечение и объединение множеств.





Правообладатель Адукацыя і выхаванне

Пример 9.6. Использование множеств при решении задач.

A = {изучающие английский язык}.

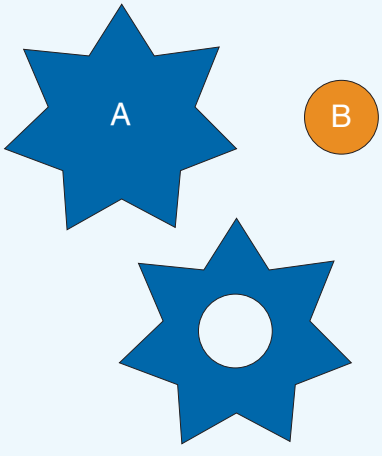
B = {занимающиеся в спортивных секциях}.

Пересечение множеств A и B соответствует множеству учащихся, изучающих английский язык и занимающихся спортом, а объединение — множеству учащихся, которые изучают английский язык или посещают секции.

Тогда: $|A \cup B| = |A| + |B| - |A \cap B| = 20 + 12 - 10 = 22$.

Число учащихся, изучающих китайский язык и не посещающих спортивных секций: $25 - 22 = 3$.

Пример 9.7. Использование множеств в компьютерной графике.



$A - |A \cap B| = A - B$

так и множеству B. Для обозначения операции пересечения используется знак $\cap$.

Объединением множеств A и B называется множество, содержащее в себе все элементы исходных множеств (как множества A, так и множества B). Для обозначения операции объединения множеств используется знак $\cup$.

Элементы теории множеств и математической логики используются при решении математических и логических задач.

Пример 9.6. В классе 25 учащихся. Среди них 20 изучают английский язык, а остальные — китайский. 12 учащихся класса занимаются в спортивных секциях, причём 10 из них изучают английский язык. Сколько учащихся, изучающих китайский язык, не посещает спортивные секции?

Множества могут состоять из самых различных элементов, поэтому теория множеств находит применение в различных областях знаний. В информатике теория множеств и математическая логика помогают создавать более надёжные и эффективные программы и системы.

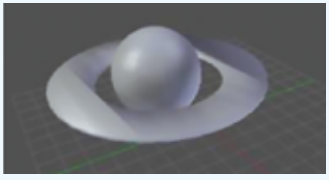
Операции над множествами также используются в компьютер-

ной графике при построении графических объектов (пример 9.7) и в моделировании 3D-объектов (пример 9.8).

Искусственный интеллект использует операции над множествами для классификации объектов: объекты распределяются по классам (множествам), и проводится анализ их пересечений или различий.

Пример 9.8. Использование множеств в 3D-моделировании.

Для получения кольца (множество C) необходимо к множеству A (шар) и множеству B (цилиндр) применить операцию разность[1]. Объединив множества C и A, можно получить планету.



1. Что понимают под множеством?
2. Какими способами можно задать множества? Приведите примеры.
3. Что понимают под подмножеством?
4. Какими геометрическими средствами можно изобразить множества?
5. Что понимают под пустым множеством?
6. Что характеризует мощность множества?
7. Может ли элемент множества одновременно принадлежать разным подмножествам данного множества?
8. Что называют пересечением множеств?
9. Что называют объединением множеств?
10. Как обозначаются операции пересечения и объединения множеств?

Упражнения

1 Дополните каждое из множеств одним-двумя элементами. Определите итоговую мощность множеств.

1. A = {математика, информатика, история, литература}.
2. B = {яблоко, груша, апельсин, слива}.
3. C = {клавиатура, монитор, мышь}.
4. D = {карандаш, ручка, ластик, фломастер}.

[1] Операция разность будет рассмотрена в параграфе 23.

Правообладатель Адукацыя і выхаванне

2 Назовите, какие элементы могут входить в множества.

1. Средства передвижения.
2. Домашние животные.
3. Детские и молодёжные общественные организации Республики Беларусь.
4. Государственные праздники Республики Беларусь.
5. Чётные числа.

3 Откройте файл с группами слов. Разделите слова каждой группы на два множества. Слова первого множества выделите красным цветом, а второго — синим. По каким признакам вы разделили слова?

Образец.

Текст в файле: гусь, лебедь, заяц, волк, павлин, курица, кабан, лось.

Результат: A = {гусь, лебедь, павлин, курица};

B = {заяц, волк, кабан, лось}.

Признаки: A — множество птиц, B — множество зверей.

1. Мяч, стол, стул, коньки, шкаф, клюшка, шайба, комод.
2. Сом, уж, карась, окунь, щука, гадюка, кобра, питон.

4 Выделите подмножества из множества геометрических фигур A = {круг, овал, квадрат, прямоугольник, треугольник, пятиугольник}.

1. Фигуры, не имеющие углов.
2. Фигуры, являющиеся четырёхугольниками.
3. Фигуры, количество углов у которых больше трёх.

5 В графическом редакторе откройте файл с изображениями бабочек. Используя операцию копирования, создайте подмножества и разместите их в прямоугольниках.



1. Бабочки, в окраске которых есть синий цвет.
2. Бабочки, в окраске которых есть красный цвет.
3. Бабочки, в окраске которых есть зелёный цвет.
4. Бабочки, в окраске которых есть жёлтый цвет.

6 Найдите пересечение и объединение множеств A и B. Изобразите их с помощью кругов Эйлера.

1. A = {математика, информатика, история, литература};
   B = {английский язык, математика, химия, история}.
2. A = {яблоко, апельсин, мандарин, лимон, киви};
   B = {апельсин, персик, мандарин, груша, лимон}.

7 Решите задачи с использованием кругов Эйлера (нарисуйте круги в графическом редакторе).

1. Об учащихся школы, которые участвовали в физико-математическом конкурсе, известно следующее: 7 из них решили задачи и по математике, и по физике, 11 — задачи по математике, 9 — задачи по физике. Сколько учащихся принимало участие в конкурсе?

2*. Из 100 туристов, отправившихся в путешествие по Беларуси, Мирский замок посетило 30 человек, Несвижский замок — 28, Лидский замок — 42. Мирский и Несвижский замки посетило 8 человек, Несвижский и Лидский замки — 10, Мирский и Лидский замки — 5, все три замка — 3 человека. Туристы, которые не посетили ни одного замка, посещали Беловежскую пущу. Сколько человек посетило Беловежскую пущу?

8 Откройте в графическом редакторе файл с изображением девочек и выполните задания.

1. Создайте два подмножества множества девочек. Для всех девочек, входящих в первое подмножество, истинно высказывание «Девочка носит брюки синего цвета **И** на её майке есть красный цвет». Для всех девочек, входящих во второе подмножество, истинно высказывание «Девочка одета не в брюки **ИЛИ** имеет волосы жёлтого цвета».
2. Найдите пересечение и объединение этих множеств.
3. Сколько девочек не попало ни в одно подмножество?

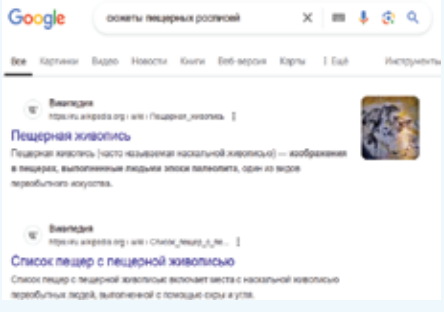
Правообладатель Адукацыя і выхаванне

4. Вокруг девочек из первого множества нарисуйте границу красного цвета, а вокруг девочек из второго множества — зелёного. Область пересечения закрасьте жёлтым цветом.

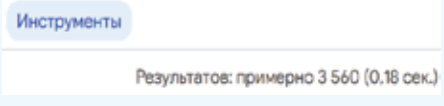


§ 10. Использование логических операций для построения поисковых запросов в Интернете

Пример 10.1. Поиск информации в сети Интернет о сюжетах пещерных росписей.



В результате поиска будет показан список страниц, содержащих искомую информацию. Посмотреть общее количество полученных результатов можно нажав на кнопку «Инструменты».



В современный век информационных технологий практически любой человек использует для поиска информации сервисы сети Интернет — поисковые системы, которые постоянно собирают информацию по всему миру, систематизируют её и сохраняют. Поиск информации в поисковой системе осуществляется по запросу.

Под **запросом** в поисковой системе понимают набор слов, фраз, символов, которые пользователь вводит в строке поиска, чтобы найти интересующую его информацию.

При поиске информации в Интернете важны полнота, точность и актуальность полученных ре-

зультатов. Пользователь может повлиять на качество результатов поиска, если будет:

- продуманно выбирать поисковую службу;
- учитывать особенности поисковой системы;
- грамотно формулировать запросы на поиск информации.

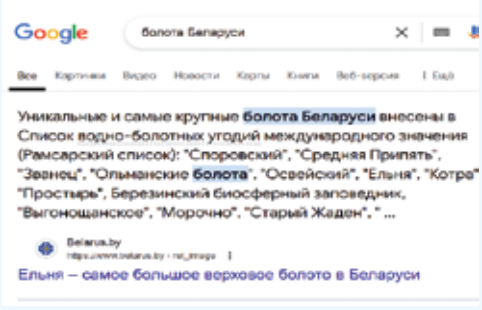
Современные поисковые системы позволяют осуществлять поиск по голосовым запросам или использовать в качестве запроса изображение.

Результатом поиска будет список сайтов (примеры 10.1–10.3). Количество найденных сайтов может быть очень большим, и просмотреть их все часто не представляется возможным. На практике обычно открывают 5–15 сайтов, расположенных в начале списка.

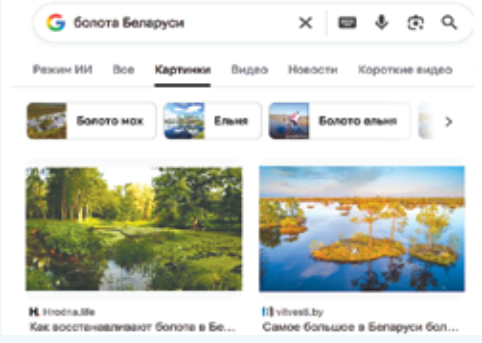
Результативность поиска в значительной степени зависит от умения пользователя корректно сформулировать поисковый запрос. Формулировка фразы или выбор слов для поиска позволит получить более точный результат.

Для каждого из сайтов, найденных в результате поискового запроса, будет истинным следующее высказывание: «На странице сайта присутствует информация,

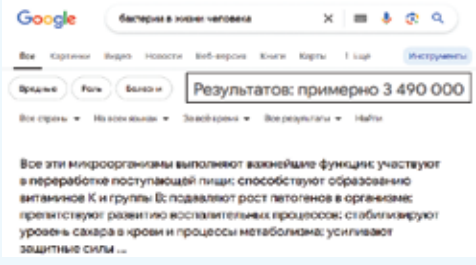
Пример 10.2. Поиск информации в сети Интернет о белорусских болотах.



Можно переключиться на раздел «Картинки».



Пример 10.3. Поиск информации о бактериях в жизни человека.



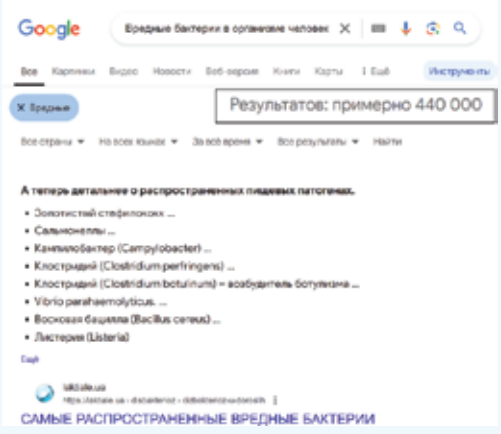
В результате поиска найдено более 3 млн сайтов.

Правообладатель Адукацыя і выхаванне

Пример 10.4. Сокращение количества сайтов, найденных в примере 10.3.

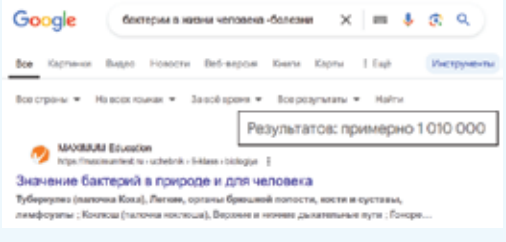
В результате поиска было найдено более 3 млн сайтов.

Если вас интересуют только вредные бактерии, можно нажать на кнопку «Вредные».



Количество найденных результатов сократилось до 440 тыс.

Если вас не интересуют бактерии, вызывающие болезни, то поисковый запрос можно изменить, добавив в конце текста запроса «-болезни». Количество найденных сайтов будет порядка 1 млн.



соответствующая поисковому запросу». Все такие сайты образуют множество сайтов, удовлетворяющих поисковому запросу.

Для многих запросов появляются дополнительные слова, которые во множестве найденных сайтов позволяют искать подмножества. Такие слова появляются на основе анализа большого количества запросов на заданную тему.

При составлении запросов можно использовать **операторы поиска** — слова или символы, добавляемые к поисковым запросам для уточнения результатов.

Для исключения некоторых сайтов из рассмотрения к запросу добавляется слово с оператором «-» перед ним. Сайты, содержащие слова, помеченные «-», не будут включены в список найденных (пример 10.4).

Полученный перечень сайтов образует подмножество множества сайтов, удовлетворяющих основному запросу. Для всех таких сайтов высказывание «На странице сайта присутствует информация, соответствующая слову, помеченному оператором “-”» будет восприниматься поисковой системой как ложное.

Оператор «+» позволяет осуществлять поиск документов,

в которых обязательно присутствует слово, стоящее за символом «+». Допустимо использовать несколько операторов «+» в одном запросе (пример 10.5).

Оператор «*» заменяет любое неизвестное слово в запросе (пример 10.6).

Если поместить слово или фразу в кавычки, то в результатах поиска будут показаны только страницы с этими словами в том же порядке, в котором они указаны в кавычках. Кавычки используются тогда, когда необходимо найти точное слово или фразу, цитату (пример 10.7).

В поисковых запросах в браузере помимо специальных символов можно также использовать операторы расширенного поиска. Оператор site: используется для поиска на конкретном сайте (site: adu.by «олимпиады» — информация о предметных олимпиадах), related: для поиска похожих ресурсов (related: online-python.com — работа с языком Python онлайн).

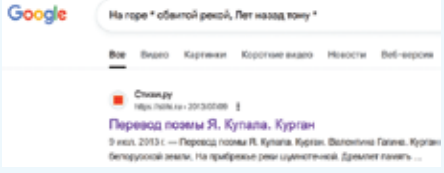
Все операторы помогают повысить точность поиска, фильтруя результаты и находя нужную информацию более эффективно.

Пример 10.5. Поиск сайтов, где есть информация о белорусских писателях З. Бядуле и Я. Брыле.



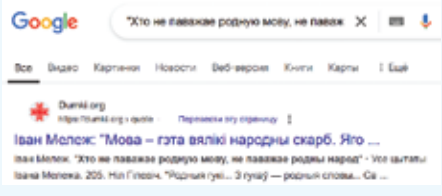
Пример 10.6. Поиск полной цитаты «На горе ... обвитой рекой, Лет назад тому ...». Из какого она произведения? Кто его автор?

Многоточие в поисковом запросе заменим оператором *.



Как видим, это цитата из произведения Я. Купалы «Курган» в переводе В. Гагиной на русский язык.

Пример 10.7. Узнать, кому принадлежат слова «Хто не паважае родную мову, не паважае родны народ».



Это цитата белорусского писателя И. Мележа.

Правообладатель Адукацыя і выхаванне

1. Что называют запросом в поисковой системе?
2. Как исключить некоторые записи из области поиска?
3. Какие операторы можно использовать в поисковых запросах?

Упражнения

1 Найдите с помощью различных поисковых систем информацию о спортсменах-легкоатлетах Республики Беларусь. Внесите количество результатов поиска для каждой из поисковых систем в таблицу (в тетради или в электронном виде). Сравните полученные результаты.

Поисковая система	Количество результатов поиска

Справка: информацию, как определить количество результатов в разных поисковых системах, можно найти в сети Интернет.

2 Найдите с помощью поисковой системы изображения монет Великого княжества Литовского. Выпишите в тетрадь 5–6 названий монет.

3 Сформулируйте запрос по поиску сюжетов пещерных росписей, исключающий роспись храмов.

4 Составьте запрос для поиска полной цитаты «Старость боится ... . Жизнь я ... куплю». Кто автор этой фразы? В каком произведении она встречается?

5 С помощью соответствующих поисковых запросов получите ответ на вопрос: какое из событий произошло раньше — открытие Д. И. Менделеевым периодического закона или изобретение Т. Эдисоном фонографа?

6 Одной из визитных карточек нашей страны являются озёра. Составьте запросы для поиска названий белорусских озёр, используя информацию из таблицы. Найдите краткую информацию об уникальности

каждого из этих озёр. Запишите названия найденных озёр и информацию о них в тетрадь.

Березинский биосферный заповедник	П						
Мядельский район Минской области				О			
Браславский район Витебской области			И				
Деревня Валевка Гродненской области	С						
Крупнейшее среди Голубых озёр						К	

7 В 6-м классе вы познакомились с сетевым этикетом и мерами безопасности при работе в сети Интернет. Определите, истинными или ложными являются следующие высказывания об Интернете.

1. В Интернете, так же как и в реальной жизни, применяются законы Республики Беларусь.
2. В сети Интернет не нужны нормы вежливости, уважительного и корректного общения.
3. Интернет совершенно безопасен.
4. Создавая почтовый ящик, нужно придумывать сложные пароли или секретные вопросы.
5. В качестве пароля нужно использовать личную информацию.
6. Письма от неизвестных авторов всегда безопасны и их нужно обязательно открывать.
7. По завершении работы на чужом компьютере нужно выйти из своего аккаунта.
8. Конфиденциальная информация — это информация с ограниченным доступом.
9. Конфиденциальной информацией можно делиться в социальных сетях и мессенджерах.
10. Самым достоверным источником информации в сети Интернет является искусственный интеллект.

Правообладатель Адукацыя і выхаванне

Глава 3
ОСНОВНЫЕ АЛГОРИТМИЧЕСКИЕ КОНСТРУКЦИИ
§ 11. Алгоритмы и исполнители

Большинству исполнителей для выполнения алгоритма нужны команды, записанные на формальном языке. Для того чтобы взаимодействие человека и технического устройства было плодотворным, человек обучается формальным языкам, например языкам программирования. Другой путь — совершенствовать исполнителей так, чтобы они понимали естественный язык человека. Человечество пока ещё в начале такого пути, однако уже сегодня существуют исполнители с голосовым управлением.

Первыми бытовыми устройствами с голосовым управлением были стиральные машины и сотовые телефоны. В настоящее время голосовое управление имеют бытовые компьютеры, автомобили, музыкальные центры, кондиционеры, лифты и др.



11.1. Понятие алгоритма

Вспомним некоторые понятия, с которыми вы познакомились в 6-м классе.

Алгоритм — понятная и конечная последовательность точных действий (команд), формальное выполнение которых позволяет получить решение поставленной задачи.

Исполнитель алгоритма — человек, группа людей или техническое устройство, которые понимают команды алгоритма и умеют правильно их выполнять.

Система команд исполнителя — команды, которые понимает и может выполнить исполнитель.

Любой исполнитель имеет ограниченную систему команд. Все они разделяются на группы:

1) команды, которые непосредственно выполняет исполнитель;

2) команды, меняющие порядок выполнения других команд исполнителя.

Компьютер является универсальным исполнителем.

Запись алгоритма в виде последовательности команд, которую может выполнить компьютер, называют **программой.**

Выделяют следующие способы представления алгоритмов (пример 11.1):

- **словесный** — описание алгоритма средствами естественного языка с точной и конкретной формулировкой фраз;
- **графический (блок-схема)** — графическое изображение команд алгоритма с использованием блоков в виде геометрических фигур, отрезков или стрелок, соединяющих эти блоки и указывающих на порядок выполнения команд;
- **программный** — запись алгоритма в виде программы.

На практике могут применяться комбинации вышеперечисленных способов записи алгоритмов, а также другие способы представления: **табличный**, описание алгоритма с помощью **математических формул**, **псевдокод** (описание структуры алгоритма на частично формализованном языке).

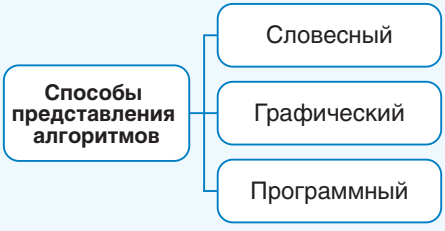
Голосовые помощники Google Assistant и Алиса, созданная компанией Яндекс, могут осуществлять поиск, поддерживать беседу, играть в игры, помогают управлять системой «Умный дом» и многое другое.





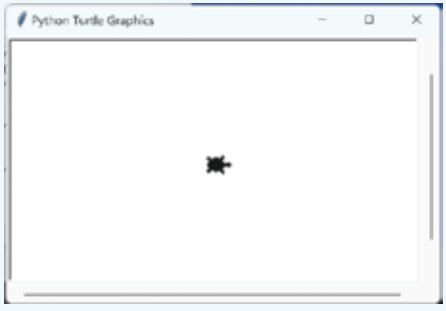
Пример 11.1. Способы представления алгоритмов.

На разных этапах решения задачи могут применяться разные способы описания алгоритма. На этапе обсуждения используются словесные и формульные способы. На этапе проектирования рекомендуется использовать графическое представление. Для проверки результатов возможно табличное представление. Программный способ используют на этапе непосредственного применения для решения прикладных задач.



Правообладатель Адукацыя і выхаванне

Пример 11.2. Среда обитания исполнителя *Черепаха*.



Пример 11.3. Основные команды исполнителя *Черепаха*.

Команда	Действие
`penup()`	Не оставлять след при движении
`pendown()`	Оставлять след при движении
`forward(X)`	Пройти вперёд X пикселей
`backward(X)`	Пройти назад X пикселей
`left(X)`	Повернуться налево на X градусов
`right(X)`	Повернуться направо на X градусов

11.2. Исполнитель *Черепаха*

В прошлом году вы познакомились с исполнителем *Черепаха*, который умеет строить рисунки и чертежи на координатной плоскости.

Среда обитания исполнителя *Черепаха* — координатная плоскость (пример 11.2). Исходное положение *Черепахи* — точка с координатами (0, 0), перо опущено.

Основные команды исполнителя *Черепаха* представлены в примере 11.3.

Пример 11.4. Прямоугольный участок, длина которого в 2 раза больше ширины, огородили забором длиной 120 метров. Определите длину и ширину участка. Составьте алгоритм и напишите программу, выполнив которую исполнитель *Черепаха* построит чертёж забора этого участка. Масштаб: 1 м соответствует 10 пикселям.

Словесное описание алгоритма вычисления длины и ширины участка:

1. Длина участка в два раза больше ширины, поэтому в сумме длина и ширина составят три одинаковых части. Забор огораживает участок по периметру. Периметр прямоугольника равен

удвоенной сумме длины и ширины, следовательно, он равен шести одинаковым частям.

2. Значение ширины участка получается делением длины забора на шесть частей.

3. Для нахождения значения длины участка удваиваем значение ширины.

Математическая запись действий:

1) $(1 + 2) \cdot 2 = 6$ (частей).

2) $120 : 6 = 20$ (м).

3) $20 \cdot 2 = 40$ (м).

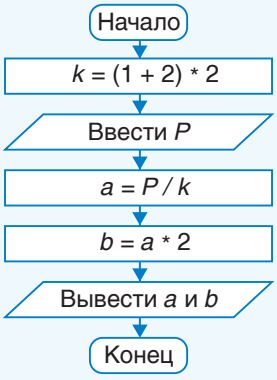
Словесное описание алгоритма рисования участка в масштабе 1 : 10:

```
Установить размеры поля 500 × 300
Поднять перо
В точку (-200, 100)
Опустить перо
Вперёд (400)
Направо (90)
Вперёд (200)
Направо (90)
Вперёд (400)
Направо (90)
Вперёд (200)
```

11.3. Алгоритмическая конструкция *следование*

Существует большое количество алгоритмов, в которых все команды выполняются последовательно одна за другой в том

Пример 11.4. Блок-схема алгоритма.



Программа для исполнителя *Черепаха*:

```
import turtle
turtle.shape('turtle')
turtle.setup(500, 300)
turtle.penup()
turtle.setpos(-200, 100)
turtle.pendown()

turtle.forward(400)
turtle.right(90)
turtle.forward(200)
turtle.right(90)
turtle.forward(400)
turtle.right(90)
turtle.forward(200)
```

Результат работы программы:



Правообладатель Адукацыя і выхаванне

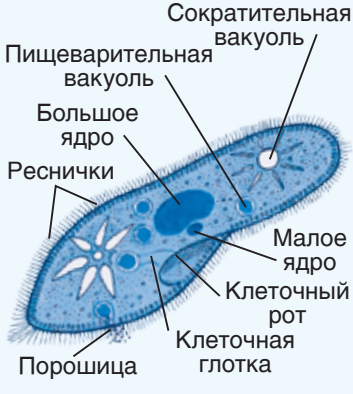
Пример 11.5. Алгоритм приготовления бутерброда.

1. Отрезать ломтик батона.
2. Положить на батон лист салата.
3. Отрезать кусочек копчёного мяса.
4. Положить мясо на лист салата.
5. Отрезать кусочек помидора.
6. Положить помидор на мясо.



Пример 11.6. Алгоритм выполнения лабораторной работы по биологии «Строение инфузории туфельки».

1. Рассмотреть под микроскопом внешний вид и внутреннее строение инфузории туфельки.
2. Зарисовать инфузорию туфельку и обозначить названия её органоидов.
3. Подвести итог работы.



порядке, в котором они записаны. В таких алгоритмах отсутствуют команды, меняющие порядок выполнения других команд. Программы такого вида вы составляли в прошлом году для исполнителя *Черепаха*.

Алгоритмическая конструкция *следование* — последовательность команд алгоритма, которые выполняются в том порядке, в котором они записаны.

Алгоритмическая конструкция *следование* отображает естественный, последовательный порядок выполнения действий в алгоритме.

Следование использовалось в примере 11.4 на с. 73, в котором описывался алгоритм вычисления длины и ширины участка и построения прямоугольника исполнителем *Черепаха*.

Алгоритмическая конструкция *следование* представлена в примерах 11.5 и 11.6.

11.4. Вспомогательные алгоритмы

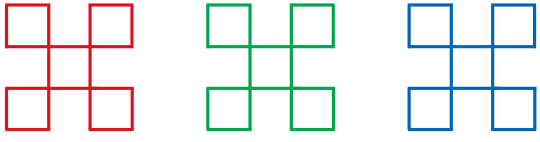
Часто в одной программе нужно рисовать одно и то же изображение несколько раз. Получе-

ние такого изображения удобно оформлять в виде вспомогательного алгоритма, который можно использовать нужное число раз.

Вспомогательный алгоритм — это алгоритм, целиком используемый в составе другого алгоритма.

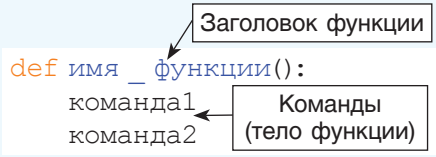
Вспомогательный алгоритм решает некоторую подзадачу основной задачи. Вызов вспомогательного алгоритма в программе заменяет несколько команд одной. В языке Python *вспомогательные алгоритмы* записываются в виде функций. Формат описания функции приведён в примере 11.7.

Пример 11.8. Написать программу, выполнив которую исполнитель *Черепаха* нарисует следующее изображение:



Данный рисунок состоит из трёх одинаковых фигур. Для рисования одной из них можно оформить вспомогательный алгоритм:

Пример 11.7. Формат описания функции.



Пример 11.8. Программа для исполнителя *Черепаха*.

```
import turtle

turtle.shape('turtle')
turtle.setup(670, 240)

def chast():
    turtle.forward(150)
    turtle.right(90)
    turtle.forward(50)
    turtle.right(90)
    turtle.forward(50)
    turtle.right(90)

def p(x, y):
    turtle.penup()
    turtle.setpos(x, y)
    turtle.pendown()
    turtle.setheading(270)

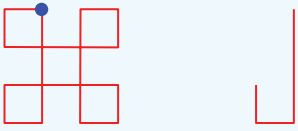
def figura(x, y, c):
    p(x, y)
    turtle.color(c)
    chast()
    chast()
    chast()
    chast()

figura(-200, 100, 'red')
figura(0, 100, 'green')
figura(200, 100, 'blue')
```

Правообладатель Адукацыя і выхаванне

Пример 11.8. *Продолжение.*

Начальная точка и часть фигуры:



Результат выполнения программы:



Над проектом по созданию программы может работать несколько человек (или десятков человек). Каждый из членов коллектива делает часть своей работы и оформляет её как отдельный вспомогательный алгоритм.

функцию `figura(x, y, c)`. Параметры `x`, `y` и `c` определяют координаты точки начала рисования и цвет фигуры. Каждая фигура в свою очередь состоит из четырёх одинаковых частей. Рисование одной части выполняет функция `chast()`. Для перемещения *Черепахи* в позицию с координатами (x, y) и поворота на 270° используется вспомогательный алгоритм `p(x, y)`.

Описание основного алгоритма:

```
Рисование фигуры
(-200, 100, 'red')
Рисование фигуры
(0, 100, 'green')
Рисование фигуры
(200, 100, 'blue')
```

1. Что такое алгоритм?
2. Какие способы записи алгоритмов вам известны?
3. На каком этапе решения задачи используется словесный способ описания алгоритма?
4. Что называют алгоритмической конструкцией *следование*?
5. Какие алгоритмы называются вспомогательными?
6. Для чего нужны вспомогательные алгоритмы?
7. Как описывается вспомогательный алгоритм на языке Python?

Упражнения

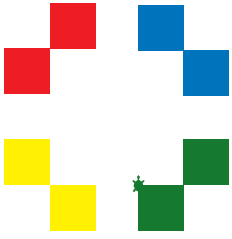
1 Определите, какие рисунки получатся после выполнения *Черепахой* следующих программ. Изобразите рисунки в тетради и проверьте правильность своих действий, выполнив программы на компьютере.

а)
```
import turtle
turtle.shape('turtle')
turtle.setup(500, 500)
turtle.penup()
turtle.setpos(-100, -100)
turtle.pendown()
turtle.setheading(108)
turtle.forward(200)
turtle.right(72)
turtle.forward(200)
turtle.right(72)
turtle.forward(200)
turtle.right(72)
turtle.forward(200)
turtle.right(72)
turtle.forward(200)
turtle.right(72)
```

б)
```
import turtle
turtle.shape('turtle')
turtle.setup(500, 500)
def chast():
    turtle.forward(200)
    turtle.right(90)
    turtle.forward(50)
    turtle.left(90)
    turtle.forward(50)
    turtle.right(90)
turtle.penup()
turtle.setpos(-100, 150)
turtle.pendown()
chast()
chast()
chast()
chast()
```

2 Напишите для исполнителя *Черепаха* программы получения следующих изображений.

а)



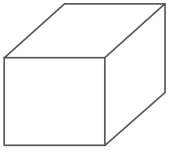
б)



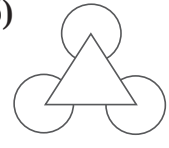
3 Придумайте свои рисунки и составьте программы для их рисования с помощью исполнителя *Черепаха*.

4* Исполнитель *Чертёжник* может рисовать только отрезки. Проанализируйте рисунки. Какие из них сможет выполнить исполнитель *Чертёжник*? Почему? Все ли из этих рисунков сможет нарисовать исполнитель *Черепаха*?

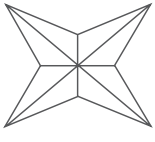
а)



б)



в)



г)



Правообладатель Адукацыя і выхаванне

§ 12. Исполнитель *Робот*

Роботы на службе человека



Робот на базе трактора BELARUS-132



Автоматизированный завод по производству комплексов с искусственным интеллектом холдинга «Горизонт»



Робот-пони для детей

12.1. Роботы в жизни человека

Человек с древности мечтал о создании, которое могло бы беспрекословно выполнять его приказы. Сегодня мечта стала реальностью — в жизни человека появились роботы. Они способны выполнять практически любую работу, доступную человеку, а также делать многие вещи, которые ему выполнить сложно или невозможно. Роботы используются на производстве и в быту, могут работать в сфере услуг и развлекать человека. Есть роботы, похожие на человека и совсем непохожие.

Робот — автоматическое устройство, которое действует по заранее составленной программе.

Робот получает информацию о внешнем мире от датчиков (аналогов органов чувств живых организмов) и предназначен для осуществления различных операций.

Мир роботов очень разнообразен. В быту современного чело-

века используются автоматические стиральные и посудомоечные машины, роботы-пылесосы. С помощью роботов можно выращивать растения или управлять домом.

На производстве роботы способны выполнять сложные и трудоёмкие работы. Широкое применение роботы находят в транспортировке грузов, расчистке завалов. Роботы эффективны для выполнения опасных и вредных видов работ, таких как опрыскивание полей ядохимикатами, тушение пожаров и исследование опасных объектов.

Постепенно робототехника становится частью образовательного процесса. В Республике Беларусь на базе учреждений общего среднего образования открыты инженерно-технические центры, оснащённые оборудованием для изучения робототехники.

Робот может быть материальным или виртуальным. Виртуальный робот — специальная программа, выполняющая определённые действия. Роботы являются



Российский лунный робот-кентавр[1] FEDOR



Робот-официант в гродненском кафе



Роботизированная рука белорусского производителя манипуляторов Rozum Robotics играет в шашки

[1] Материалы о роботах взяты с сайтов https://sputnik.by, https://iz.ru, http://www.robotics.by, https://tech.onliner.by, http://www.robogeek.ru, http://robotrends.ru.

Правообладатель Адукацыя і выхаванне



Робототехническое оборудование для инженерно-технических центров



Программируемые роботы (конструктор LEGO Education)

В отличие от исполнителя *Черепаха*, исполнитель *Робот* не входит в набор стандартных модулей языка Python. Сегодня можно выбрать несколько разных реализаций *Робота* на этом языке. В учебном пособии использован *Робот*, разработанный МФТИ[1].

Разработанная для исполнителя *Робот* библиотека была незначительно изменена и дополнена оригинальным задачником, в который входят все примеры и упражнения из учебного пособия.

исполнителями. Виртуального робота можно рассматривать как компьютерного исполнителя. Для исполнителей обычно определяют среду обитания и систему команд.

Общим для всех роботов является то, что человек может ими управлять. Робот может получать команды от оператора и выполнять их по одной или действовать автономно по предварительно составленной программе.

12.2. Среда обитания и система команд компьютерного исполнителя *Робот*

Средой обитания исполнителя *Робот* является прямоугольное клетчатое поле. Между некоторыми клетками, а также по периметру поля находятся стены. Серое кольцо обозначает начальное положение *Робота*, чёрная точка — конечное. *Робот* может передвигаться по полю и закрашивать указанные клетки. В заданиях из встроенного задачника *Робота* закрашенные клетки помечены серо-голубым цветом, те клетки, которые *Робот* дол-

[1] http://judge.mipt.ru/mipt_cs_on_python3/labs/lab2.html.

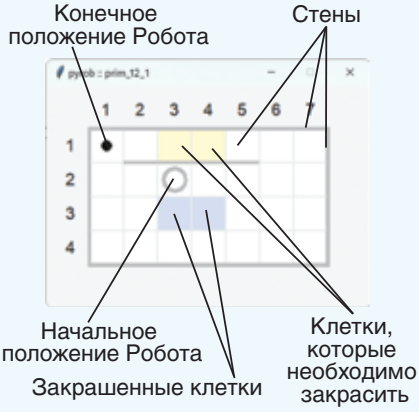
жен закрасить, — светло-жёлтым (пример 12.1).

Поле *Робота*, на котором определено положение стен, начальное и конечное положение исполнителя, а также клетки, которые уже закрашены и (или) должны быть закрашены, называют **обстановкой.**

Для подключения исполнителя *Робот* в программе прописывается команда `from pyrob.api import*`. Готовые задания с обстановками для *Робота* хранятся в задачнике и вызываются командой `@task`.

Структура программы для записи решения задачи из задачника показана в примере 12.2. Имя функции в программе должно соответствовать имени задачи. Все имена задач, которые соответствуют примерам из учебного пособия, начинаются со слова `prim`. Задачи, соответствующие упражнениям, начинаются с `upr`. Далее, после слов `prim` или `upr` следует соответствующий номер. В теле функции записываются команды исполнителя *Робот*.

Пример 12.1. Поле исполнителя *Робот* с начальной обстановкой.



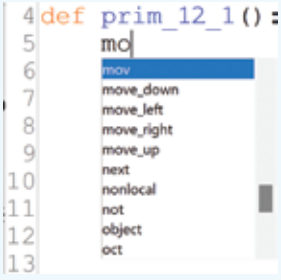
Пример 12.2. Вызов задачи `prim_12_1` из встроенного задачника.

```
*12_1.py - C:/prog_7kl/robot/9_1.p...
File Edit Format Run Options Window Help
1 from pyrob.api import *
2
3 @task
4 def prim_12_1():
5
6
7 run_tasks()
8
Ln: 8 Col: 0
```

Библиотека `pyrob` с задачником и примерами из учебного пособия размещена на образовательном портале https://eior.by. После скачивания и распаковки архива свои программы можно сохранять в ту же папку `robot`, в которой сохранены примеры. В этом случае установка библиотеки `pyrob` с помощью менеджера пакетов `pip` не потребуется.

Правообладатель Адукацыя і выхаванне

Пример 12.3. Выбор команды для *Робота* из выпадающего списка.



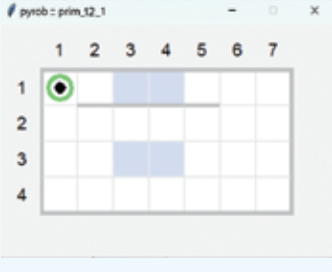
Пример 12.4. Решение задачи `prim_12_1`.

```
from pyrob.api import *

@task
def prim_12_1():
    move_right(4)
    move_up()
    move_left(2)
    fill_cell()
    move_left()
    fill_cell()
    move_left(2)

run_tasks()
```

Пример 12.5. Обстановка *Робота* после выполнения программы.



Система команд исполнителя *Робот*.

Команда	Действие
`move_right()`	Перемещает *Робота* вправо
`move_left()`	Перемещает *Робота* влево
`move_up()`	Перемещает *Робота* вверх
`move_down()`	Перемещает *Робота* вниз
`fill_cell()`	Закрашивает текущую ячейку

Команды при наборе можно выбирать из выпадающего списка. Для этого достаточно набрать первые буквы команды и нажать Ctrl + пробел (пример 12.3). Команды перемещения *Робота* могут содержать в качестве параметра число, которое указывает, на сколько клеток должен переместиться *Робот* в указанном направлении. Если число не указано, то *Робот* сдвигается на одну клетку.

Решение задачи `prim_12_1` приведено в примере 12.4. Если задача решена верно, кольцо, изображающее *Робота*, окрашивается в зелёный цвет (пример 12.5).

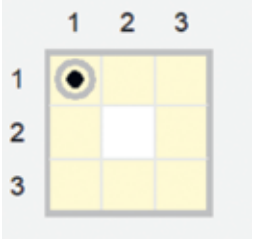
Робот может становиться на любую клетку. *Робот* не может

переместиться с клетки на клетку, если они разделены стеной. *Робот* не может переместиться за границы поля. При попытке выполнить недопустимое действие *Робот* изображается красным кругом (пример 12.6). *Робот* может закрасить уже закрашенную клетку, такое действие ошибку не вызывает. Если *Робот* выполнил все команды, но закрасил не все клетки, или закрасил лишние клетки, или оказался не в конечной точке, то он изображается серым кругом (пример 12.7).

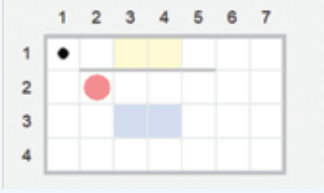
12.3. Использование алгоритмической конструкции *следование* для исполнителя *Робот*

Рассмотрим примеры решения задач для исполнителя *Робот*.

Пример 12.8. *Робот* находится на поле размером 3 × 3 клетки. Нужно закрасить все клетки, кроме средней.

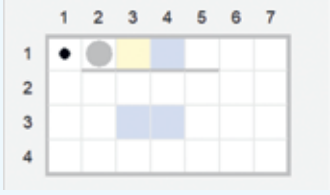


Пример 12.6. *Робот* врезался в стену.



Эта ситуация возникнет, если первой командой *Робота* будет команда `move_up()`.

Пример 12.7. *Робот* не закрасил одну клетку и не пришёл в конечную точку.

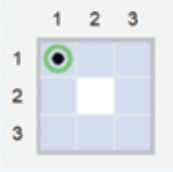


Пример 12.8. Программа для *Робота*.

```
from pyrob.api import *
@task
def prim_12_8():
    fill_cell()
    move_right()
    fill_cell()
    move_right()
    fill_cell()
    move_down()
    fill_cell()
    move_down()
    fill_cell()
    move_left()
    fill_cell()
    move_left()
    fill_cell()
    move_up()
    fill_cell()
    move_up()
run_tasks()
```

Правообладатель Адукацыя і выхаванне

Пример 12.8. *Продолжение.* Результат работы программы:



Другой способ записи программы для *Робота*:

```
from pyrob.api import *
@task
def prim_12_8():
    fill_cell(), move_right()
    fill_cell(), move_right()
    fill_cell(), move_down()
    fill_cell(), move_down()
    fill_cell(), move_left()
    fill_cell(), move_left()
    fill_cell(), move_up()
    fill_cell(), move_up()
run_tasks()
```

Пример 12.9. Программа для *Робота*.

```
from pyrob.api import *
@task
def prim_12_9():
    move_down()
    fill_cell()
    move_right(), move_up()
    fill_cell()
    move_right(), move_up()
    fill_cell()
    move_left(4)
    fill_cell()
    move_right()
    move_down()
    fill_cell()
    move_right()
run_tasks()
```

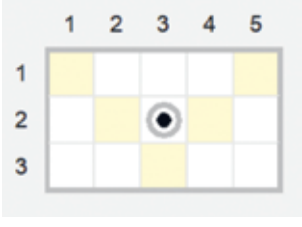
Для решения задачи *Робот* должен выполнить следующий алгоритм:

```
Закрасить, вправо
Закрасить, вправо
Закрасить, вниз
Закрасить, вниз
Закрасить, влево
Закрасить, влево
Закрасить, вверх
Закрасить, вверх
```

В данном алгоритме *Робот* обходит клетки, двигаясь по часовой стрелке. Тот же результат можно было бы получить, если бы *Робот* обходил поле против часовой стрелки, изначально двигаясь вниз.

При записи команд в программе в одной строке можно располагать несколько команд. В этом случае они отделяются запятой или точкой с запятой. Для исполнителя *Робот* такой способ записи делает программу более компактной и, иногда, более понятной.

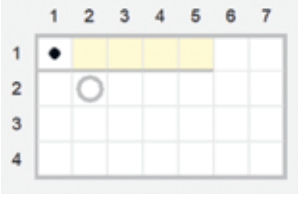
Пример 12.9. Составить программу для закраски клеток поля *Робота* по образцу.



Для решения задачи *Робот* должен выполнить следующий алгоритм:

```
Вниз
Закрасить
Вправо, вверх
Закрасить
Вправо, вверх
Закрасить
Влево на 4 клетки
Закрасить
Вправо
Вниз
Закрасить
Вправо
```

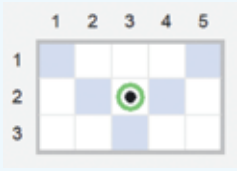
Пример 12.10. Составить программу для закраски клеток поля *Робота* по образцу.



Для решения задачи *Робот* должен обойти линию (стену), закрасить указанные клетки, переместиться в клетку, определяющую конечное положение. Начальная обстановка похожа на обстановку из примера 12.1 на с. 81, поэтому можно воспользоваться уже имеющейся программой (пример 12.4 на с. 82) и внести в неё нужные изменения.

Пример 12.9. *Продолжение.*

Результат работы программы:



Пример 12.10. Изменение программы для *Робота* из примера 12.4.

```
from pyrob.api import *
@task
def prim_12_10():
    move_right(4)
    move_up()
    move_left(2)
    fill_cell()
    move_left()
    fill_cell()
    move_left(2)
run_tasks()
```

10

Скопировать и вставить 2 раза перед последней командой

Для более компактной записи можно объединить команды `move_left()`, `fill_cell()` в одну строку.

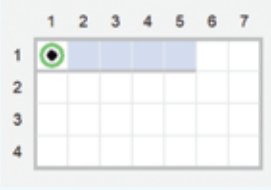
Изменённая программа:

```
from pyrob.api import *
@task
def prim_12_10():
    move_right(4)
    move_up()
    move_left(), fill_cell()
    move_left(), fill_cell()
    move_left(), fill_cell()
    move_left(), fill_cell()
    move_left()
run_tasks()
```

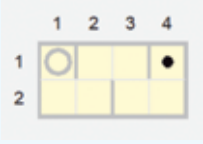
Правообладатель Адукацыя і выхаванне

Пример 12.10. *Продолжение.*

Результат работы программы:



Пример 12.11. Начальная обстановка.



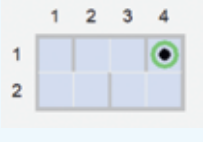
Программа 1
для исполнителя *Робот*:

```
from pyrob.api import *

@task
def prim_12_11():
    fill_cell(), move_down()
    fill_cell(), move_right()
    fill_cell(), move_up()
    fill_cell()
    move_right()
    fill_cell(), move_down()
    fill_cell(), move_right()
    fill_cell(), move_up()
    fill_cell()

run_tasks()
```

Результат работы программы:



Алгоритм решения задачи:

```
Вправо на 4 клетки
Вверх
Влево, закрасить
Влево, закрасить
Влево, закрасить
Влево, закрасить
Влево
```

В примерах 12.8–12.10 команды исполнителя *Робот* выполнялись последовательно, одна за другой, в том порядке, в котором они были записаны. Все приведённые алгоритмы реализованы с использованием алгоритмической конструкции *следование*.

12.4. Вспомогательные алгоритмы для исполнителя *Робот*

Пример 12.11. *Робот* должен закрасить все клетки на поле размером 2 × 4. Но двигаться по прямой линии ему мешают стены, которые нужно обойти.

Алгоритм решения задачи:

```
Закрасить, вниз
Закрасить, вправо
Закрасить, вверх
Закрасить
Вправо
Закрасить, вниз
Закрасить, вправо
Закрасить, вверх
Закрасить
```

Если проанализировать алгоритм, то можно заметить, что дважды повторяется последовательность команд, которая закрашивает квадрат из четырёх клеток:

```
Закрасить, вниз
Закрасить, вправо
Закрасить, вверх
Закрасить
```

Оформим эти команды как вспомогательный алгоритм, который назовём `квадрат`. Тогда алгоритм решения исходной задачи может быть записан так:

```
Квадрат
Вправо
Квадрат
```

Использование вспомогательного алгоритма при решении этой задачи позволило не записывать дважды одну и ту же последовательность команд. Вспомогательные алгоритмы можно использовать и в том случае, когда исходная задача разбивается на несколько независимых друг от друга задач. Тогда каждую из них можно оформить как вспомогательный алгоритм.

Пример 12.11. *Продолжение.*

Программа 2 (с использованием вспомогательного алгоритма) для исполнителя *Робот*.

```
from pyrob.api import *

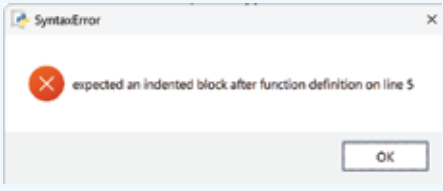
@task
def prim_12_11():
    def kvadrat():
        fill_cell(), move_down()
        fill_cell(), move_right()
        fill_cell(), move_up()
        fill_cell()

    kvadrat()
    move_right()
    kvadrat()

run_tasks()
```

При наборе текста программы обратите внимание на расположение описания вспомогательного алгоритма `kvadrat()` в программе и правильную расстановку отступов при записи команд. Функция `kvadrat()` располагается внутри функции `prim_12_11`, поэтому для всех команд, составляющих тело функции `kvadrat()`, необходим двойной отступ.

При неправильном отступе получим ошибку.



Правообладатель Адукацыя і выхаванне

Пример 12.12. Программа для исполнителя *Робот*.

```
from pyrob.api import *

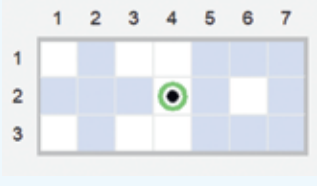
@task
def prim_12_12():
    def krest():
        move_left()
        fill_cell(), move_left()
        fill_cell(), move_left()
        fill_cell()
        move_down(), move_right()
        fill_cell(), move_up()
        fill_cell(), move_up()
        fill_cell()

    def ramka():
        fill_cell(), move_right()
        fill_cell(), move_right()
        fill_cell(), move_down()
        fill_cell(), move_down()
        fill_cell(), move_left()
        fill_cell(), move_left()
        fill_cell(), move_up()
        fill_cell(), move_up()

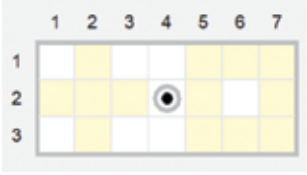
    krest()
    move_right(3)
    ramka()
    move_left()
    move_down()

run_tasks()
```

Результат работы программы:



Пример 12.12. Написать программу для закраски клеток поля *Робота* по образцу.



В данной задаче *Робот* должен нарисовать две отдельные фигуры: крест и рамку. Составим два вспомогательных алгоритма: крест и рамка.

Вспомогательный алгоритм крест:

```
Влево
Закрасить, влево
Закрасить, влево
Закрасить, вниз, вправо
Закрасить, вверх
Закрасить, вверх
Закрасить
```

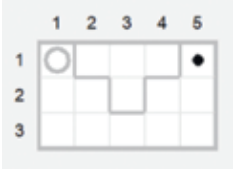
В качестве вспомогательного алгоритма рамка можно использовать программу из примера 12.8 на с. 83. Для перехода от одной фигуры к другой нужно сдвинуться на 3 клетки вправо, а после рисования рамки — вернуться в исходную клетку. Описание основного алгоритма:

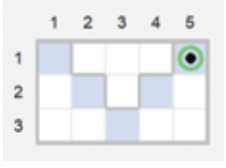
```
Крест
Вправо на 3 клетки
Рамка
Влево, вниз
```

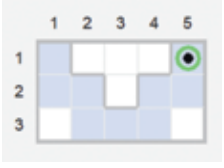
1. Что такое робот?
2. Какие команды входят в систему команд учебного компьютерного исполнителя *Робот*?
3. Опишите среду обитания учебного исполнителя *Робот*.
4. В чём особенность применения алгоритмической конструкции *следование*?
5. В каких случаях полезно использовать вспомогательные алгоритмы?

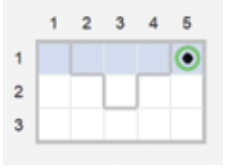
Упражнения

1 Начальная обстановка на поле *Робота* изображена на рисунке справа. Трое учащихся составили и выполнили алгоритм, по которому *Робот* закрасил все клетки пути от начальной к конечной. На каком из рисунков — а, б или в — изображено решение этой задачи? Поясните.



а)


б)


в)


2 Определите, какой из приведённых алгоритмов решает задачу, сформулированную в упражнении 1. Объясните, почему другие программы не могут быть алгоритмами решения данной задачи.

а)
```
fill_cell(), move_down()
fill_cell(), move_right()
fill_cell(), move_down()
fill_cell(), move_right()
fill_cell(), move_right()
fill_cell(), move_up()
fill_cell(), move_right()
fill_cell(), move_up()
fill_cell()
```

б)
```
fill_cell()
move_down()
move_right()
fill_cell()
move_down()
move_right()
fill_cell()
move_right()
move_up()
fill_cell()
move_right()
move_up()
fill_cell()
```

в)
```
turtle.right(90)
turtle.forward(50)
turtle.left(90)
turtle.forward(50)
turtle.right(90)
turtle.forward(50)
turtle.left(90)
turtle.forward(50)
turtle.left(90)
turtle.forward(50)
turtle.right(90)
turtle.forward(50)
turtle.left(90)
turtle.forward(50)
```

Правообладатель Адукацыя і выхаванне

3 Определите, для какого исполнителя приведён алгоритм в упражнении 2, пункт в). Сформулируйте для этого исполнителя задачу, решением которой будет приведённый алгоритм.

4 Предложите другие способы решения задачи из примера 12.9.

5 Для исполнителя *Робот* была составлена следующая программа:

```
fill_cell()
move_right()
move_up()
fill_cell()
move_right()
move_down()
```

Изобразите в тетради «узор», который нарисует *Робот*. При каких минимальных размерах поля *Робот* сможет выполнить данную программу?

6 Все команды в программе из упражнения 5 учащийся скопировал три раза. Как изменится «узор» после выполнения программы? Как можно по-другому записать этот алгоритм? Какого размера поле нужно создать? Подсказка: воспользуйтесь вспомогательным алгоритмом.

7 Программа решения задачи была записана на доске. Двое учащихся, записывая этот алгоритм для исполнителя *Робот*, пропустили из-за невнимательности по одной команде из этой программы. Какую команду пропустил каждый из учащихся? Что будет результатом работы каждой программы? Проверьте свой ответ на компьютере.

Программа, записанная первым учащимся	Программа, записанная вторым учащимся

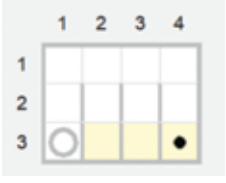
`from pyrob.api import *`
`@task`
`def upr_12_7():`
`  fill_cell(), move_right()`
`  fill_cell(), move_right()`
`  fill_cell(), move_down()`
`  fill_cell(), move_down()`
`  fill_cell(), move_down()`
`  move_left()`
`  move_down()`
`  move_left()`
	`from pyrob.api import *`
	`@task`
	`def upr_12_7():`
	`  fill_cell(), move_right()`
	`  fill_cell(), move_right()`
	`  fill_cell(), move_down()`
	`  fill_cell(), move_down()`
	`  fill_cell(), move_down()`
	`  move_left()`
	`  fill_cell()`
	`  move_left()`

Продолжение таблицы

Программа, записанная первым учащимся	Программа, записанная вторым учащимся

`fill_cell(), move_down()`
`fill_cell(), move_down()`
`fill_cell(), move_right()`
`fill_cell(), move_right()`
`fill_cell(), move_up()`
`fill_cell(), move_up()`
`fill_cell(), move_up()`
`move_left()`
`fill_cell(), move_up()`
`move_left()`
`fill_cell(), move_up()`
`fill_cell(), move_up()`
`run_tasks()`
	`fill_cell(), move_down()`
	`fill_cell(), move_down()`
	`fill_cell(), move_right()`
	`fill_cell(), move_right()`
	`fill_cell(), move_up()`
	`fill_cell(), move_up()`
	`fill_cell(), move_up()`
	`move_left()`
	`fill_cell(), move_up()`
	`move_left()`
	`fill_cell(), move_up()`
	`fill_cell(), move_up()`
	`run_tasks()`

8 Составьте программу для решения задачи `upr_12_8`. Реализуйте два алгоритма: один с использованием алгоритмической конструкции *следование*, другой — с использованием вспомогательного алгоритма. Сравните решения.



§ 13. Алгоритмическая конструкция *повторение*

13.1. Алгоритмы с циклами

В окружающем мире можно наблюдать ситуации, при которых различные действия, процессы и события повторяются. Одни повторяются несколько раз и завершаются. Другие могут повторяться очень долго (например, круговорот воды в природе, бесконечное движение планет в космическом пространстве и т. д.).

Понятие цикла используется в различных сферах человеческой деятельности. Под циклом понимают совокупность явлений, процессов, составляющих кругооборот в течение определённого промежутка времени. С этой точки зрения можно говорить о годовом цикле вращения Земли вокруг Солнца или о производственном цикле.

Правообладатель Адукацыя і выхаванне

Пример 13.1. Приготовление пельменей.

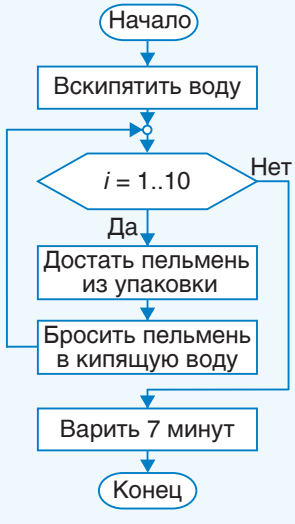


Алгоритм:

1. Вскипятить воду.
2. Для i от 1 до 10 повторять:
   а) достать пельмень из упаковки;
   б) бросить пельмень в кипящую воду.
3. Варить 7 минут.

В данном примере параметр цикла i изменяется от 1 до 10. Действия «Достать пельмень из упаковки» и «Бросить пельмень в кипящую воду» составляют тело цикла. Тело цикла выполнится 10 раз (для значений i = 1, 2, ..., 10).

Блок-схема данного алгоритма будет выглядеть следующим образом:



Человеку также регулярно приходится выполнять повторяющиеся действия: умываться, готовить еду, посещать парикмахерскую, ходить на работу и др.

Как правило, человек составляет программы, в которых каждая команда в отдельности и весь алгоритм в целом выполняются за конечное число повторений.

Алгоритмическая конструкция *повторение* (цикл) определяет последовательность действий, выполняемых многократно. Эту последовательность действий называют **телом цикла.**

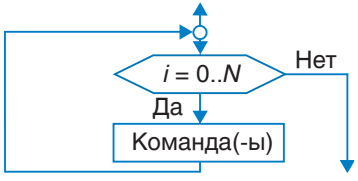
Существует несколько способов управления тем, сколько раз будет повторяться тело цикла.

Команда ***цикл с параметром* (цикл со счётчиком, цикл «для»)** — способ организации алгоритмической конструкции *повторение* (цикл), при котором количество повторов зависит от начального и конечного значений параметра цикла.

Цикл с параметром организует выполнение команд тела цикла заранее известное число раз (пример 13.1).

Параметр цикла играет роль счётчика и определяет нумерацию повторов тела цикла. Часто нумерацию начинают с 0 и ограничивают числом N. Цикл выполнится N раз для значений 0, 1, ..., $N - 1$. Если нумерация установлена двумя произвольными числами $N1$ (начальное значение) и $N2$ (конечное значение), то цикл выполнится ($N2 - N1$) раз (примеры 13.2, 13.3).

Алгоритмическая конструкция цикла с параметром может изображаться на блок-схеме следующим образом:



В этой конструкции в прямоугольнике(-ах) записываются повторяющиеся команды алгоритма (тело цикла), которые выполняются N раз (да). При этом после каждого выполнения команд тела цикла происходит проверка, сколько раз уже выполнился цикл. На блок-схеме переход на проверку условия изображается в виде стрелки, выходящей из тела цикла и возвращающейся

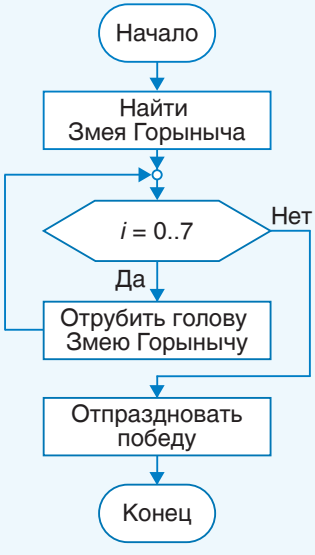
Пример 13.2. Вычислить a^n (например, $3^5 = 243$). Алгоритм возведения числа в степень может быть описан следующим образом:

1. Ввести значения a и n.
2. Определить начальное значение результата $r = 1$.
3. Для i от 0 до n повторять:
   умножить результат на a.
4. Записать результат.

Пример 13.3. В фольклорных произведениях часто встречается многоголовый Змей Горыныч (количество голов может быть равно, например, 7). Алгоритм победы над Змеем Горынычем может быть описан следующим образом:

1. Найти Змея Горыныча.
2. Для i до 7 повторять:
   отрубить голову
   Змею Горынычу.
3. Отпраздновать победу.

Блок-схема данного алгоритма:



Правообладатель Адукацыя і выхаванне

Пример 13.4. Работа цикла `for i in range (N1, N2)`.

Вначале цикл выполняется для значения $i = N1$. Затем значение i увеличивается на 1 и цикл выполняется для значения $i = N1 + 1$. Так продолжается до тех пор, пока i не достигнет значения $N2$. При значении $i = N2$ цикл завершает работу и команды в теле цикла больше не выполняются. Последний раз команды в теле цикла выполняются для значения $i = N2 - 1$.

Многие роботы, которые используются в быту и на производстве, могут выполнять циклические алгоритмы. Примером такого робота является суши-робот, который может производить от 450 до 4000 заготовок для суши в час.



к проверке. Как только команды тела цикла выполнятся N раз (нет), цикл завершится. Если $N \leqslant 0$, то команды из тела цикла не выполнятся ни разу.

Для записи цикла с параметром в языке Python используется команда `for`. Формат записи команды:

```
for i in range (N1, N2):
    тело цикла
```

Строка `for i in range (N1, N2):` является **заголовком цикла**. Эту строку можно прочитать следующим образом: «Для переменной i в диапазоне от $N1$ до $N2$ делай...». Переменную i называют **переменной (параметром) цикла**. Одно выполнение тела цикла называют **итерацией цикла**. Процесс работы цикла `for` описан в примере 13.4.

Команды тела цикла записываются со сдвигом относительно заголовка цикла.

В языке Python существуют другие варианты записи цикла `for`. Если нужно указать только количество выполнений цикла, то его можно записать так:

```
for i in range (N):
    тело цикла
```

Такая запись соответствует записи:

```
for i in range (0, N):
    тело цикла
```

Если переменная цикла должна изменяться не на 1, то можно задать шаг её изменения:

```
for i in range (N1, N2, h):
    тело цикла
```

В этом случае при каждой итерации цикла значение переменной i будет изменяться на величину h. Значение h может быть положительным или отрицательным целым числом, но не может равняться нулю (пример 13.5).

13.2. Использование команды *цикл с параметром* для исполнителя *Робот*

Пример 13.6. Написать программу для закрашивания 10 клеток поля *Робота* по образцу.

Для закрашивания всех 10 клеток поля нужно в цикле выполнить 10 раз команды:

```
Закрасить
Вправо
```

Данные команды образуют тело цикла.

Командами, образующими тело цикла, могут быть любые команды из системы команд исполнителя. Кроме того, в теле цикла

Пример 13.5. Варианты цикла for.

Уменьшение переменной цикла на 1:

```
for i in range (5, 0, -1):
```

Переменная цикла будет принимать значения: 5, 4, 3, 2, 1.

Увеличение переменной цикла на 10:

```
for i in range (-20, 30, 10):
```

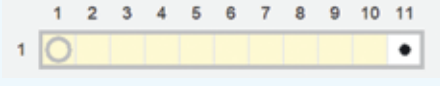
Переменная цикла будет принимать значения: –20, –10, 0, 10, 20.

Использование только нечётных чисел от 1 до 100:

```
for i in range (1, 100, 2):
```

Переменная цикла будет принимать значения: 1, 3, 5, ..., 97, 99.

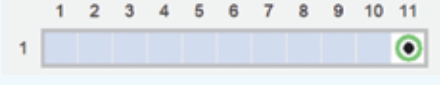
Пример 13.6. Начальная обстановка.



Программа для исполнителя *Робот*:

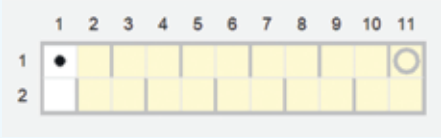
```
from pyrob.api import *
@task
def prim_13_5():
    for i in range (10):
        fill_cell()
        move_right()
run_tasks()
```

Результат работы программы:



Правообладатель Адукацыя і выхаванне

Пример 13.7. Начальная обстановка.



Программа для исполнителя *Робот*:

```
from pyrob.api import *

@task
def prim_13_7():
    def kvadrat():
        fill_cell()
        move_down()
        fill_cell()
        move_left()
        fill_cell()
        move_up()
        fill_cell()
        move_left()

    for i in range (5):
        kvadrat()

run_tasks()
```

может вызываться вспомогательный алгоритм. Использование вспомогательного алгоритма позволяет сократить запись тела цикла и делает программу более понятной.

Пример 13.7. Написать программу для закрашивания клеток поля *Робота* по образцу.

На поле исполнителя *Робот* присутствуют стены. При обходе стен *Робот* выполняет следующие команды:

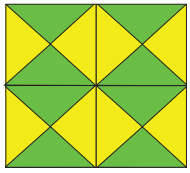
```
Закрасить, вниз
Закрасить, влево
Закрасить, вверх
Закрасить, влево
```

Для решения задачи данную группу команд *Робот* должен повторить 5 раз. Оформим эти команды как вспомогательный алгоритм `kvadrat` и вызовем в цикле этот алгоритм. В данном примере тело цикла состоит из одной команды `kvadrat`.

1. Какие циклические процессы, происходящие в окружающем мире, вы можете назвать?
2. Что понимают под алгоритмической конструкцией *повторение*?
3. Что такое тело цикла?
4. Что такое команда *цикл с параметром*?
5. Какие ключевые слова входят в описание заголовка цикла с параметром в Python?
6. Что такое итерация цикла?
7. Какие есть способы записи команды цикла в языке Python?

Упражнения

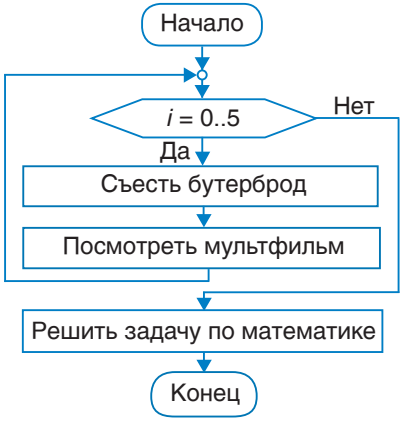
1 Опишите словесно или изобразите с помощью блок-схемы следующие алгоритмы.



1. Рисование в графическом редакторе следующего изображения (повторить 4 раза рисование квадрата с диагоналями и закрашенными областями).
2. Сверление 10 отверстий.
3. Сервировка стола к обеду на 6 персон.
4. Каждую минуту бактерия делится надвое. Изначально есть одна бактерия. За бактериями наблюдали 10 минут. Определите количество бактерий в конце наблюдения. Заполните таблицу согласно алгоритму.

Время, мин	0	1	2	3	4	5	6	7	8	9	10
Количество бактерий	1	2									

2 Семиклассник Вася после школы пригласил своего друга Колю помочь ему в решении 5 задач по математике. В гостях Коля посоветовал Васе провести остаток дня, воспользовавшись следующим алгоритмом, записанным в виде блок-схемы. Почему Вася получил двойку по математике?

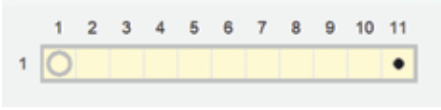


3 Составьте программу для решения задачи `upr_13_3`. Сравните алгоритм решения этой задачи с примером 13.5. Что у них общего? Чем они отличаются?

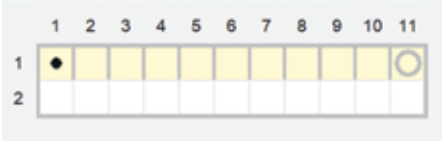


Правообладатель Адукацыя і выхаванне

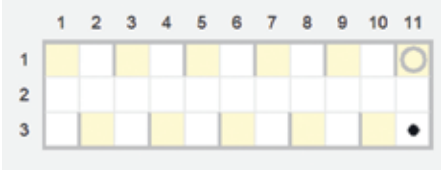
4 Составьте программу для решения задачи `upr_13_4`. Сравните её решение с программой из упражнения 3 и примера 13.5.



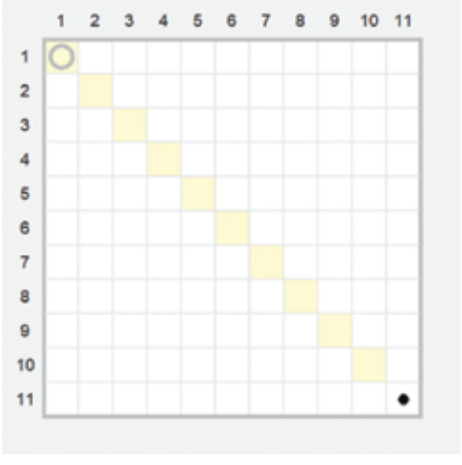
5 Составьте программу для решения задачи `upr_13_5`. Используйте вспомогательный алгоритм для решения задачи.



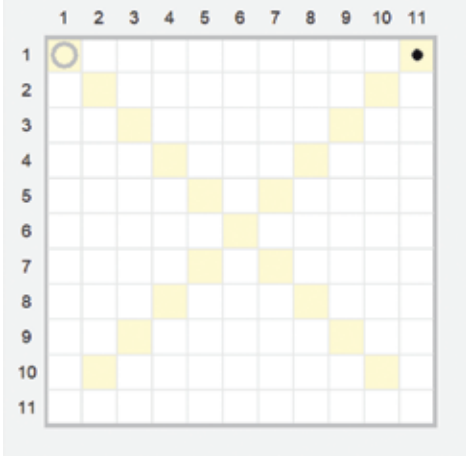
6* Составьте программу для решения задачи `upr_13_6`. Используйте вспомогательный алгоритм для решения задачи.



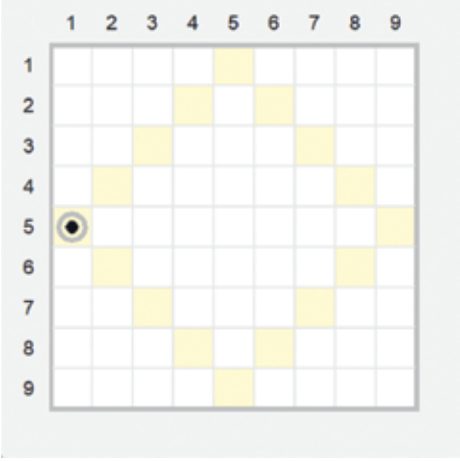
7 Составьте программу для решения задачи `upr_13_7`.



8* Составьте программу для решения задачи `upr_13_8`.



9 Для решения задачи `upr_13_9` Петя составил алгоритм и записал программу. Петин младший брат Олег удалил несколько команд. Сколько команд удалил Олег? Восстановите программу, которую написал Петя.



```
from pyrob. api import *
@task
def upr_13_9():
    for i in range (4):
        fill_cell()
        move_right()
        move_down()
    for i in range (4):
        move_right()
        move_up()
    for i in range (4):
        fill_cell()
        move_down()
run _tasks()
```

10 Вася пытается представить, как можно было бы использовать роботов в различных ситуациях, описанных в литературных произведениях. Например, для Тома Сойера, которого тётушка Поли отправила красить забор, Вася придумал робота-маляра. Вася решил, что такому роботу в системе команд достаточно одной команды: покрась доску. Алгоритм покраски забора из 20 досок Вася записал так:

```
1. Установить робота у левого края забора
2. Для i = 0..20 повторять
3.     Покрась доску
```

Сможет ли робот-маляр покрасить забор? В чём ошибка Васи? Исправьте алгоритм, добавив необходимую(-ые) команду(-ы).

§ 14. Использование условий

14.1. Условия для исполнителя *Робот*

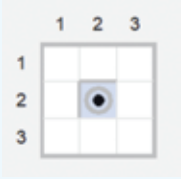
Как и многие другие исполнители, исполнитель *Робот* может не только выполнять действия, но и проверять условия.

Условием для *Робота*, как и для любого исполнителя, является

Робот-пылесос проверяет истинность условий с помощью датчиков. Датчики расстояния позволяют ему «видеть» препятствия и не касаться их. Если он наезжает на препятствие под углом, то срабатывают датчики столкновения. В этом случае пылесос меняет своё направление по заданному алгоритму.

Правообладатель Адукацыя і выхаванне

Пример 14.1. Начальная обстановка *Робота*.



В данном случае для *Робота* будут истинны следующие условия:

```
wall_up()
wall_left()
cell_is _ filled()
```

Ложными будут условия:

```
wall_down()
wall_right()
```

Пример 14.2. Сбор грибов.

Использование цикла с параметром при составлении алгоритма решения этой задачи может привести к разным результатам. Или корзина может быть полупустой, или не все найденные грибы в неё поместятся.





понятное ему высказывание, которое может быть истинным (соблюдаться) либо ложным (не соблюдаться). Рассмотрим систему условий для компьютерного исполнителя *Робот*.

Условие	Результат
`wall_up()`	Истинно, если стена *сверху*
`wall_down()`	Истинно, если стена *снизу*
`wall_left()`	Истинно, если стена *слева*
`wall_right()`	Истинно, если стена *справа*
`cell_is_filled()`	Истинно, если текущая клетка закрашена

Истинность условий для исполнителя *Робот* проверяется для конкретной обстановки (пример 14.1).

14.2. Команда *цикл с предусловием*

Цикл с параметром используется при составлении алгоритма в том случае, когда заранее известно количество повторений. Однако часто до выполнения цикла количество повторений неизвестно.

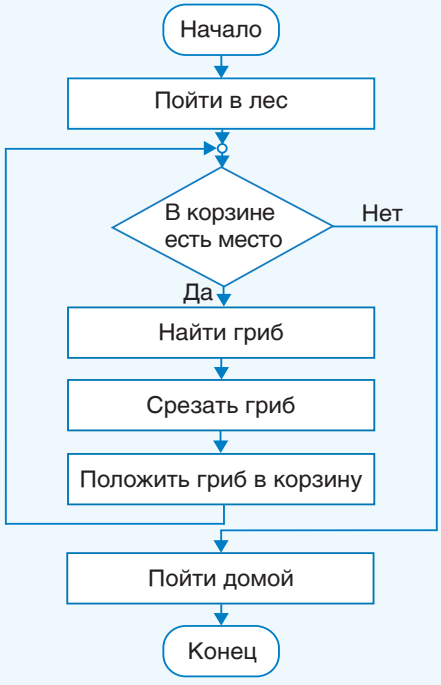
Пример 14.2. Вы с родителями пошли в лес собирать грибы (предполагается, что они там есть). Ваши действия можно описать командами: найти гриб, срезать гриб, положить гриб в корзину. Эти действия будут выполняться в цикле, но вы заранее не знаете, сколько грибов поместится в корзину. Поэтому можно говорить не о количестве повторений (количество грибов), а об условии, при котором вы будете продолжать сбор грибов: пока корзина не заполнена.

Команда **цикл с предусловием (цикл «пока»)** — это такой способ организации алгоритмической конструкции *повторение* (цикл), при котором количество выполнений команд тела цикла зависит от истинности или ложности условия цикла.

Цикл с предусловием используется в том случае, когда количество повторений тела цикла заранее неизвестно, но известно условие продолжения работы.

Условие цикла определяет, как долго будет выполняться цикл.

Пример 14.2. *Продолжение.*
При использовании цикла с предусловием домой унесём полную корзину грибов.

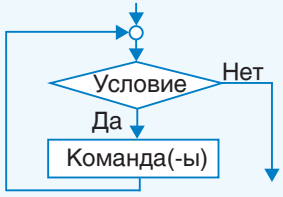


Циклы выполняются до тех пор, пока условие цикла истинно. Иногда нужно прервать выполнение цикла без проверки условия. Для этого используют команду `break`, которая прекращает выполнение цикла и передаёт управление команде, следующей за циклом.

Иногда возникает необходимость принудительно начать следующую итерацию цикла, пропустив часть команд в его теле. Для таких случаев существует ключевое слово `continue`.

Правообладатель Адукацыя і выхаванне

Пример 14.3. Блок-схема цикла с предусловием.



Пример 14.4. Принцип работы команды *цикл с предусловием*.

В качестве условия записывается логическое выражение, которое определяет, как долго будет выполняться цикл. В теле цикла записываются повторяющиеся команды алгоритма, которые выполняются пока верно условие цикла (да).

На блок-схеме команды тела цикла заключены в прямоугольники, а условие — в ромб.

Сначала проверяется условие, и если оно истинно, то выполняются команды тела цикла. После каждого выполнения команд тела цикла автоматически происходит возврат к проверке истинности условия.

Как только условие станет ложным (нет), цикл завершится.

Количество итераций цикла будет на 1 меньше, чем количество проверок истинности условия цикла. Если условие сразу ложно, то цикл не выполнится ни разу, т. е. количество итераций цикла будет равно нулю.

Пока условие истинно, выполняются команды, составляющие тело цикла. Цикл прекращает выполняться тогда, когда условие становится ложным. *Цикл с предусловием* имеет такое название, поскольку проверка условия предваряет выполнение команд тела цикла.

В примере 14.3 показана блок-схема *цикла с предусловием*. Принцип работы *цикла с предусловием* описан в примере 14.4.

Если условие в цикле будет всегда истинно (всегда да), то такой цикл не сможет завершиться. Возникшую ситуацию называют **зацикливанием**.

Для записи цикла с предусловием в Python используется команда `while`. Формат записи команды:

```
while <условие>:
    тело цикла
```

Строка `while <условие>:` является **заголовком цикла**. Эту строку можно прочитать следующим образом: «Пока верно условие, делай...». Команды тела цикла записываются со сдвигом относительно заголовка цикла.

14.3. Использование команды *цикл с предусловием* для исполнителя *Робот*

Пример 14.5. Написать программу для решения следующей задачи. *Робот* находится в верхнем левом углу поля. Снизу от *Робота* вдоль всего поля расположена горизонтальная линия с проходом в одну нижнюю клетку. Составить алгоритм, выполнив который, *Робот* сможет пройти через проход и закрасить клетку. Расположение прохода заранее неизвестно.

Нам известно, что проход не ограничен стеной снизу. *Робот* может двигаться вправо до тех пор, пока внизу есть стена.

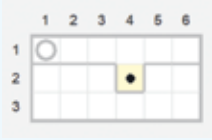
```
Пока снизу стена повторять
    Вправо
```

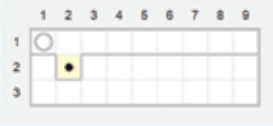
Робот остановится в той клетке, у которой снизу нет стены. После этого *Робот* должен переместиться вниз и закрасить клетку.

Пример 14.6. Написать программу для закраски клеток коридора переменной длины.

В данной задаче нам не известна длина коридора. *Робот* может двигаться, пока справа пусто, и закрашивать клетки:

Пример 14.5. Возможные начальные обстановки.





Задачи, в которых могут быть различные обстановки, проверяются на 4–6 различных тестовых примерах. Задача считается решённой, если программа правильно работает для каждого из них.

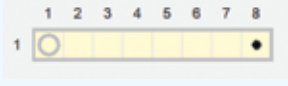
Программа для исполнителя *Робот*:

```
from pyrob.api import *

@task
def prim_14_5():
    while wall_down():
        move_right()
    move_down()
    fill_cell()

run_tasks()
```

Пример 14.6. Одна из возможных начальных обстановок.



Правообладатель Адукацыя і выхаванне

Пример 14.6. *Продолжение.*

Программа для исполнителя *Робот*:

```
from pyrob.api import *

@task
def prim_14_4():
    while not wall_right():
        fill_cell()
        move_right()
    fill_cell()

run_tasks()
```

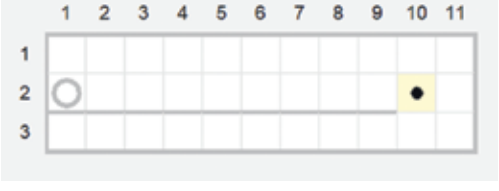
```
Пока нет стены справа повторять
        Закрасить
        Вправо
```

После прохождения всего коридора *Робот* должен закрасить последнюю клетку. Это действие происходит после выполнения цикла (команда `fill_cell()`), так как для последней клетки условие «справа пусто» уже не выполняется — там расположена стена.

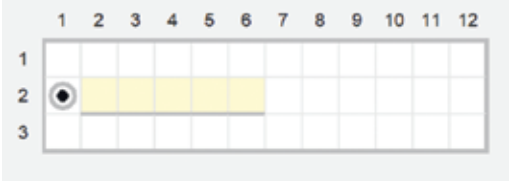
1. Что понимают под условием для исполнителя?
2. В каких случаях в алгоритмах используется команда *цикл с предусловием*?
3. Как работает *цикл «пока»*?
4. Как записывается команда *цикл с предусловием* на языке Python?
5. Когда возникает ситуация зацикливания?

Упражнения

1 Напишите программы для решения задач `upr_14_1_a` и `upr_14_1_b` из встроенного задачника. Обращайте внимание на начальное и конечное положение *Робота*.



`upr_14_1_a`



`upr_14_1_b`

2 Для исполнителя *Робот* был написан следующий алгоритм:

```
from pyrob.api import *

@task
def upr_14_2():
    while not wall_right():
        fill_cell()
        move_down()
        move_right()
        fill_cell()
        move_up()
        move_right()

run_tasks()
```

Нарисуйте в тетради результат работы алгоритма. Какими должны быть размеры поля, чтобы *Робот* не врезался в стену? Определите начальное положение *Робота*.

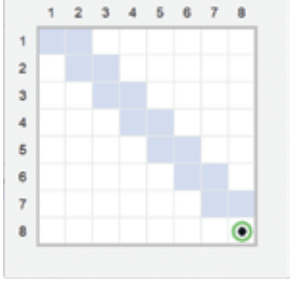
3 Составьте алгоритм, выполнив который, *Робот* нарисует «узор» из упражнения 2 вдоль левого края поля исполнителя (задача `upr_14_3`). Каким должен быть вертикальный размер поля исполнителя?

4 *Робот* находится на квадратном поле неизвестного размера. Начальное положение *Робота* — верхний левый угол. Составьте и выполните алгоритм, по которому *Робот* переместится из начального положения в нижний правый угол и закрасит все клетки своего пути. На каком (каких) из рисунков изображено решение этой задачи? Поясните.

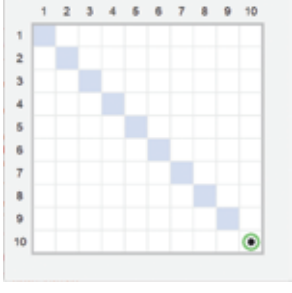
а)



б)

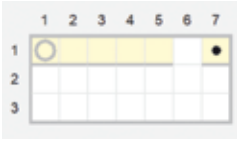


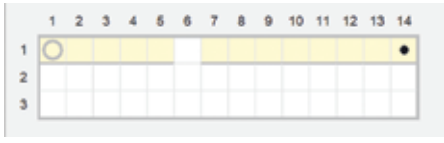
в)



Правообладатель Адукацыя і выхаванне

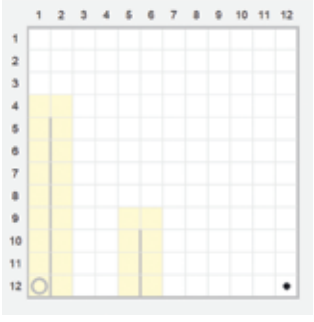
5 На поле *Робота* размещён «забор» — горизонтальная стена. Забор нужно «покрасить» — написать программу для закрашивания всех клеток сверху стены (задача `upr_14_5`). В «заборе» могут быть одни «ворота» — клетка без линий. Длина «забора» и расположение «ворот» неизвестны.





6* Решите задачу `upr_14_6`. Используйте вспомогательный алгоритм для обхода каждой из двух стен, расположенных на поле *Робота*.





§ 15. Алгоритмическая конструкция *ветвление*

Пример 15.1. Выбор обуви весной в зависимости от погоды.

```
Если на улице дождь, то
    надеть резиновые сапоги
Иначе
    надеть туфли
```

В данном примере в текущий момент времени может быть выполнена только одна команда из двух: или надеть сапоги, или надеть туфли.

15.1. Команда *ветвление*

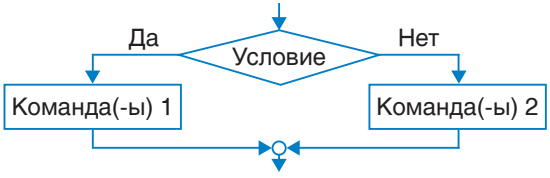
Довольно часто на поставленный вопрос человек получает ответ «да» или «нет». В зависимости от ответа человек определяет свои действия и выполняет одну или другую «команду» или группу «команд» (пример 15.1).

Роботы и другие технические устройства также могут выпол-

нять различные действия в зависимости от условий. Если условие истинно (на вопрос получен ответ «да»), то выполняется один набор действий, если ложно, то другой.

Алгоритмическая конструкция *ветвление* обеспечивает выполнение одной или другой последовательности команд в зависимости от истинности или ложности некоторого условия.

Команда *ветвление* может изображаться на блок-схеме следующим образом:



Принцип работы команды *ветвление* описан в примере 15.2.

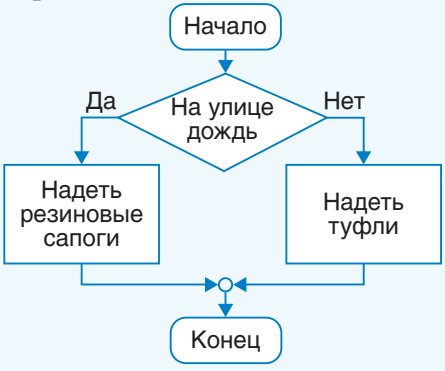
В языке программирования Python для записи конструкции *ветвление* используется команда `if`. Формат записи команды:

```
if <условие>:
    команда(-ы) 1
else:
    команда(-ы) 2
```

Строка `if <условие>:` является **заголовком ветвления**. Эту строку можно прочитать так:

Пример 15.1. *Продолжение.*

Блок-схема данного алгоритма будет выглядеть следующим образом:



Понятие ветвления используется в различных сферах человеческой деятельности.



В ботанике под ветвлением побегов понимают процесс увеличения числа побегов у растений.



Ветвления используются в дорожной разметке и картографии.

При употреблении термина в переносном смысле под ветвлением понимают наличие нескольких путей, направлений, сюжетных линий и т. д.

Правообладатель Адукацыя і выхаванне

Пример 15.2. Принцип работы команды *ветвление*.

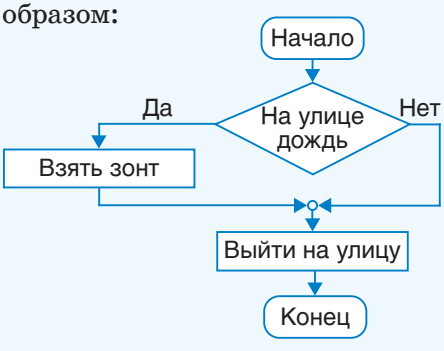
Выполнение конструкции начинается с проверки условия. Если условие истинно, то выполняется последовательность команд 1, если условие ложно, то выполняется последовательность команд 2. При такой организации алгоритма может выполниться **только одна** из двух последовательностей команд. Другая последовательность команд будет проигнорирована.

Пример 15.3. Выход на улицу осенью.

```
Если на улице дождь, то
    взять зонт
выйти на улицу
```

В данном примере используется сокращённая форма команды *ветвление*. Если условие истинно, то выполняется команда *взять зонт*. Если условие ложно, то никаких действий не происходит. Команда *выйти на улицу* выполняется всегда, независимо от истинности или ложности условия.

Блок-схема данного алгоритма будет выглядеть следующим образом:

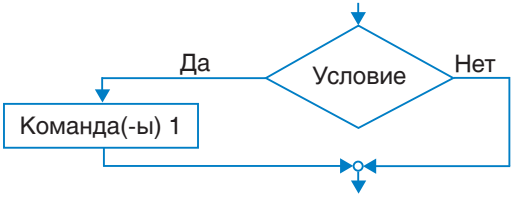


«Если условие верно, то...». В следующей строке со сдвигом записывается последовательность команд 1. После слова `else` в следующей строке со сдвигом записывается последовательность команд 2. Обратите внимание на то, что символ «:» ставится как в конце заголовка ветвления, так и после слова `else`.

Команда *ветвление* может быть записана в **полной** или **сокращённой** формах.

Полная форма команды *ветвления* организует выполнение двух разных наборов команд, из которых выполняется только один. В сокращённой форме один из наборов команд отсутствует. Если отсутствует набор команд, соответствующих ситуации, когда условие ложно, то никакие действия не выполняются (пример 15.3).

Блок-схема сокращённой формы *ветвление*:



На языке программирования Python команда записывается следующим образом:

```
if <условие>:
    команда(-ы) 1
```

В качестве последовательности команда(-ы) 1 и/или 2 может выступать другая команда ветвления. То есть, если истинно (ложно) одно условие, то может потребоваться проверить другое условие. Такие конструкции образуют **вложенные ветвления** (пример 15.4).

В языке Python для записи вложенных ветвлений (в случае ложности первого условия) можно применять конструкцию `elif`. Запись конструкции:

```
if <условие1>:
    команды 1
elif <условие2>:
    команды 2
else:
    команды 3
```

Ключевое слово `elif` в записи команды по существу заменяет два других: `else` и `if`. Блоков `elif` может быть несколько в том случае, если необходимо проверить не два условия, а больше.

15.2. Составные условия

В качестве условия в алгоритмах с циклами и ветвлениями используется любое понятное исполнителю этого алгоритма

Пример 15.4. Имеется три монеты, среди которых одна фальшивая. Известно, что фальшивая монета легче настоящих монет. Как найти фальшивую монету за минимальное число взвешиваний на чашечных весах без гирь? За какое количество взвешиваний можно определить фальшивую монету?

Представим словесное описание алгоритма решения этой задачи.

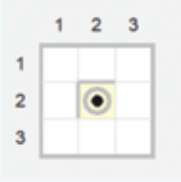
```
Положить на каждую чашу весов по одной монете (монета 1 и монета 2)
Если весы в равновесии, то
    фальшивая монета 3
Иначе
    Если монета 1 тяжелее, то
        фальшивая монета 2
    Иначе
        фальшивая монета 1
```

За одно взвешивание мы можем определить фальшивую монету.



Правообладатель Адукацыя і выхаванне

Пример 15.5. Начальная обстановка поля *Робота*.



Первое условие состоит из двух простых: *A* = `wall_left()` и *B* = `cell_is_filled()`. Условие может быть записано как *A* **И** *B*. Это условие верно только тогда, когда верны и *A*, и *B*. Условие *A* = 1 (истинно), условие *B* = 1 (истинно), условие *A* **И** *B* = 1 (истинно).

Второе условие может быть записано как *A* **ИЛИ** *B*, где *A* = `wall_up()`, *B* = `wall_down()`. Условие *A* = 1, условие *B* = 0. Значит, условие *A* **ИЛИ** *B* = 1 (истинно).

В третьем условии операция `not` отрицает составное условие `wall_right() or not wall_up()`. Условие может быть записано как **НЕ (***A* **ИЛИ НЕ** *B***)**. Соблюдая порядок выполнения логических операций, определим истинность или ложность данного условия:

1) **НЕ** *B* = 0, так как *B* = 1;

2) *A* **ИЛИ НЕ** *B* = 0, так как *A* = 0;

3) **НЕ (***A* **ИЛИ НЕ** *B***)** = 1.

Для других обстановок *Робота* истинность или ложность приведённых составных условий может быть другой.

высказывание, которое может быть либо истинным, либо ложным.

Все условия, с которыми нам приходилось до сих пор встречаться при составлении алгоритмов для *Робота*, были простыми высказываниями. Для исполнителя *Робот* можно строить и составные условия. Составное условие для *Робота*, так же как и для любого другого исполнителя, образуется из нескольких простых условий, соединённых друг с другом логическими операциями.

С логическими операциями над высказываниями вы уже знакомы. В Python используют следующие логические операции:

Логическая операция	Запись в Python
НЕ	`not`
И	`and`
ИЛИ	`or`

Истинность или ложность составного условия для исполнителя *Робот* определяется для конкретной обстановки.

Пример 15.5. Проверить для *Робота* следующие составные условия:

1. `wall_left() and cell_is_filled()`
2. `wall_up() or wall_down()`
3. `not (wall_right() or not wall_up())`

15.3. Использование команды *ветвление* для исполнителя *Робот*

Пример 15.6. *Робот* должен закрасить клетку, которая находится за стеной. В зависимости от обстановки обход стены может осуществляться по-разному.

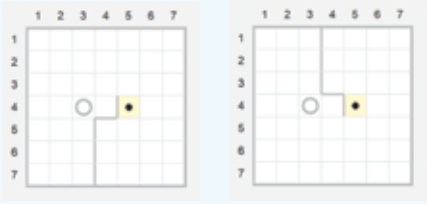
В начале *Робот* должен сдвинуться вправо. Если стена снизу, то сверху свободно и можно обойти стену сверху, в противном случае *Робот* обходит стену снизу. После обхода стены *Робот* закрашивает клетку. Алгоритм можно записать следующим образом:

```
Вправо
Если стена снизу, то
    Вверх
    Вправо
    Вниз
Иначе
    Вниз
    Вправо
    Вверх
Закрасить
```

Для решения этой задачи использована полная форма команды *ветвление*.

Пример 15.7. *Робот* находится на неизвестной клетке поля без линий. Он должен закрасить клетку слева от себя.

Пример 15.6. Две разные обстановки поля *Робота*.



Программа для исполнителя *Робот*:

```
from pyrob.api import *

@task
def prim_15_6():
    move_right()
    if wall_down():
        move_up()
        move_right()
        move_down()
    else:
        move_down()
        move_right()
        move_up()
    fill_cell()

run_tasks()
```

Пример 15.7. Программа для исполнителя *Робот*.

```
from pyrob.api import *

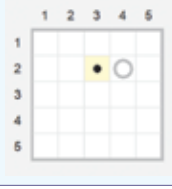
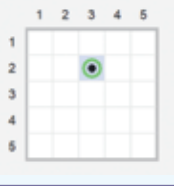
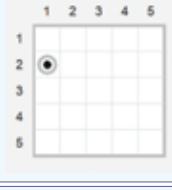
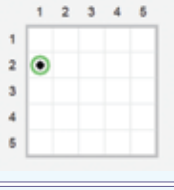
@task
def prim_15_7():
    if not wall_left():
        move_left()
        fill_cell()

run_tasks()
```

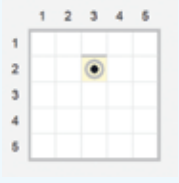
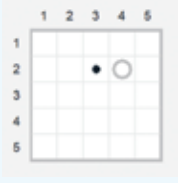
Правообладатель Адукацыя і выхаванне

Пример 15.7. *Продолжение.*

Результат работы программы для разных начальных обстановок:

Начальная обстановка	Результат
	
	

Пример 15.8. Две разные обстановки поля *Робота*.

Программа для исполнителя *Робот*:

```
from pyrob.api import *
@task
def prim_15_8():
    if wall_up():
        fill_cell()
    elif wall_down():
        fill_cell()
    else:
        move_left()
run_tasks()
```

Робот может переместиться влево для закрашивания клетки только в том случае, если слева нет стены, иначе он не выполнит никаких действий и останется на месте. Поэтому, прежде чем сдвинуться влево, *Робот* должен проверить, свободно ли слева. В том случае, когда *Робот* находится возле левой границы поля, он ничего не делает, оставаясь на своём месте.

Результат работы данной программы зависит от начального положения *Робота*. Для решения этой задачи использована сокращённая форма команды *ветвление*.

Пример 15.8. *Робот* находится на произвольной, не прилежащей к границе, клетке поля. Вокруг клетки сверху или снизу могут быть стены. Он должен закрасить клетку, на которой находится, если у неё есть хоть одна стена, иначе *Робот* должен сдвинуться влево.

Программа для решения этой задачи может быть записана двумя способами:

1) с использованием конструкции `elif`;

2) с использованием составного условия.

В первом случае *Робот* проверяет условие «стена сверху» и в случае его ложности переходит на проверку условия «стена снизу». Во втором случае используется составное условие: «стена сверху ИЛИ стена снизу».

Пример 15.8. *Продолжение.*
Исполняемый фрагмент программы для второго способа:

```
if wall_up() or wall_down():
    fill_cell()
else:
    move_left()
```

1. Что такое алгоритмическая конструкция *ветвление*?
2. Из чего состоит заголовок команды *ветвление*?
3. Чем отличается форма записи полной конструкции *ветвление* от сокращённой?
4. Какая конструкция может использоваться для записи вложенных ветвлений?
5. Что такое составное условие?
6. Какие логические операции можно использовать для записи составных условий?
7. Как определить истинность составного условия?

Упражнения

1 Выделите алгоритмическую конструкцию *ветвление* в отрывке из поэмы А. С. Пушкина «Руслан и Людмила» и изобразите эту конструкцию с помощью блок-схемы.

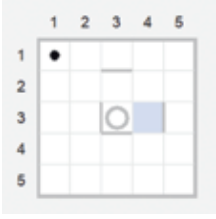
У Лукоморья дуб зелёный;
Златая цепь на дубе том:
И днём и ночью кот учёный
Всё ходит по цепи кругом;
Идёт направо — песнь заводит,
Налево — сказку говорит.
Там чудеса: там леший бродит,
Русалка на ветвях сидит...[1]

[1] Пушкин А. С. Руслан и Людмила : поэма. М., 1996. С. 5.

Правообладатель Адукацыя і выхаванне

2 Определите, какие из составных условий истинны, а какие ложны для заданной обстановки поля *Робота*.

Начальная обстановка	Условия

	`wall_left() or cell_is_filled()`
	`wall_up() and wall_down()`
	`not cell_is_filled() and not wall_right()`
	`not (wall_up() or not wall_right())`
	`wall_down() and cell_is_filled()`
	`(wall_up() or wall_down()) and not wall_right()`

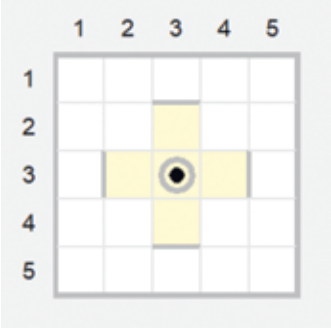
3 В каждом из условий упражнения 2 заменили операцию `and` на `or`, а операцию `or` на `and`. Изменится ли истинность приведённых условий?

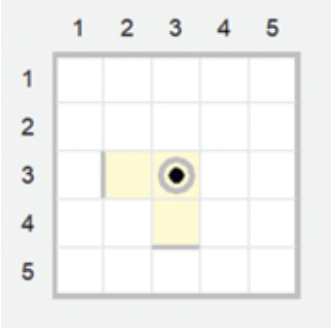
4 Для каждого из ложных условий упражнения 2 придумайте обстановку поля *Робота*, в которой данное условие будет истинным, а для каждого истинного — обстановку, в которой условие будет ложным.

5 Решите задачи `upr_15_5_1` и `upr_15_5_2`. Эти задачи аналогичны примеру 15.7 на с. 111. В задаче `upr_15_5_1` *Робот* должен закрасить клетку справа, а в задаче `upr_15_5_2` — снизу.

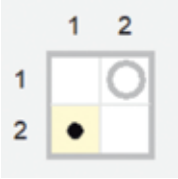
6* Решите задачи `upr_15_6_1` и `upr_15_6_2`.

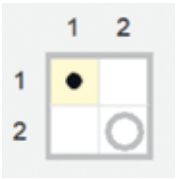
1. Клетка, на которой находится *Робот*, должна быть закрашена. Остальные клетки закрашиваются только тогда, когда есть стена с соответствующей стороны.





2. *Робот* находится в какой-то клетке поля размером 2 × 2. Он должен закрасить клетку в углу, противоположном начальному.





§ 16. Использование основных алгоритмических конструкций для исполнителя *Робот*

Последовательное выполнение команд в программе определяется структурой *следование*. Для организации повторяющихся действий в алгоритме используется команда *цикл*. Команда *ветвление* позволяет выполнять одну или другую последовательность команд в зависимости от истинности условия.

Следование, *цикл* и *ветвление* — **базовые алгоритмические конструкции.** Используя эти конструкции как элементы некоего «конструктора», можно составлять и разрабатывать любые алгоритмы.

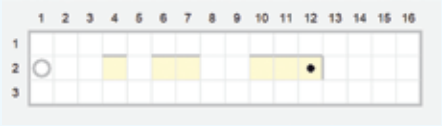
Команды *цикл* и *ветвление* управляют порядком выполнения других команд в программе и относятся к командам управления. Использование алгоритмической

Перед человеком постоянно возникают разнообразные задачи, для которых существуют различные алгоритмы решения. При всём многообразии алгоритмов для их записи достаточно трёх алгоритмических конструкций (структур): *следование*, *цикл*, *ветвление*.

Это положение было выдвинуто в середине 1970-х гг. учёным Эдсгером Вибе Дейкстрой (1930–2002 гг.). Его труды оказали влияние на развитие информатики и информационных технологий. Дейкстра являлся одним из разработчиков концепции структурного программирования, участвовал в разработке языка программирования Алгол. Известен своими разработками в области математической логики и теории графов.

Правообладатель Адукацыя і выхаванне

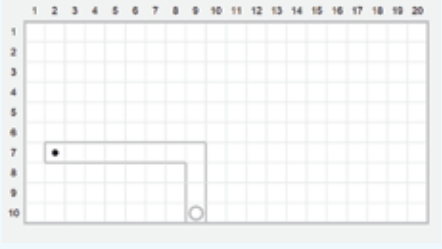
Пример 16.1. Одна из возможных начальных обстановок.



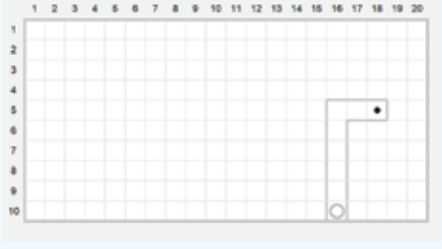
Программа для исполнителя *Робот*:

```
from pyrob.api import *
@task
def prim_16_1():
    while not wall_right():
        if wall_up():
            fill_cell()
        move_right()
    if wall_up():
        fill_cell()
run_tasks()
```

Пример 16.2. Одна из возможных начальных обстановок.



Другая обстановка:



конструкции *следование* предполагает отсутствие управляющих конструкций.

Рассмотрим примеры алгоритмов, содержащих несколько алгоритмических конструкций.

Пример 16.1. Написать программу для закрашивания некоторых клеток на поле. Окончание движения *Робота* определяет стена справа. По пути нужно закрасить те клетки, над которыми есть стена.

Для решения задачи *Робот* при движении вправо должен проверять каждую клетку на своём пути. Если условие «сверху стена» выполняется, то *Робот* закрашивает эту клетку. После проверки клетки *Робот* сдвигается вправо. Эти действия выполняются в цикле, пока справа нет стены.

После цикла необходима команда *ветвление*, так как для крайней клетки поля условие «нет стены справа» является ложным и клетка в цикле не закрашивается.

В этой задаче внутри конструкции цикла используется конструкция *ветвление*.

Пример 16.2. *Робот* должен дойти до конца «коридора» переменного размера. В «коридоре» может быть поворот влево или вправо.

Для решения задачи *Робот* сначала перемещается вверх, до тех пор, пока истинно условие «нет стены сверху». Стена, появившаяся сверху, означает, что начался поворот «коридора». Если слева нет стены, то в «коридоре» надо поворачивать влево, иначе в «коридоре» *Робот* поворачивает вправо. Дальше он двигается в выбранном направлении, пока не дойдёт до стены.

При решении данной задачи используется сначала конструкция *цикл*, а затем конструкция *ветвление*. В команде *ветвление* каждая последовательность команд представляет собой цикл.

Пример 16.3*. *Роботу* необходимо переместиться из верхнего левого угла поля в нижний левый угол. При этом на поле присутствуют стены. *Робот* начинает движение вправо до правой границы поля и спускается на одну клетку вниз. Затем движение продолжается влево до левой границы поля, после — спуск на одну клетку. Эти действия *Робот* должен повторить 4 раза.

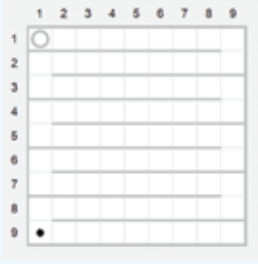
В данной задаче внутри цикла с параметром используются два других цикла с предусловием.

Пример 16.2. *Продолжение.*

Программа для исполнителя *Робот*:

```
from pyrob.api import *
@task
def prim_16_ 2():
    while not wall_up():
        move_up()
    if wall_left():
        while not wall_right():
        move_right()
    else:
        while not wall_left():
        move_left()
run_tasks()
```

Пример 16.3*. Начальная обстановка.

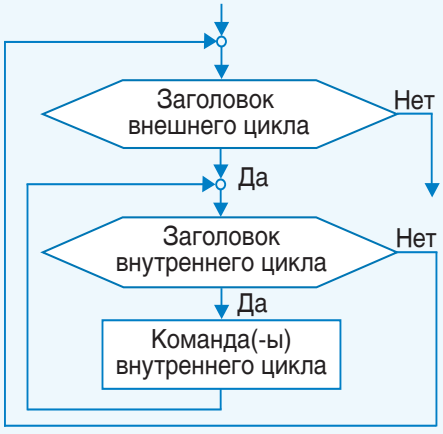


Программа для исполнителя *Робот*:

```
from pyrob.api import *
@task
def prim_16_ 3():
    for i in range (4):
    while wall_down():
        move_right()
        move_down()
    while wall_down():
        move_left()
        move_down()
run_tasks()
```

Правообладатель Адукацыя і выхаванне

Пример 16.4. Блок-схема вложенных циклов.



Структуру, когда внутри одного цикла выполняется другой цикл, называют **вложенным циклом.**

Один из возможных вариантов вложенных циклов представлен на блок-схеме в примере 16.4.

Как видно из примеров, базовые алгоритмические структуры можно комбинировать друг с другом так, как этого требует алгоритм решения задачи.

1. Назовите базовые алгоритмические конструкции.
2. Какие базовые алгоритмические конструкции относят к элементам управления?
3. Приведите примеры использования базовых алгоритмических конструкций.
4*. Что такое вложенный цикл?

Упражнения

1 Объясните, какие алгоритмические конструкции используются в приведённых ниже программах. Нарисуйте блок-схемы данных алгоритмов. Предложите пример начальной обстановки, в которой алгоритм выполнится корректно.

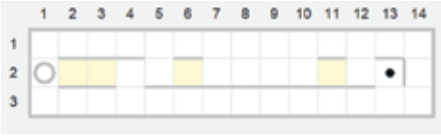
а)

```
from pyrob.api import *
@task
def upr_16_1_a():
    while wall_left():
        fill_cell()
        move_down()
run_tasks()
```

б)

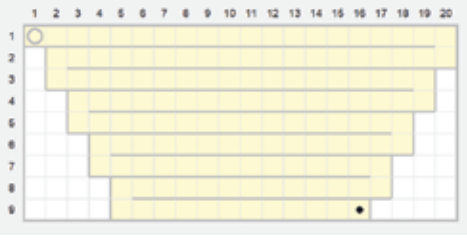
```
from pyrob.api import *
@task
def upr_16_1_b():
    while cell_is_filled():
        if not wall_left():
            move_left()
run_tasks()
```

2 Для решения задачи upr_16_2 Миша написал программу, но она работает неправильно. Какие ошибки допустил Миша?



```
from pyrob.api import *
@task
def upr_16_2():
    while  wall_right():
        if wall_up() or wall_down():
            fill_cell()
        move_right()
    if wall_up() and wall_down():
        fill_cell()
run_tasks()
```

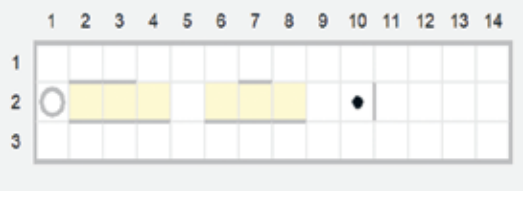
3 Заполните пропуски (многоточие на жёлтом фоне) в программе решения задачи upr_16_3 так, чтобы она работала верно.

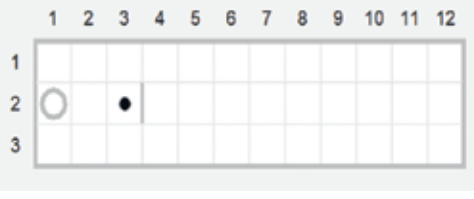


В строке поля *Робота* может быть произвольное количество клеток.

```
from pyrob.api import *
@task
def  upr_16_3():
     for  i  in  range  (4):
          fill_cell()
          while  ...:
               move_right()
               fill_cell()
          move_down()
          fill_cell()
          while  ...:
               move_left()
               fill_cell()
     ...
     while  not  wall_right():
          move_right()
          fill_cell()
run _tasks()
```

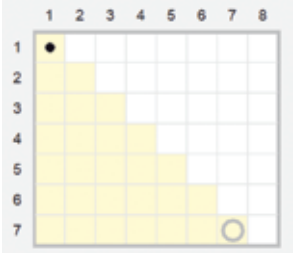
4 Решите задачу upr_16_4, используя внутри цикла команду *ветвление*.





Правообладатель Адукацыя і выхаванне

5* Решите задачу `upr_16_5`, используя вложенные циклы. Размер поля заранее не известен.



6* Придумайте задачу для исполнителя *Робот*, в которой будут использоваться различные алгоритмические конструкции.

§ 17. Язык программирования Python

Компьютер (англ. computer — *вычислитель*) — устройство (или система), способное выполнять заданную чётко определённую изменяемую последовательность операций. Чаще всего это операции численных расчётов.

Гвидо ван Россум приступил к созданию языка Python в декабре 1989 г. в центре математики и информатики в Нидерландах. Ван Россум является основным автором Python. Он продолжал выполнять центральную роль в принятии решений относительно развития языка вплоть до 12 июля 2018 г.

Желание упростить расчёты присуще человеку с древних времён. Создавая различные приспособления для счёта, человек прошёл долгий путь. Сегодняшний компьютер способен выполнять сотни миллионов операций в секунду. Для решения вычислительных задач требуется сначала составить алгоритм, а затем записать его в виде программы, используя какой-либо язык программирования.

Язык программирования устанавливает набор правил, определяющих внешний вид программы и действия, которые выполнит исполнитель под её управлением.

Сегодня язык Python является одним из самых популярных и востребованных языков программирования. Существует большое количество сред программирования с поддержкой языка Python: IDLE, PyCharm, Spyder, Thonny, Eclipse + PyDev, Visual Studio и др. В учебном курсе используется среда IDLE, с которой вы работали, знакомясь с учебными компьютерными исполнителями.

17.1. Команда вывода

Демонстрировать работу любой программы имеет смысл только тогда, когда она выводит какую-либо информацию.

Пример 17.1. Написать программу для вывода слов «Привет, мир!».

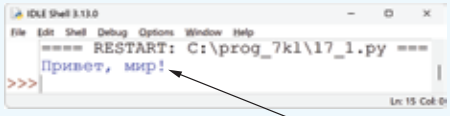
```
print('Привет, мир!')
```

Результат работы программы отражается в главном окне среды IDLE.

Команда `print()` предназначена для **вывода данных**. Она является функцией, аргументы которой — выводимые значения.

Аргументами функции `print` могут быть разделители (пример 17.2).

Пример 17.1. Окно среды IDLE с результатом работы программы.



Результат работы программы

По традиции, начавшейся в 1978 г. с примера из книги Б. Кернигана и Д. Ритчи «Язык программирования Си», первая программа на любом языке программирования должна просто выводить на экран приветствие миру.

Пример 17.2. Аргументы команды `print()` для управления разделителями.

`sep` — задаёт текст, который будет служить разделителем при выводе различных значений в одной команде `print()`. По умолчанию — пробел.

`end` — задаёт текст, который появится на экране после вывода всех значений одной команды `print()`. По умолчанию — перевод строки.

В программе перевод строки задаётся как `'\n'`. Если использовать `sep = '\n'`, то каждое выводимое значение одной команды `print()` будет расположено в новой строке.

Аргументы, управляющие выводом, могут использоваться в команде `print()` по одному или оба сразу.

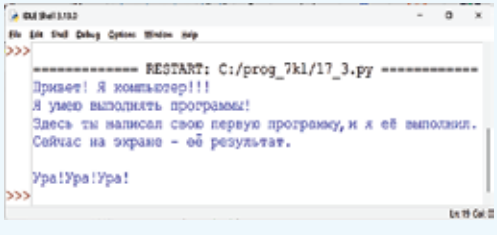
Правообладатель Адукацыя і выхаванне

У функции `print` есть ещё два необязательных аргумента: `file` (определяет, куда выводятся данные, по умолчанию — на экран) и `flush` (определяет, нужно ли принудительно очищать поток вывода, по умолчанию — `false`).

Пример 17.3. Текст программы.

```
print('Привет!',
      'Я компьютер!!!')
print('Я умею выполнять',
      end = ' ')
print('программы!', 'Здесь',
      sep = '\n', end = ' ')
print('ты', 'написал свою',
      end = ' ')
print('первую программу,',
      end = ' ')
print('и я её выполнил.')
print('Сейчас на экране',
      end = ' - ')
print('её результат.')
print()
print('Ура!', 'Ура!', 'Ура!',
      sep = '!')
```

Результат работы программы:



Текст, который нужно вывести на экран, заключают в кавычки. Кавычки могут быть одинарными или двойными. Текст выводится на экран в том виде, в котором он записан в программе. Его можно записать как на русском, так и на любом другом языке. Текстом может быть произвольный набор символов.

В программе может быть несколько команд вывода. После выполнения команды вывода курсор переводится на следующую строку. В одной команде можно указать вывод нескольких значений, которые отделяются друг от друга запятыми. Если функция `print()` используется без аргументов, то будет выведена пустая строка.

Пример 17.3. Вывести на экран компьютера следующий текст, используя различные значения разделителей: «Привет! Я компьютер!!! Я умею выполнять программы! Здесь ты написал свою первую программу, и я её выполнил. Сейчас на экране — её результат. Ура! Ура! Ура!».

Используя сочетания аргументов `sep` и `end`, можно управлять размещением текста на экране.

Текст в команде print(), записанный в кавычках, не анализируется. Если кавычки опустить, то производится анализ тех данных, которые записаны в скобках. Например, если в скобках написать арифметическое выражение, то сначала вычисляется его значение, а затем выводится результат.

Пример 17.4. Посчитать значение выражения $2 + 2 \cdot 2$.

Если в команде print() записать выражение в кавычках, то на экран будет выведено само выражение, при отсутствии кавычек — значение данного выражения.

Пример 17.4. Текст программы.

```
print('2 + 2 * 2 =')
print(2 + 2 * 2)
```

Результат работы программы:

```
2 + 2 * 2 =
6
```

Для вывода результата в одной строке программу нужно записать так:

```
print('2 + 2 * 2 =', end = ' ')
print(2 + 2 * 2)
```

Или так:

```
print('2 + 2 * 2 =',2 + 2 * 2)
```

Результат работы программы:

```
2 + 2 * 2 = 6
```

17.2. Понятие типа данных

На практике редко приходится писать программы, которые решают только одну задачу. Обычно программы создаются для решения целого класса задач, которые можно сформулировать в общем виде.

С такими задачами вы уже сталкивались в курсе математики. Например, решение задачи «Найти площадь прямоугольника» можно записать так: $S = a \cdot b$, где переменные a и b обозначают соответственно длину и ширину прямоугольника, а S — площадь.

Программисты ЭВМ до начала 1950-х гг. при создании программ пользовались машинным кодом. Запись программы на машинном коде состояла из единиц и нулей. Типы данных не использовались. Машинный код принято считать языком программирования первого поколения.

Первым языком программирования, в котором появилась возможность создавать переменные, считается Ассемблер. В этом языке вместо машинного кода стали использовать команды, записанные текстом. Ассемблер относится к языкам программирования второго поколения.

Правообладатель Адукацыя і выхаванне

Зная эту формулу, можно найти площадь любого прямоугольника.

В программировании для решения задач в общем виде также используют переменные. Для работы компьютера с переменными они должны храниться в его памяти.

Информацию, представленную в формализованном виде и пригодную для обработки на компьютере, называют **данными.**

Переменная в программировании — это именованная ячейка памяти, хранящая значение переменной.

Компьютер может обрабатывать данные разных типов: целые и рациональные числа, строки и др.

Тип данных определяет способ их хранения в памяти компьютера, диапазон возможных значений и операции, которые можно выполнять с этим типом данных.

Каждая переменная задаётся своим именем. Для обозначения имени переменной используют буквы латинского алфавита, арабские цифры и знак «_».

В 1957 г. появился язык Фортан, открывший эру языков программирования третьего поколения. Он позволил использовать разные числовые типы данных, необходимые для сложных расчётов: целые, вещественные (действительные) и комплексные (комплексные числа изучаются на факультативах по математике в старших классах).

Дальнейшее развитие языков программирования позволило добавить возможность работы с другими типами данных. Нынешние языки программирования поддерживают возможность работы с большим количеством разных типов данных.

Пример 17.5. Числовые типы данных.

Язык Python поддерживает работу с целыми и рациональными числами. Значения переменных могут определяться так же, как в математике.

```
a = 2
b_1 = 7.5
radius = 12.0
```

Переменная `a` будет определена как целая, а переменные `b_1` и `radius` — как рациональные. Целая часть дробного числа отделяется от дробной части точкой.

Заглавные и строчные буквы считаются различными.

При использовании переменной выделяется память для хранения значения этой переменной. В процессе выполнения программы значение переменной может изменяться. Тип переменной в языке Python определяется по её значению (пример 17.5 на с. 124). Это происходит в процессе выполнения программы.

17.3. Оператор присваивания

Одной из основных команд для обработки данных в программе является оператор присваивания.

Оператор присваивания предназначен для того, чтобы:

- задавать значения переменных;
- вычислять значение арифметического выражения, результат вычисления которого будет записан как значение переменной.

Формат записи оператора присваивания:

<имя переменной> = <выражение>

В записи арифметического выражения используются знаки математических действий: сложения, вычитания, умножения, деления

Пример 17.5. *Продолжение.*

Задавать значения можно и таким образом:

```
a, b_1, radius = 2, 7.5, 12.0
```

Чтобы убедиться, что переменные получили указанные значения, выведем их с помощью команды `print()`:

```
print(a, b_1, radius)
```

Результат работы программы:

```
2  7.5  12.0
```

Пример 17.6. Запись оператора присваивания.

Запись оператора присваивания в программе и вывод значений переменных:

```
x = 9
x1 = 3.5
a_1 = 20 * (x + x1) - 32
y = 3
  chastnoe = x / y
y = 7 + 2 * y ** 3
print('x =', x)
print('x1 =', x1)
print('a_1 =', a_1)
print('chastnoe =',
      chastnoe)
print('y =', y)
```

Результат работы программы:

```
x = 9
x1 = 3.5
a_1 = 218.0
chastnoe = 3.0
y = 61
```

Правообладатель Адукацыя і выхаванне

Если в выражении встречаются переменные разных числовых типов (как целых, так и рациональных), то результат будет рациональным числом (значение переменной `a_1`).

Пример 17.7. Запись оператора присваивания на Python для математических выражений:

Выражение	Запись на Python
$S = 2(a + b)$	`S = 2 * (a + b)`
$S = a^2$	`S = a * a` или `S = a ** 2`
$a = \frac{x + y}{3}$	`a = (x + y) / 3`

Пример 17.8*. Изменение значения переменной.

В Python допустимы команды присваивания следующего вида:

```
a = a * 2 или a *= 2
```

Смысл команды следующий: из ячейки памяти извлекается значение переменной a, затем оно умножается на 2, результат записывается в ту же ячейку памяти. Старое значение переменной a будет потеряно.

Запись оператора присваивания для изменения значения переменной b следующая:

```
b = b – 3 или b –= 3
```

и возведение в степень. В языке Python им соответствуют следующие символы:

Математические операции	Запись в Python
+ (сложение)	+
– (вычитание)	–
· (умножение)	*
: (деление)	/
возведение в степень	**

Приоритет выполнения операций соответствует принятому в математике: сначала выполняется возведение в степень, затем умножение и деление, далее — сложение и вычитание. В выражениях для изменения порядка действий можно использовать скобки (примеры 17.6 и 17.7).

Для записи обыкновенной дроби используется знак деления. Знак умножения опускать нельзя. Вычисления, производимые над целыми числами, всегда точные. При вычислениях с рациональными числами результат может быть приблизительным.

Пример 17.8*. Записать оператор присваивания, после выполнения которого значение переменной a увеличится в два раза, а переменной b уменьшится на 3.

17.4. Ввод данных

Начальные значения переменных можно задавать не только с помощью оператора присваивания, но и вводить с клавиатуры. В этом случае текст программы не нужно изменять для нового набора данных.

Команда `input()` предназначена для **ввода данных**. Для того чтобы значение переменной вводилось с клавиатуры, нужно присвоить ей значение функции `input()`.

При использовании в программе команды ввода данных в виде `a = input()` все символы, вводимые с клавиатуры, при выполнении команды воспринимаются как текст (в том числе и числа). Поэтому в команде ввода нужно указать, в какой числовой тип данных мы хотим преобразовать вводимый текст (пример 17.9).

В качестве параметра функции `input()` можно задать текст, который будет служить подсказкой при вводе данных (пример 17.10).

Ввод данных осуществляется в главном окне среды IDLE (пример 17.10). После завершения работы программы в этом же окне будет выведен результат.

Пример 17.9. Ввод чисел.

Для преобразования в целое число используется функция `int()`, а в рациональное число — функция `float()`.

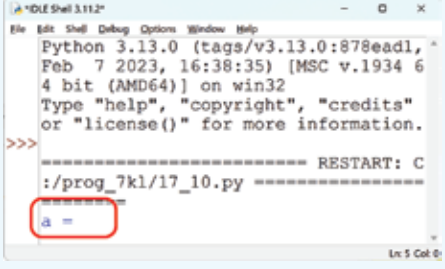
```
x = int(input('целое'))
y = float(input('рациональное'))
```

Пример 17.10. Ввести два рациональных числа, найти их сумму.

Текст программы:

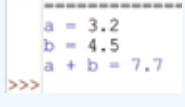
```
a = float(input('a = '))
b = float(input('b = '))
s = a + b
print('a + b =', s)
```

После запуска программы в главном окне среды IDLE появится строка `a =`.



В этой строке нужно набрать числовое значение. Обратите внимание на то, что программа выводит текст синим цветом, а значения, вводимые пользователем, отображаются чёрным.

Результат работы программы:



Правообладатель Адукацыя і выхаванне

1. Какая команда языка программирования Python предназначена для вывода данных?
2. Как можно изменить разделитель при выводе данных?
3. Что такое переменная?
4. Что определяет тип данных?
5. Для чего используется команда присваивания?
6. Какая команда языка программирования Python предназначена для ввода данных?

Упражнения

1 Выполните следующие задания для программы из примера 17.3 на с. 122 (файл с программой можно скачать).

1. Замените все аргументы `sep` на `end` и выполните программу. Что произошло? Объясните почему.

2. Как изменится результат работы программы, если в исходном тексте заменить все аргументы `end` на `sep`?

3. Измените программу так, чтобы текст на экране выглядел следующим образом:

```
Привет! Я компьютер!!! Я умею выполнять программы!
Ты сегодня написал свою первую программу!!!
Я выполнил твою программу. Посмотри на результат!
```

2 Для примера 17.4 на с. 123 внесите изменения в программу так, чтобы действия выполнялись в том порядке, в котором записаны, то есть сначала сложение, а потом умножение.

3 Составьте программу для определения возраста пользователя через 5 лет и выведите результат. Возраст пользователя вводится в годах.

4* Составьте программу, которая позволяет ввести два числа *a* и *b*, затем первое число уменьшает в два раза, а второе увеличивает на 30. Выведите изменённые значения переменных.

5 Напишите программу для вычисления значения числового выражения.

1) $23 + 45 \cdot 11 - 15$. 2) $\frac{37 + 2^5}{41}$. 3) $\frac{5638 - 2347}{49} + \frac{12^3 \cdot (7 + 56)}{455}$.

§ 18. Организация вычислений

При решении любой задачи человеку приходится выполнять следующие действия:

- определение исходных данных (что дано в задаче);
- определение результатов (что нужно получить);
- обработка исходных данных в соответствии с известными правилами так, чтобы получить результат;
- проверка и анализ полученного результата.

Решения задач по физике и химии принято оформлять соответствующим образом (пример 18.1).

Применив данные правила к решению задачи по программированию, получим следующие этапы:

I. Определение исходных данных.

II. Определение результатов.

III. Составление алгоритма решения задачи.

IV. Определение типов данных для переменных, используемых при реализации алгоритма.

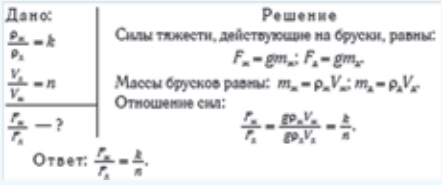
V. Написание программы.

VI. Тестирование программы.

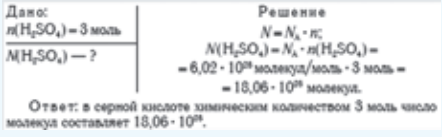
VII. Анализ результатов.

Пример 18.1. Оформление задач.

Физика:



Химия:



Слева записывается, что дано (определение исходных данных) и что нужно получить (определение результатов), справа — последовательность действий, приводящая к решению задачи (обработка исходных данных). Проверка и анализ полученного результата проводятся вручную.

В программировании, в отличие от физики, химии и др., проверка правильности работы программы производится с помощью компьютерного тестирования.

Для анализа данных можно, например, произвести вычисления на калькуляторе. В промышленном программировании для проверки правильности работы одной программы часто пишется другая программа.

Правообладатель Адукацыя і выхаванне

Пример 18.2. Этапы решения задачи по программированию.



Пример 18.3. Решение.

V. Программа:

```
x = float(input('x = '))
y = float(input('y = '))
z = float(input('z = '))
a = (2 * x + y - z) / (3 + x * x)
print('a =', a)
```

В примере 18.2 описанные этапы отображены в виде блок-схемы.

Тестирование программы — это проверка правильности работы программы при разных наборах исходных данных.

Анализ данных предполагает проверку правильности результата работы программы. Он может выполняться различными способами: вручную, с помощью технических средств, программно.

Для каждой задачи, рассматриваемой далее, будем описывать все этапы решения.

18.1. Вычисление значения арифметического выражения

Пример 18.3. Даны x, y, z. Написать программу для вычисления значения выражения $a = \dfrac{2x + y - z}{3 + x^2}$.

Этапы выполнения задания:

I. Определение исходных данных: переменные x, y, z.

II. Определение результатов: переменная a.

III. Алгоритм решения задачи.

1. Ввод исходных данных.

2. Вычисление значения выражения.

3. Вывод результата.

IV. Описание переменных: все переменные, определённые для решения задачи, имеют тип `float`.

В приведённом примере для каждой команды ввода записан текст с пояснениями о том, значение какой переменной нужно вводить.

При написании программ для вычисления значения арифметического выражения часто допускают следующие ошибки (пример 18.4)

18.2. Использование языка программирования для решения задач

Пример 18.5. Напишите программу для решения геометрической задачи. Дан квадрат с длиной стороны a. Найдите его площадь и периметр.

Этапы выполнения задания:

I. Определение исходных данных: переменная a (длина стороны).

II. Определение результатов: переменные S (площадь) и P (периметр).

III. Алгоритм решения задачи.

1. Ввод исходных данных.

Вычисление значений площади производится по формуле $S = a^2$, а периметра — по формуле $P = 4a$.

Пример 18.3. *Продолжение.*

VI. Тестирование программы.

Запустите программу и введите значения: $x = 2$, $y = 3.1$, $z = 4$. Проверьте, результат должен быть следующим:

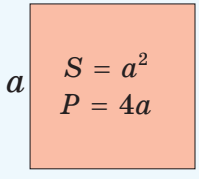
```
x = 2
y = 3.1
z = 4
a = 0.4428571428571428
```

VII. Проверить правильность вычислений можно на калькуляторе. Все данные в программе были определены как рациональные числа. При тестировании допускается ввод и целых значений (переменные x и z), так как целые числа являются подмножеством рациональных.

Пример 18.4. Часто допускаемые ошибки.

`2x + 3` `(x – 4)(x + 2)`	Пропущен знак *
`(2 + y) / (x * x`	Пропущена скобка

Пример 18.5. Решение.



Правообладатель Адукацыя і выхаванне

Пример 18.5. *Продолжение.*

V. Программа:

```
a = float(input('a = '))
S = a ** 2
P = 4 * a
print('площадь =', S)
print('периметр =', P)
```

VI. Тестирование программы.

Запустите программу и введите значение $a = 5.17$.

Проверьте, результат должен быть следующим:

```
a = 5.17
площадь = 26.7289
периметр = 20.68
```

VII. Правильность вычислений можно проверить на калькуляторе.

Пример 18.6. Решение.

V. Программа:

```
s = float(input('s = '))
t = float(input('t = '))
v = s / t
print('скорость =', v)
```

VI. Тестирование программы.

Запустите программу и введите значения $s = 3550$ и $t = 4$.

Проверьте, результат должен быть следующим:

```
s = 3350
t = 4
скорость = 837.5
```

VII. Правильность вычислений можно проверить на калькуляторе.

В программе этим формулам будут соответствовать команды присваивания `S = a ** 2` и `P = 4 * a`.

2. Вывод результата.

IV. Описание переменных: все переменные имеют тип `float`.

Обратите внимание на запись операторов присваивания, соответствующих математическим формулам.

Пример 18.6. Напишите программу для решения физической задачи. Расстояние между двумя городами s км. Самолёт пролетает это расстояние за t ч. Определите скорость самолёта.

Этапы выполнения задания.

I. Определение исходных данных: переменные s (расстояние) и t (время).

II. Определение результатов: переменная v (скорость).

III. Алгоритм решения задачи.

1. Ввод исходных данных.

2. Согласно формуле расстояния $s = vt$. Отсюда выразим v как $v = \frac{s}{t}$.

3. Вывод результата.

IV. Описание переменных: все переменные имеют тип `float`.

В программе можно использовать **комментарии** — текст, который не анализируется при запуске программы на выполнение.

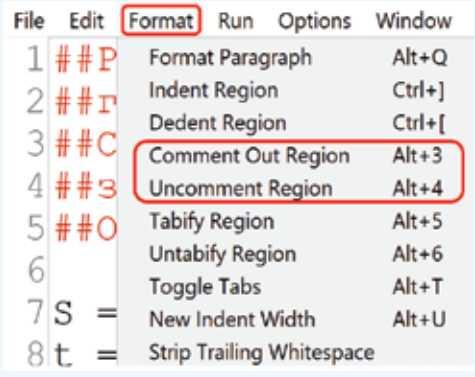
Текст после символов ## считается комментарием и выделяется на экране красным цветом. В комментариях удобно записывать условие задачи и пояснения к командам (пример 18.7).

Для того чтобы сделать комментарием сразу несколько строк, нужно их выделить и выполнить команду Format → Comment Out Region или воспользоваться комбинацией клавиш Alt + 3. Команда вставит в начале каждой выделенной строки знаки ##. Команда Format → Uncomment Region (Alt + 4) превращает комментарии в обычный текст (пример 18.8).

Пример 18.7. Программа с комментариями.

```
File Edit Format Run Options Window Help
1 ##Расстояние между двумя
2 ##городами s км.
3 ##Самолет пролетает это расстояние
4 ##за t часов.
5 ##Определите скорость самолета.
6
7 s = float(input('s = '))
8 t = float(input('t = '))
9 #вычисление скорости
10 v = s / t
11 print('скорость =', v)
12
                                Ln: 12 Col: 0
```

Пример 18.8. Меню Format.



1. Перечислите основные этапы решения задачи по программированию.

2. Что понимают под тестированием программы?

3. Для чего можно использовать комментарии?

Упражнения

1 Напишите программу для вычисления значения арифметического выражения.

1) $a = \dfrac{x + y - z}{x^2 + 2}$. 2) $a = 5 + \dfrac{2x - z}{3 + y^3}$. 3) $a = (1 + z)\dfrac{x + \dfrac{y}{x^2 + 4}}{2 + \dfrac{1}{x^4 + 4}}$.

Правообладатель Адукацыя і выхаванне

2 Напишите программу для решения геометрической задачи.

1. Найдите длину окружности и площадь круга заданного радиуса.

2. Найдите угол при вершине равнобедренного треугольника, если известны величины углов при основании треугольника.

3*. Найдите углы при основании равнобедренного треугольника, если известен угол при вершине.

3 Напишите программу для решения физической задачи.

1. Велосипедист едет с постоянной скоростью v км/ч. За сколько минут он проедет путь в s километров?

2. Давление воды в трубе водопровода на первом этаже здания составляет p кПа. Определите, на какой высоте находится поверхность воды в водонапорном баке относительно крана, расположенного на первом этаже. Плотность воды $\rho = 1000$ кг/м^3, ускорение свободного падения $g = 10$ м/с^2.

3*. Автомобиль проходит первую часть пути длиной s_1 км за t_1 минут, участок пути длиной s_2 км за t_2 минут и, наконец, участок длиной s_3 км за t_3 минут. Найдите среднюю скорость движения автомобиля, выраженную в км/ч.

4 Напишите программу для решения химической задачи.

1. В организме человека на долю атомов кислорода приходится 65 % от массы тела. Найдите массу атомов кислорода для своей массы тела.

2. Рассчитайте массу сахара, который нужно растворить в воде массой 250 г, для получения раствора с массовой долей сахара, равной 5 %.

3*. Масса одного атома кислорода $2{,}656 \cdot 10^{-26}$ кг (это число в программе на языке Python записывается как 2.656E-26, буква E — английская). Определите, сколько атомов кислорода содержится в вашем теле.

§ 19. Реализация алгоритмов работы с целочисленными данными

19.1. Целочисленный тип данных

Часто при решении задач используются целые числа. В языке Python целые числа могут быть сколь угодно большими. Для этого типа данных определены следующие операции:

Математические операции	Запись в Python
+ (сложение)	+
− (вычитание)	−
· (умножение)	*
целочисленное деление	//
нахождение остатка	%
возведение в степень	**

Для целочисленных данных не определена операция деления (как для рациональных чисел). Если использовать эту операцию, то результат всегда будет рациональным, даже если одно число делится на другое без остатка.

Для организации вычислений с целыми числами определены операции // и %. Значения результатов операций // и % могут отличаться от принятых в математике (пример 19.1).

Чтобы облегчить визуальную оценку величины числа, в Python, начиная с версии 3.6, между цифрами разрешается вставлять одиночные символы подчёркивания. Запись `a = 1000000` эквивалентна записи `a = 1_000_000`. Такую запись можно использовать как при написании текста программы, так и при вводе чисел с клавиатуры.

Пример 19.1. Результат операций // и % для чисел разных знаков.

a	b	a // b	a % b
17	3	5	2
–17	3	–6	1
17	–3	–6	–1
–17	–3	5	–2

При выполнении операции // знак результата определяется так же, как в математике: он положительный, если исходные числа одного знака, и отрицательный, если разного. Значение результата округляется до ближайшего целого в меньшую сторону.

Правообладатель Адукацыя і выхаванне

Пример 19.1. *Продолжение.*

Результат операции % может быть отрицательным, хотя в математике под остатком понимают неотрицательное число. Если остаток не равен нулю, то знак числа, которое является результатом операции %, определяется знаком делителя. При анализе данных из таблицы можно понять, что

`a % b = a - (a // b) * b`.

Пример 19.2. Решение.

V. Программа:

```
a = int(input('a = '))
b = int(input('b = '))
c = a // b
d = a % b
print('целая часть =', c)
print('остаток =', d)
```

VI. Тестирование программы.

Запустите программу и введите значения $a = 11$ и $b = 4$.

Проверьте, результат должен быть следующим:

```
a = 11
b = 4
целая часть = 2
остаток = 3
```

Пример 19.3. Решение.

V. Программа:

```
c = int(input('c = '))
#минуты
m = c // 60
#секунды
s = c % 60
print(m, s, sep = ':')
```

Эти операции имеют такой же приоритет, как и операции деления и умножения.

Пример 19.2. Даны два целых числа a и b. Напишите программу, которая находит целую часть от деления a на b и остаток от деления a на b.

Этапы выполнения задания.

I. Определение исходных данных: переменные a и b.

II. Определение результатов: переменные c (целочисленное частное) и d (остаток).

III. Алгоритм решения задачи.

1. Ввод исходных данных.

2. Целочисленное частное находим как результат операции `a // b`, остаток — `a % b`.

3. Вывод результата.

IV. Описание переменных: все переменные имеют тип `int`.

19.2. Использование целочисленных данных для решения задач

Пример 19.3. Пусть таймер показывает время только в секундах. Написать программу, которая переведёт время в минуты и секунды.

Этапы выполнения задания.

I. Определение исходных данных: переменная c (время в секундах).

II. Определение результатов: переменные m (полное количество минут) и s (остаток секунд).

III. Алгоритм решения задачи.

1. Ввод исходных данных.

2. Для нахождения полного числа минут нужно найти целую часть от деления исходного числа секунд на 60.

3. Оставшиеся секунды можно найти как остаток от деления исходного числа секунд на 60.

4. Вывод результата.

IV. Описание переменных: все переменные имеют тип `int`.

Пример 19.4. Задано положительное двузначное число. Написать программу, которая поменяет местами первую и вторую цифры числа.

Этапы выполнения задания.

I. Определение исходных данных: переменная a (исходное число).

II. Определение результатов: переменная b (преобразованное число).

III. Алгоритм решения задачи.

1. Ввод исходных данных.

2. Для преобразования числа необходимо выполнить следующие действия:

Пример 19.3. *Продолжение.*

VI. Тестирование программы.

Запустите программу и введите значение $c = 137$. Результат должен быть следующим:

```
c = 137
2:17
```

Для $c = 24$ получим:

```
c = 24
0:24
```

Операция нахождения остатка в разных языках программирования реализуется по-разному для отрицательного делимого или делителя. В некоторых языках такая операция не определена (Basic) или остаток всегда неотрицательный, как в математике (Oberon). В других языках остаток может быть отрицательным. Возможны два варианта:

- знак остатка совпадает со знаком делимого (C++, C#, Pascal);
- знак остатка совпадает со знаком делителя (Python, Prolog).

Есть языки, в которых реализованы оба варианта (Java, Haskell). В этом случае операции обозначаются по-разному. Для обозначения операции целочисленного деления часто используют \, //, `div`, `quot` или знак, которым также обозначают деление для рациональных чисел, /. Для операции нахождения остатка используют %, \\, `mod`, `rem`. Могут использоваться другие обозначения.

Правообладатель Адукацыя і выхаванне

Пример 19.4. Решение.

V. Программа:

```
a = int(input('a = '))
#выделение последней цифры
a1 = a % 10
#выделение первой цифры
a2 = a // 10
b = a1 * 10 + a2
print('результат =', b)
```

VI. Тестирование программы.

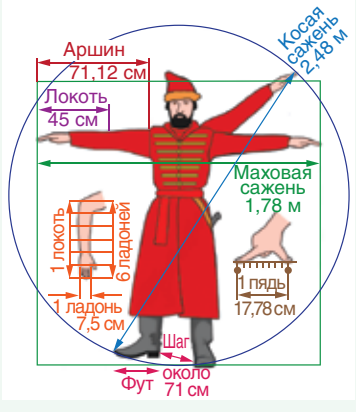
Запустите программу и введите значение $a = 25$.

Проверьте, результат должен быть следующим:

```
a = 25
результат = 52
```

На Руси традиционно применялась русская система мер. Сегодня, несмотря на то, что официально принята метрическая система мер, старорусские названия мер используются в исторических исследованиях и фразеологических оборотах.

Например: 1 аршин = 16 вершков, 1 вершок ≈ 4,5 см.



1) в переменной $a1$ сохраним вторую цифру числа. Для выделения цифры из числа нужно найти остаток от деления исходного числа на 10 (`a % 10`);

2) для выделения первой цифры (переменная $a2$) нужно найти целую часть от деления числа на 10;

3) искомое число b получим, если умножим $a1$ на 10 и к полученному произведению прибавим значение переменной $a2$.

3. Вывод результата.

IV. Описание переменных: все переменные имеют тип `int`.

Пример 19.5. В исторической книге, которую читала Таня, длина отреза ткани измерялась в локтях. Напишите программу, которая переведёт локти в метры и сантиметры.

Этапы выполнения задания.

I. Определение исходных данных: переменная l (локти).

II. Определение результатов: переменные m (метры) и s (сантиметры).

III. Алгоритм решения задачи.

1. Ввод исходных данных.

2. Сначала переведём локти в сантиметры. Для этого количество локтей нужно умножить на 45 и сохранить значение в переменной x.

3. Для определения числа метров найдём целую часть от деления x на 100.

4. Оставшиеся сантиметры можно найти как остаток от деления x на 100.

5. Вывод результата.

IV. Описание переменных: все переменные имеют тип `int`.

В Python, как и в большинстве других языков программирования, запрещено деление на нуль. Если попытаться это сделать, появится сообщение об ошибке `ZeroDivisionError`.

Пример 19.5. Решение.

V. Программа:

```
l = int(input('l = '))
x = l * 45
#метры
m = x // 100
#сантиметры
s = x % 100
print(l, 'локтей =', end = ' ')
print(m, 'м', s, 'см')
```

V. Тестирование программы.

Запустите программу и введите значение $l = 7$. Проверьте, результат должен быть следующим:

```
l = 7
7 локтей = 3 м 15 см
```

?

1. Какой тип данных можно использовать в Python для работы с целочисленными данными?

2. Какие операции определены для целочисленных данных?

3. Как определяется результат целочисленной операции // для чисел с разными знаками?

4. Как определяется результат целочисленной операции % для чисел с разными знаками?

Упражнения

1 Вася написал программу, которая переводит длину из метров в километры и метры. Но он не может решить, где нужно использовать //, а где — %. Помогите ему. Откройте файл и исправьте программу.

```
d = int(input('d = '))
k = d ... 1000
m = d ... 1000
print(d, 'м =', end = ' ')
print(k, 'км', m, 'м')
```

Правообладатель Адукацыя і выхаванне

2 Ответьте на вопросы по примеру 19.4 на с. 137.

1. При каких значениях переменной a значение переменной b будет таким же?

2. Всегда ли в результате выполнения программы получается двузначное число? Почему?

3. Попробуйте ввести трёхзначное число (например, 125). Объясните получившийся результат.

3 Напишите программы для решения задач. Используйте операции // и %.

1. Задано положительное двузначное число. Найдите среднее арифметическое цифр числа.

2. Задано положительное двузначное число. Найдите разность между количеством десятков и единиц.

3. Дана масса в граммах. Переведите её в килограммы и граммы.

4. Площадь участка измеряется в арах. Выразите эту площадь в квадратных километрах.

5. Размер файла задан в килобайтах. Переведите его в мегабайты и килобайты.

4* Для старорусской системы ве́сов известны следующие соотношения:

1 берковец = 10 пудов = 400 фунтов = 38 400 золотников.

Напишите программу, которая переводит массу, заданную в золотниках, в фунты, пуды и берковцы.

Глава 4
РАБОТА С ВЕКТОРНОЙ ГРАФИКОЙ
§ 20. Понятие векторной графики

Одно из направлений использования компьютера — создание и обработка графических изображений. Например, схем, чертежей, рисунков, фотографий.

Компьютерная графика — область информатики, изучающая методы и средства создания и обработки изображений с помощью аппаратного и программного обеспечения компьютера.

В зависимости от способа представления в памяти можно рассматривать различные виды компьютерной графики. В 6-м классе вы познакомились с технологией создания растровых изображений. Такое изображение представляет собой совокупность пикселей, окрашенных в разные цвета.

Векторная графика — изображение в виде геометрических фигур (графических примитивов), описанных математическими формулами.

Рассмотрите пример 20.1.

Основные области применения компьютерной графики: научная, деловая, конструкторская, иллюстративная сферы.

Различия в представлении графической информации в растровом и векторном виде существенны только для хранения графических файлов и способов их обработки. На экран монитора можно вывести графическую информацию только в растровом виде.

Пример 20.1. Векторное изображение.



Векторное изображение можно сравнить с аппликацией, состоящей из кусочков цветной бумаги, наклеенных (наложенных) один на другой. Однако, в отличие от аппликации, в векторном изображении можно легко менять форму и цвет составных частей.

Правообладатель Адукацыя і выхаванне

Пример 20.2. Графические примитивы.

Название	Изображение
Линия	
Прямоугольник	
Эллипс	
Многоугольник, треугольник	
Звезда	

Пример 20.3. Интересной особенностью векторных редакторов является возможность изменения формы нарисованных от руки кривых. Также у векторных редакторов имеются средства расположения контуров относительно друг друга и команды спецэффектов.

Изменение формы кривых

Взаимное расположение объектов


Графические примитивы — простые геометрические фигуры: прямоугольник, окружность, эллипс, линия и т. д. (пример 20.2). Для составляющих векторное изображение графических примитивов с помощью математических формул описываются их форма, цвет и положение на холсте.

Графический примитив — независимый объект, который можно редактировать.

Положение и форма графических примитивов задаются в системе графических координат, которая связана с экраном. Начало координат расположено в верхнем левом углу. Ось *OX* направлена слева направо, ось *OY* — сверху вниз. Координатная сетка совпадает с сеткой пикселей.

Достоинства векторного изображения:

1) небольшой размер графического файла;

2) преобразования без искажений;

3) рисование осуществляется быстро и просто;

4) независимое редактирование частей изображения;

5) высокая точность прорисовки.

Однако в векторной графике практически невозможно достичь фотореалистичности.

Прочитайте и рассмотрите пример 20.3 на с. 142.

Для описания цвета изображений используются различные цветовые модели.

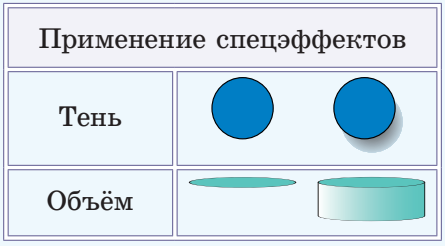
Под **цветовой моделью** понимается способ описания цвета.

Цветовые модели описывают цветовые оттенки с помощью смешивания нескольких основных цветов. Любой цвет можно разложить на оттенки и сопоставить с ним набор чисел — цветовых координат.

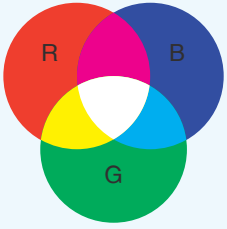
Основные цвета разбиваются на оттенки по яркости — от тёмного к светлому. Каждому оттенку присваивается числовое значение (например, самому тёмному — 0, самому светлому — 255).

Одна из наиболее распространённых цветовых моделей называется RGB (пример 20.4). Любой цвет в этой модели представляет

Пример 20.3. *Продолжение.*



Пример 20.4. Цветовая модель RGB.



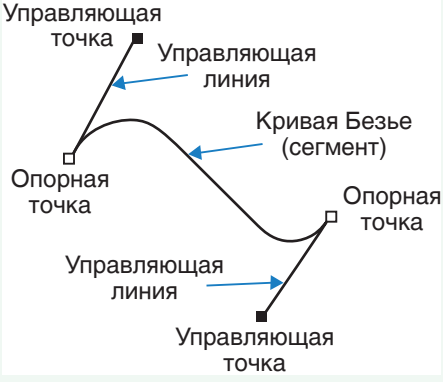
В прошлом инженеры, создавая чертежи больших деталей в натуральную величину, использовали тонкие планки, чтобы провести кривые по заданным точкам. Эти планки назывались сплайнами (гибкими лекалами).



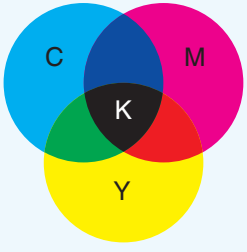
Сейчас в векторных редакторах тоже используются сплайновые кривые — кривые Безье. Своё название они получили в честь французского математика Пьера Безье (1910–1999 гг.).

Правообладатель Адукацыя і выхаванне

Учёный предложил описывать кривую, опираясь на вершины многоугольника, заключающего её в себе:



Пример 20.5. Цветовая модель CMYK.



Пример 20.6. Области применения векторной графики:

- промышленное проектирование;
- визуализация трёхмерных объектов;
- архитектура и строительство;
- ландшафтный дизайн;
- построение графиков поверхностей;
- полиграфия, реклама.

собой сложение трёх основных цветов: красного (Red), зелёного (Green) и синего (Blue). Именно на такой модели построено воспроизведение цвета современными мониторами и телевизорами.

В полиграфии и для цветных принтеров используется цветовая модель CMYK (пример 20.5). Основные цвета в ней — голубой (Cyan), пурпурный (Magenta), жёлтый (Yellow) и ключевой (Key) чёрный.

Уже знакомый вам графический редактор Paint предназначен для работы с растровой графикой. Растровые графические редакторы используют не только для создания изображений, но и для их обработки. Векторные редакторы ориентированы только на создание изображений. Однако векторная графика может включать в себя элементы растровой графики.

Векторные графические редакторы позволяют сохранять изображения в различных векторных форматах, среди которых можно выделить универсальные графические форматы

и форматы отдельных векторных редакторов.

Одним из недостатков векторной графики является программная зависимость. Изображение, созданное в одном векторном редакторе, как правило, не преобразуется в формат другой программы без погрешностей.

Программы векторной графики нашли широчайшее применение в области технического рисования, чертёжно-графических и оформительских работ, графического и полиграфического дизайна (пример 20.6 на с. 144). Одним из направлений применения векторной графики является создание логотипов (пример 20.7).

Векторные графические редакторы позволяют выполнять разнообразные операции над графическими объектами. Известны векторные редакторы: CorelDraw, Adobe Illustrator, Inkscape (пример 20.8).

Несмотря на разнообразие векторных графических редакторов, основные приёмы работы с векторными изображениями остаются неизменными.

Пример 20.7. Логотип Государственного знака качества.

Представляет собой пятиугольник красного цвета, в центре которого расположено стилизованное изображение перевёрнутой буквы К и надпись Беларусь. Пять углов знака символизируют качество белорусской продукции, достигаемое сочетанием пяти показателей производства: безопасности, экологичности, инновационности, технологичности и эстетичности.



Пример 20.8. Векторные графические редакторы.

Значок	Название
	CorelDraw
Ai	Adobe Illustrator
	Inkscape

CorelDraw и Adobe Illustrator — платные программы. Редактором Inkscape можно пользоваться бесплатно.

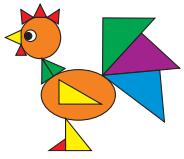
Правообладатель Адукацыя і выхаванне

1. Какая графика называется векторной?
2. Что такое цветовая модель?
3. Как воспроизводится цвет в цветовой модели?
4. На какой модели построено воспроизведение цвета мониторами?
5. Что такое графический примитив?
6. Как называются программы, позволяющие работать с векторной графикой?

Упражнения

1 Приведите примеры графических примитивов.

2 Определите, из каких графических примитивов составлено изображение петуха.

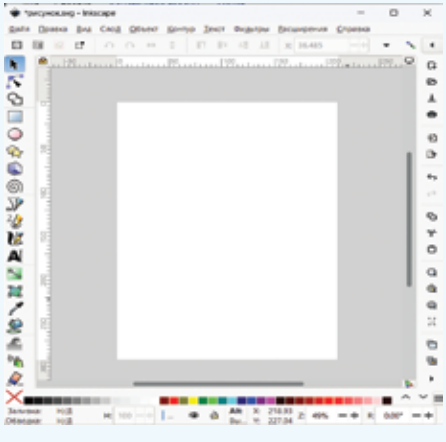


3 Составьте в тетради с помощью графических примитивов изображения:

1) домика;
2) цветка;
3) кошки.

§ 21. Интерфейс векторного графического редактора

Пример 21.1. Окно векторного графического редактора Inkscape.



Рассмотрим технологию работы с векторной графикой на примере редактора Inkscape (пример 21.1). Редактор имеет встроенный учебник на русском языке и удобный интерфейс, позволяющий легко и быстро освоить основные приёмы работы с векторной графикой.

Основную часть окна редактора Inkscape занимает **холст**, на котором пользователь создаёт и редактирует изображения. На холсте выделена **страница**.

Перемещаться по холсту можно при помощи полос прокрутки. Увеличение или уменьшение масштаба страницы осуществляется при помощи клавиш «+» или «−» на клавиатуре. Границы отображаемой на холсте страницы определяют границы изображения для печати или сохранения.

Многие действия в редакторе Inkscape можно выполнить несколькими способами:

- через пункты меню;
- при помощи кнопок на панелях;
- с помощью комбинаций клавиш.

Через меню **Правка** можно отменить последнее действие и повторить отменённое. Также доступна история действий.

Запуск программы осуществляется с помощью меню **Пуск: Все программы → Inkscape → → Inkscape** или двойным щелчком по ярлыку  на Рабочем столе. Для каждого документа редактор Inkscape открывает отдельное окно.

Чтобы сохранить изображение в редакторе Inkscape, в пункте меню **Файл** нужно выбрать **Сохранить как** и нажать кнопку **Сохранить.** Файл будет сохранён

Пример 21.2. Форматы файлов, с которыми может работать векторный редактор Inkscape.

SVG — собственный формат редактора Inkscape;

PNG — растровый формат изображений, поддерживающий прозрачность фона;

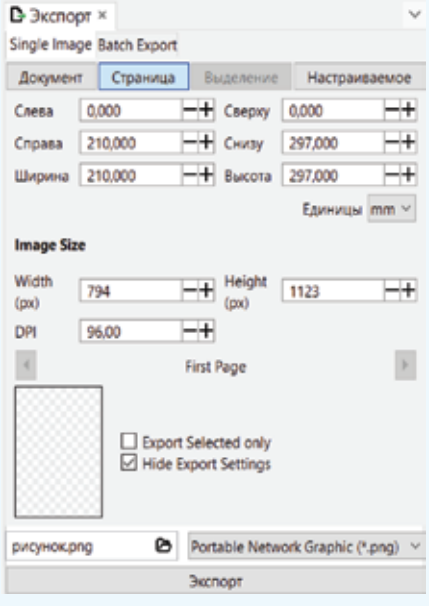
JPG — сжатый растровый формат изображений;

TIFF — растровый формат с высоким качеством картинки;

PDF — формат обмена документов от Adobe, который может содержать текст, шрифты, растровую и векторную графику.

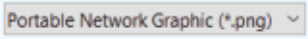
Пример 21.3. Сохранение изображения в формате, отличном от собственного.

1. В пункте меню **Файл** выбрать **Export**;

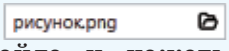


Правообладатель Адукацыя і выхаванне

Пример 21.3. *Продолжение.*

В открывшемся окне расположены поля для настройки параметров сохранения. В выпадающем списке 

необходимо выбрать формат для сохранения файла.

2. В поле 

ввести имя файла и нажать кнопку  . В открывшемся окне выбрать папку для сохранения файла и нажать **Сохранить**.

Если на холсте выделить часть изображения, то в файл сохранится только выделенное.

в собственном формате редактора Inkscape — SVG. Изображение, имеющее данный формат, следует открывать самим редактором Inkscape, а использование других программ может привести к некорректному результату. Основные форматы файлов, с которыми можно работать в Inkscape, представлены в примере 21.2 на с. 147.

В примере 21.3 описан процесс сохранения изображения в формате, отличном от собственного.

Для загрузки изображения нужно в меню **Файл** выбрать команду **Открыть**.

1. Как запустить векторный редактор Inkscape?
2. Какой формат файла является собственным форматом редактора Inkscape?
3. Как увеличить (уменьшить) масштаб страницы?
4. С файлами каких форматов может работать векторный редактор Inkscape?
5. Как сохранить часть изображения в формате, отличном от собственного?

Упражнения

1 Загрузите векторный редактор Inkscape. Откройте встроенный в него учебник. Для этого в главном меню выполните команду **Справка** → → **Учебники** → **Inkscape: Основы**. Руководствуясь материалом разделов **Перемещение по холсту** и **Изменение масштаба**, выполните:

1. Вертикальное перемещение по холсту, горизонтальное перемещение по холсту.

2. Увеличение и уменьшение масштаба холста.

2 Внесите изменения в интерфейс редактора.

1. Измените ориентацию холста на альбомную. Для этого в меню **Файл** нужно выбрать **Свойства документа.** В диалоговом окне **Свойства документа** выбрать соответствующую ориентацию: Ориентация: ○ ▯ ◉ ▭.

2. Отобразите сетку. Для этого в меню **Вид** следует выбрать **Сетка страницы:** ☑ Сетка страницы .

3 Скройте линейки и палитру цветов. Для этого в меню **Вид** нужно выбрать пункт **Показать или скрыть** и в развернувшемся списке команд убрать «птички» в соответствующих пунктах. Повторив команду, верните отображение линеек и палитры цветов.

4 Просмотрите историю своих действий, выбрав в меню **Правка** пункт **История действий...** .

§ 22. Создание и редактирование векторного изображения

Работу в векторных графических редакторах можно представить как создание фигур (объектов) нужной формы и задание им определённых заливок и обводок (абрисов) (пример 22.1).

Рассмотрим технологию создания фигур в редакторе Inkscape.

Начинать работу над созданием векторного изображения необходимо с определения:

1) набора фигур, из которых будет состоять изображение;

2) стиля и цвета обводки;

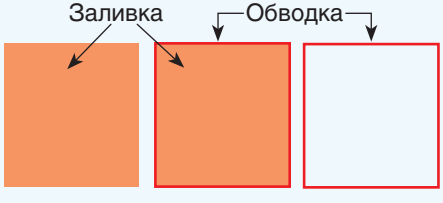
3) стиля заливки;

4) взаимного расположения фигур.

Пример 22.1. Основные инструменты создания фигур в векторном редакторе Inkscape.

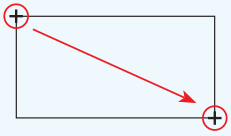
Изображение	Название
	Прямоугольники и квадраты
	Круги, эллипсы и дуги
	Звёзды и многоугольники

Образцы заливки и обводки:

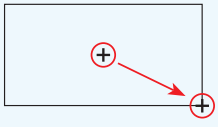


Правообладатель Адукацыя і выхаванне

Пример 22.2. Рисование прямоугольника.

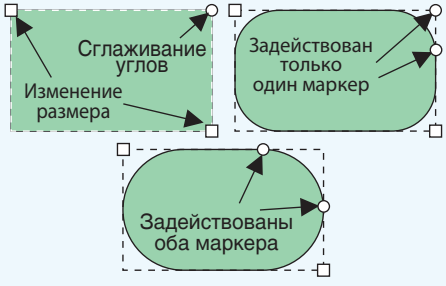


Начальная точка — вершина прямоугольника



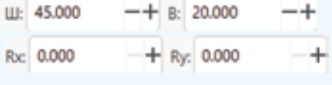
Начальная точка — центр прямоугольника (при нажатой клавише Shift)

Пример 22.3. Использование инструмента **Прямоугольники и квадраты.**



Созданный прямоугольник всегда выделен. В его углах расположено четыре маркера, два из них (вверху справа) накладываются друг на друга, если углы не закруглены.

Задать точные размеры прямоугольника и радиусы закругления углов можно на панели свойств:



22.1. Создание фигур

Прямоугольники создают с помощью инструмента **Прямоугольники и квадраты** . Прямоугольник строят, перемещая указатель мыши по холсту и удерживая левую кнопку мыши. Если при этом удерживать клавишу Ctrl, то получится квадрат. Если при рисовании прямоугольника удерживать клавишу Shift, то начальная точка прямоугольника будет соответствовать не вершине, а центру (пример 22.2). Квадратные маркеры в противоположных углах служат для изменения размера прямоугольника, а круглый маркер в верхнем правом углу позволяет управлять сглаживанием углов (пример 22.3).

Сглаживание (закругление) угла — замена угла на часть окружности или эллипса, вписанного в этот угол.

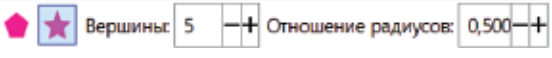
Заменить угол частью окружности можно перетащив круглый маркер вниз. При этом появится второй маркер. Перемещение каждого из маркеров позволяет заменять угол частью эллипса.

Убрать сглаживание можно нажав на кнопку на панели

свойств инструмента **Прямоугольники и квадраты** (кнопка появляется, если инструмент активен). На этой же панели можно задать ширину и высоту прямоугольника с помощью числовых параметров.

Для рисования эллипсов и кругов нужно выбрать инструмент **Круги, эллипсы и дуги** . Эллипсы и круги можно превращать в секторы, сегменты и дуги (пример 22.4). Рисуются эллипсы и круги так же, как и прямоугольники. При удержании клавиши Shift начальная точка рисования будет соответствовать центру эллипса.

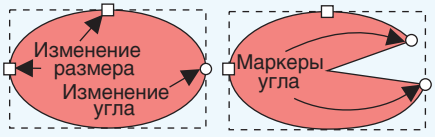
Для рисования звёзд и многоугольников используется инструмент **Звёзды и многоугольники** (пример 22.5). Начальная точка соответствует центру звезды, конечная — одной из её вершин. Определить тип фигуры (звезда или многоугольник) можно на панели свойств инструмента, на которой есть два значка с соответствующим изображением многоугольника и звезды:



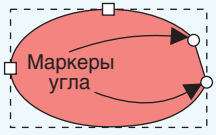
Там же можно изменить количество вершин и соотношение

Пример 22.4. Использование инструмента **Круги, эллипсы и дуги.**

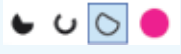
Чтобы нарисовать сектор, нужно потянуть круглый маркер вверх. Появится другой круглый маркер, перемещением которого вниз можно задать нужный угол сектора.



Если попробовать переместить круглый маркер внутрь эллипса, то окружность заменится дугой.



Чтобы заменить дуги на сегмент или на сектор, а также чтобы вернуться к эллипсу, следует воспользоваться кнопками на панели свойств инструмента:

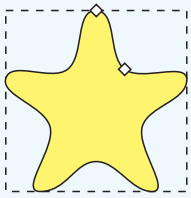


Пример 22.5. Использование инструмента **Звёзды и многоугольники** для рисования.

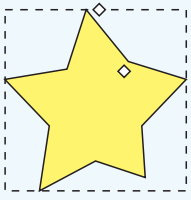


Правообладатель Адукацыя і выхаванне

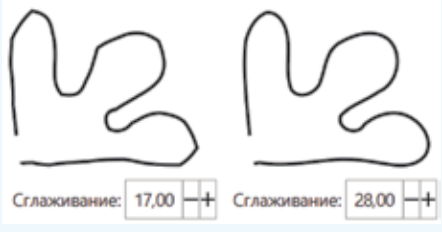
Пример 22.6. Использование инструмента **Звёзды и многоугольники** для сглаживания углов.



Пример 22.7. Использование инструмента **Звёзды и многоугольники** для перемещения вершин.



Пример 22.8. Использование инструмента **Карандаш**.



Активировать инструмент **Карандаш** можно нажав клавишу P или F6 на клавиатуре.

Этим инструментом можно создавать не только кривые, но и отдельные точки. Для этого нужно, удерживая Ctrl, щёлкнуть по холсту. Будет создан маленький круг.

внутреннего и внешнего радиусов (только для звезды).

Для изменения звезды или многоугольника используются два маркера узлов. Верхний узел служит для управления положением конца луча звезды или угла многоугольника. Второй маркер (базовый узел) контролирует положение внутреннего угла звезды.

Для изменения формы звезды используются следующие клавиши:

- Shift — позволяет управлять сглаживанием углов звезды (пример 22.6);
- Alt — все вершины звезды или многоугольника будут перемещаться в произвольном порядке (пример 22.7);
- Ctrl — узлы звезды или многоугольника будут перемещаться строго по прямой линии.

Сбросить значения свойств инструмента к значениям по умолчанию можно с помощью кнопки, расположенной в правой части панели свойств — .

Для рисования произвольных кривых используется инструмент **Карандаш** . При его использовании нужно нажать левую кнопку мыши в начальной точке

рисования и, не отпуская, переместить курсор, проводя линию.

Чтобы получить замкнутый контур, достаточно вернуться к первоначальной точке. Когда указатель мыши приближается к ней, точка меняет цвет на красный, в этот момент следует отпустить кнопку мыши.

Для инструмента **Карандаш** можно установить степень сглаживания: Сглаживание: 15,00 . Чем выше значение сглаживания, тем более гладкой будет кривая (пример 22.8).

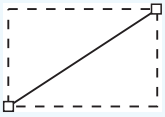
Для создания кривых и прямых линий предназначен инструмент **Кривая Безье и прямые линии** . Рисуя прямую, нужно нажать левую кнопку мыши в начальной точке и переместить курсор. При этом за курсором будет следовать прямая линия от её начала. Завершается рисование линии двойным щелчком или щелчком сначала левой, а потом правой кнопкой мыши в точке окончания линии. Линия изменит цвет на чёрный, и на её конце появится квадратный маркер (пример 22.9).

Создание кривой линии описано в примере 22.10.

Пример 22.9. Использование инструмента **Кривая Безье и прямые линии** для рисования прямых линий.

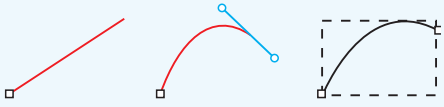


Начало рисования линии



Завершение рисования линии

Пример 22.10. Использование инструмента **Кривая Безье и прямые линии** для рисования кривых.



1. В месте начала линии щёлкнуть левой кнопкой мыши. За курсором будет тянуться прямая. Нажать на левую кнопку мыши в точке, где должен находиться конец линии.

2. Не отпуская левую кнопку мыши, перенести курсор в сторону. Появится прямая линия, к которой как касательная будет примыкать кривая.

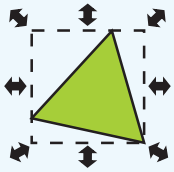
3. Установить нужный изгиб, отпустить левую кнопку мыши и щёлкнуть правой. Появится кривая, а прямая пропадёт. Можно продолжать формировать следующий участок кривой, не нажимая правую кнопку мыши.

Пьер Безье разработал метод вычерчивания кривых в 1960 г., работая в автомобилестроительной фирме «Рено».

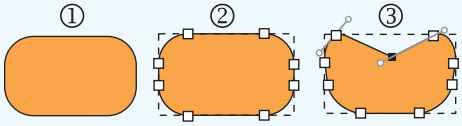
Правообладатель Адукацыя і выхаванне

Пример 22.11. Использование инструмента **Выделить и трансформировать.**

Если выполнить щелчок по уже выделенной фигуре, вид стрелок изменится и фигуру можно будет поворачивать и наклонять.



Пример 22.12. Использование инструмента **Редактировать узлы контура.**



1. Нарисовать графический примитив.

2. Выбрать инструмент и настройку (преобразовать в контур).

3. Перемещая узлы, изменить контур. При необходимости добавить или удалить узлы, сгладить или сделать острыми, используя инструменты панели свойств:

 — добавить узел;

 — удалить узел;

 — сделать выделенные узлы сглаженными;

 — сделать выделенные узлы острыми.

22.2. Редактирование фигур

Инструментом **Выделить и трансформировать**  выделяют фигуры (пример 22.11). Выделенные фигуры можно: удалять клавишей Delete; перемещать в рабочей области (с помощью Ctrl строго по горизонтали или вертикали); масштабировать (Ctrl позволяет сохранить пропорции); отражать горизонтально (клавиша H) и вертикально (клавиша V); поворачивать; наклонять.

После выделения вокруг фигуры появляется восемь двунаправленных стрелок, и объект можно удалять, перемещать и масштабировать.

Группа фигур выделяется при нажатой клавише Shift. Щелчок по выделенному в группе объекту исключает его из выделения. Нажатие клавиши Esc снимает всякое выделение, а комбинация клавиш Ctrl + A выделяет все объекты.

Форму объекта меняют с помощью инструмента **Редактировать узлы контура**  (пример 22.12).

После выбора этого инструмента по контуру выделенной фигуры отображаются маркеры узлов (пример 22.12).

22.3. Обводка и заливка

Обводка — линия, ограничивающая фигуру.

Для обводки можно определять:

1) толщину;

2) стиль (сплошной, пунктирный, штрихпунктирный);

3) стрелки (маркеры);

4) цвет.

Рассмотрите пример 22.13.

Заливкой называется закрашивание внутренней области фигуры.

Заливка фигуры может быть сделана одним из нижеперечисленных способов:

1) сплошным цветом;

2) градиентом;

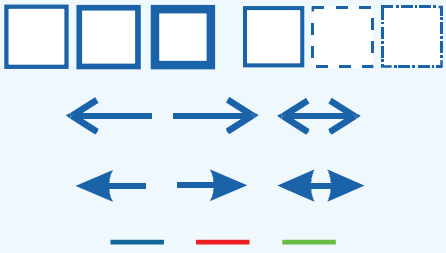
3) текстурой из набора декоративных заливок, в которых используются заранее созданные авторами программы заготовки.

Рассмотрите пример 22.14.

Цветовой градиент — плавный переход одного цвета в другой.

В редакторе Inkscape изменить обводку и заливку фигур можно

Пример 22.13. Стили обводки фигур и стрелки.

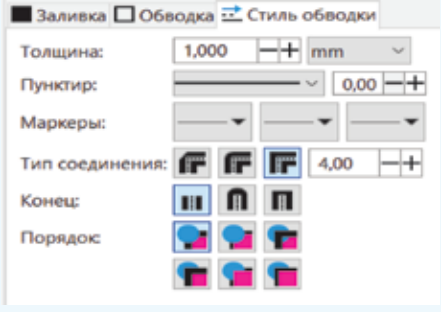


Пример 22.14. Способы заливки.

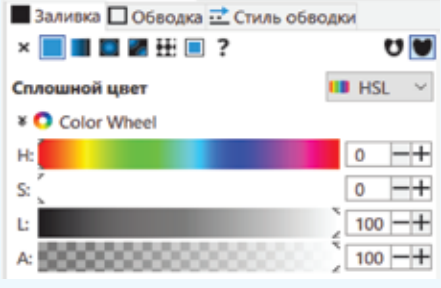


Правообладатель Адукацыя і выхаванне

Пример 22.15. Окно **Заливка и обводка**.

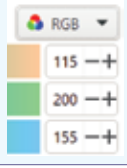


Изменение стиля обводки

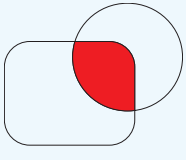


Изменение заливки

Пример 22.16. Настройка цвета в цветовой модели RGB.

Цифровые значения	Цвет
	

Пример 22.17. Использование инструмента **Заливка**.



в диалоговом окне **Заливка и обводка** или с помощью палитры.

Окно открывается по команде главного меню редактора **Объект → Заливка и обводка** (пример 22.15). В этом окне можно выбрать цветовую модель, настроить цвет и его прозрачность.

В палитре цвет заливки изменяется щелчком мыши по образцу цвета. Для изменения цвета обводки щелчок производится при нажатой клавише Shift.

Объекту можно назначить отсутствие обводки и заливки с помощью красного крестика в левой части палитры. Градиентную заливку можно применять и к обводке.

22.4. Работа с цветом

Задавать цвет в окне **Заливка и обводка** удобно, меняя цифровые значения основных цветов (пример 22.16).

Для заливки замкнутой области используется инструмент **Заливка**  (пример 22.17).

При необходимости использования цвета, который уже присутствует в какой-либо части

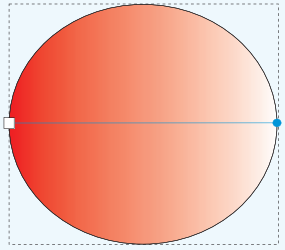
изображения на холсте, используется инструмент **Пипетка** .

Для создания градиентных заливок предназначен инструмент **Градиент** . На панели свойств инструмента можно выбрать вид градиента (линейный, радиальный) и к чему он будет применён (заливка, обводка). При переходе от сплошной заливки к градиенту создаётся настройка градиента, которая использует текущий цвет с переходом от непрозрачного к прозрачному. Цвета градиента настраиваются при помощи опорных точек (пример 22.18). Количество точек соответствует количеству цветов в градиенте. Опорные точки можно добавлять или удалять. Добавляется опорная точка двойным щелчком мыши по линии градиента. Удаляется выделенная опорная точка с помощью клавиши Delete.

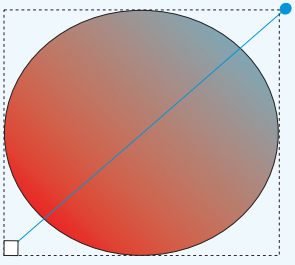
Состояние опорной точки определённого цвета:

- — невыделенная (неактивная);
- — выделенная (активная, текущая, выбранная);
- — наведённая (под указателем мыши).

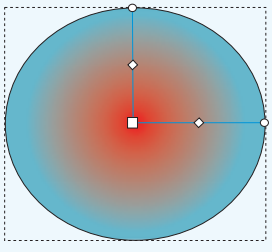
Пример 22.18. Использование инструмента **Градиент.**



Для линейного градиента все опорные точки расположены на одной прямой.



Опорные точки могут располагаться и за пределами объекта.



Радиальный градиент имеет центральную опорную точку, которая является начальной. Из неё выходят две линии градиента.

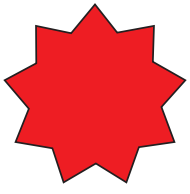
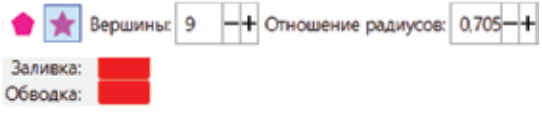
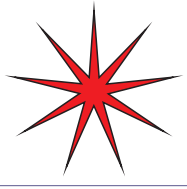
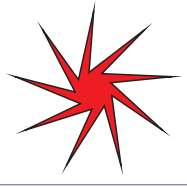
Правообладатель Адукацыя і выхаванне

1. С помощью какой клавиши можно нарисовать правильную фигуру?
2. Как сгладить углы прямоугольника в редакторе Inkscape?
3. Какие фигуры можно нарисовать с помощью инструмента ?
4. Для чего служит инструмент ?
5. Какой инструмент позволяет редактировать узлы фигуры?
6. Что называется обводкой векторного изображения?
7. В чём состоит процесс заливки фигуры?
8. Что такое цветовой градиент?
9. Как настроить градиент в редакторе Inkscape?

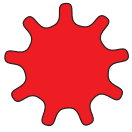
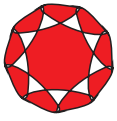
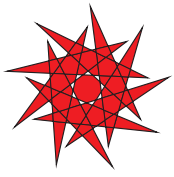
Упражнения

1 Нарисуйте несколько разных фигур и примените к ним масштабирование, отражение, поворот, наклон.

2 Создайте изображения звёзд.

№	Изображение	Приёмы создания и редактирования
1.		Инструмент: 
2.		Выполнить п. 1 и изменить форму, перемещая верхний узел звезды к центру
3.		Выполнить п. 1 и изменить форму, перемещая базовый узел звезды вправо

Продолжение таблицы

№	Изображение	Приёмы создания и редактирования
4.		Выполнить п. 1 и изменить форму, перемещая верхний узел при нажатой клавише Alt
5.		Выполнить п. 1 и изменить форму, перемещая верхний узел вправо при нажатой клавише Shift
6.		Выполнить п. 1 и изменить форму, перемещая верхний узел влево при нажатой клавише Shift
7.		Выполнить п. 1 и изменить форму, перемещая базовый узел звезды влево и вверх

3 Нарисуйте пять фигур: прямоугольник, квадрат, овал, круг и правильный шестиугольник. Примените к каждой фигуре соответствующие заливки: сплошная (с прозрачностью и без), градиент (линейный и радиальный), текстура.

4* Создайте изображение по примеру.



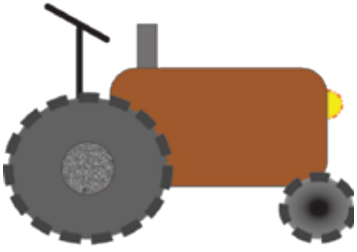
Нарисуйте малиновый квадрат без обводки. Откройте панель **Заливка и обводка**. Выберите цветовую схему RGB. Перейдя на вкладку

Правообладатель Адукацыя і выхаванне

Заливка, установите R = 255, G = 0, B = 125, A = 100, размытие — 0, непрозрачность — 100. На вкладке **Обводка** установите стиль без обводки (✕).

Нарисуйте фиолетовый прямоугольник с чёрной обводкой так, чтобы он перекрывал часть квадрата. На вкладке **Заливка** установите R = 125, G = 0, B = 200, A = 100, размытие — 0, непрозрачность — 55. На вкладке **Стиль обводки** — толщина 15 px.

5 Создайте изображение.



В этом вам поможет таблица.

Элемент изображения	Инструмент и настройки	Обводка	Заливка
		Сплошная, цвет чёрный	Однородная, цвет тёмно-серый
	 Переключиться на 	Штрих-пунктирная, цвет красный, толщина 4 px	Однородная, цвет жёлтый (ffff00ff)

Продолжение таблицы

Элемент изображения	Инструмент и настройки	Обводка	Заливка
		Сплошная, цвет чёрный	Однород-ная, цвет коричневый (a05a2cff)
		Пунктирная, цвет чёрный, толщина 15 px	Градиент радиальный (переход от чёрного к се-рому)
	 Режим — прямые линии	Сплошная, цвет чёрный	Нет
		Сплошная, цвет чёрный, концы круглые	
		Пунктирная, цвет чёрный, толщина 15 px	Однородная, цвет серый (4d4d4dff)
		Сплошная, цвет чёрный	Текстура «песок»

6 Создайте изображение.



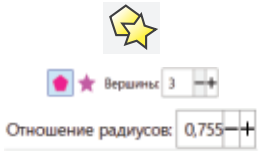
Правообладатель Адукацыя і выхаванне

В этом вам поможет таблица.

<table>
<tr><th>Элемент изображения</th><th>Инструмент и настройки</th><th>Обводка</th><th>Заливка</th></tr>
<tr><td>  </td><td rowspan="3">  </td><td rowspan="3">Сплошная, цвет чёрный, толщина 3 px</td><td>Линейный градиент, два цвета</td></tr>
<tr><td>  </td><td>Линейный градиент, три цвета</td></tr>
<tr><td>  </td><td>Однородная, цвет чёрный</td></tr>
<tr><td rowspan="2">  </td><td>Для треугольника:
 
 
</td><td rowspan="2">Для треугольника: сплошная, цвет серый, толщина 3 px</td><td rowspan="2">Для треугольника: однородная, цвет 918a6fff</td></tr>
<tr><td>Для линии: 
(режим — прямые линии)</td></tr>
<tr><td>  </td><td> 
 

Изменить форму
  </td><td>Сплошная, цвет чёрный, толщина 3 px</td><td>Однородная, цвет серый</td></tr>
</table>

Продолжение таблицы

Элемент изображения	Инструмент и настройки	Обводка	Заливка
	 Переместить базовый узел вправо, изменить размер и повернуть	Сплошная, цвет тёмно-серый, толщина 7 px	Однородная, цвет серый

§ 23. Операции над объектами векторного изображения

23.1. Копирование, выравнивание и взаимное расположение объектов

Создать копию объекта в редакторе Inkscape можно, как и в других программах, воспользовавшись буфером обмена. В этом случае копия появится рядом с оригиналом. В редакторе Inkscape есть дополнительные операции копирования объектов — дублирование и клонирование (пример 23.1).

Чтобы создать дубль объекта, нужно выбрать пункт меню

Пример 23.1. Дублирование и клонирование фигур.



После редактирования оригинала получим:



Правообладатель Адукацыя і выхаванне

Пример 23.2. Создание дубля и клона инструментом **Распылитель** .

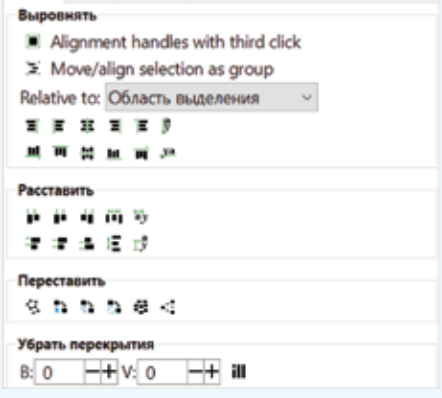
Распылитель рисует объектами из буфера обмена. Распыление выполняется в трёх режимах:

— создаёт копии объекта;

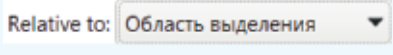
— создаёт клоны объекта;

— распыление одиночных объектов.

Пример 23.3. Окно **Выровнять и расставить.**



После выбора способа выравнивания или распределения следует выбрать область выравнивания в выпадающем списке:



Чаще всего областью служит страница, рисунок или область выделения. В большинстве случаев достаточно выбрать и использовать режим **Область выделения.**

Правка → Дублировать или нажать на панели команд. Созданная копия помещается поверх оригинала. Как видно из примера 23.1 на с. 163, дубль является самостоятельным объектом. При изменении оригинала дубль не меняется.

Клон создаётся по команде **Правка → Клоны → Создать клон** или с помощью кнопки на Панели команд. При создании клон, как и дубль, помещается над оригиналом. Клоны изменяются при изменении оригинала. Ещё один способ создания дубля или клона приведён в примере 23.2.

Для размещения объектов векторного изображения определённым образом относительно друг друга используются операции выравнивания.

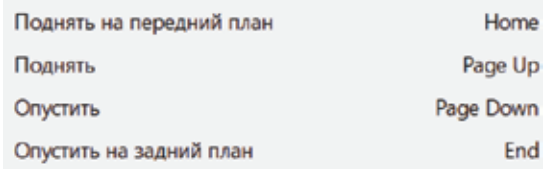
Выравниванием называется такое размещение всех выделенных объектов, при котором определённые точки объектов располагаются на одной прямой.

Выравнивание объектов можно осуществить с помощью окна **Выровнять и расставить.** Окно можно открыть, выполнив команду **Объект → Выровнять и расставить** (пример 23.3).

На панели **Выровнять** в первой строке выбирается способ выравнивания фигур по вертикали, а во второй строке — по горизонтали (пример 23.4).

На панели **Расставить** определяется способ распределения выделенных объектов на некотором расстоянии друг от друга. В первой строке объекты распределяются по вертикали, а во второй строке — по горизонтали (пример 23.5).

Для изменения **порядка расположения** объектов их необходимо выделить. Затем воспользоваться кнопками  или командами меню **Объект:**

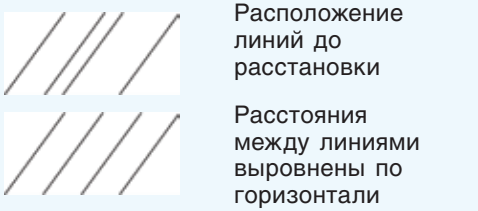


Команды **Поднять на передний план** и **Опустить на задний план** ставят выделенные объекты на самую верхнюю или самую нижнюю позицию. Две другие команды, **Поднять** и **Опустить**, опускают или поднимают выделенные объекты на один уровень относительно ближайшего невыделенного объекта (пример 23.6).

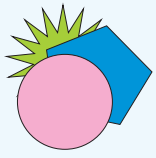
Пример 23.4. Выравнивание фигур.



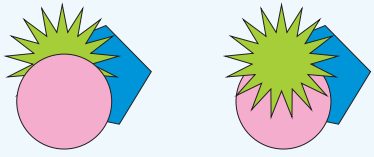
Пример 25.5. Распределение объектов.



Пример 23.6. Порядок расположения фигур.



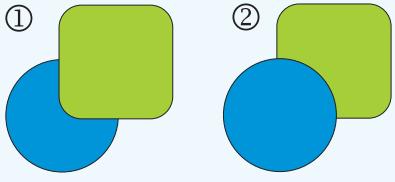
Изменяем положение фигуры на заднем плане (зелёной звезды):

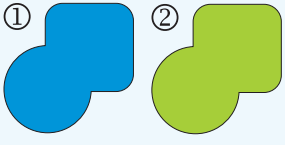
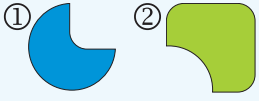
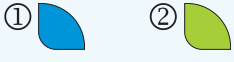


Правообладатель Адукацыя і выхаванне

Группировка предотвращает случайные изменения положения объекта относительно других объектов. При группировке можно создавать вложенные группы, группируя существующие группы. Выполнение логических операций над объектами позволяет создавать различные фигуры неправильной формы.

Пример 23.7. Применение операций из меню **Контур**.



Операция	Результат операции
Сумма	
Разность	
Пересечение	

23.2. Группировка. Операции объединения и пересечения объектов

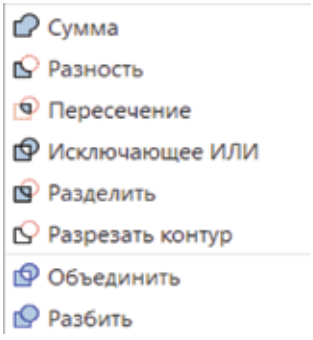
В Inkscape несколько объектов можно объединить в группу. При перемещении и трансформации группа ведёт себя как один объект.

Чтобы сгруппировать несколько объектов, нужно выделить их все и выбрать в меню **Объект** → → **Сгруппировать** или нажать Ctrl + G. Чтобы разгруппировать одну или несколько выделенных групп, нужно выбрать в меню **Объект** → **Разгруппировать** или выполнить двойной щелчок мышью по группе. Сами группы можно объединять в группы. Двойной щелчок отменяет только группировку верхнего уровня.

Необязательно разбивать группу, если в ней нужно отредактировать один из объектов. Достаточно щёлкнуть по объекту, удерживая Ctrl (или Shift + Ctrl, если нужно отобрать несколько объектов), и можно будет работать с объектом в группе отдельно.

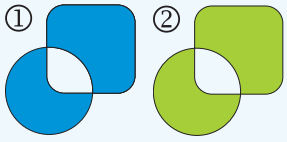
При создании изображений часто возникает необходимость выполнить операции преобразования контуров группы объектов, что соответствует применению операций над множествами. В редакторе Inkscape для этого

используются команды меню **Контур:**



В примере 23.7 показан результат того, как могут соединяться два объекта, расположенных в разном порядке, после применения некоторых операций из меню **Контур.**

Пример 23.7. *Продолжение.*

Операция	Результат операции
Исключающее ИЛИ	
Разделить	

1. Какие операции над объектами векторного изображения можно выполнять в редакторе Inkscape?
2. Что произойдёт, если к объектам применить сначала выравнивание по вертикальной оси, а потом по горизонтальной оси?
3. Что происходит при группировке объектов?
4. Важно ли расположение элементов при выполнении операций преобразования контуров группы?

Упражнения

1 Создайте изображение божьей коровки, как показано на рисунке.



Правообладатель Адукацыя і выхаванне

Для этого подготовьте элементы рисунка, используя инструменты **Эллипс** и **Кривая Безье.**



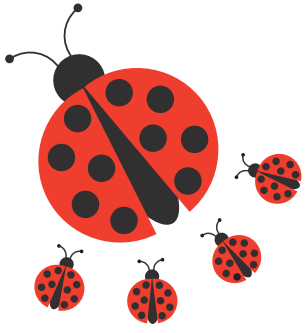
Примените к созданным элементам операции, указанные в таблице.

Исходные объекты	Операции	Результат
	Дублировать объект и уменьшить размер дубля. Создать 4 дубля изменённого объекта	
	Разместить чёрные круги в красном полукруге и сгруппировать объекты	
	Дублировать объект и уменьшить размер дубля.	
	Соединить чёрный круг с дугой и сгруппировать объекты	

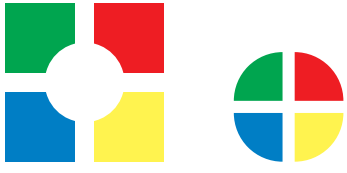
Продолжение таблицы

Исходные объекты	Операции	Результат
	Продублировать объект и отразить дубль по горизонтали. Выполнить повороты	
		
	Продублировать объект и изменить размеры и пропорции дубля. Выполнить поворот	

2 Используя рисунок из задания 1, создайте следующее изображение.

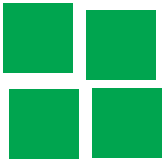
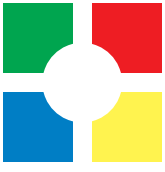


3 Создайте изображение, применив операции над объектами.

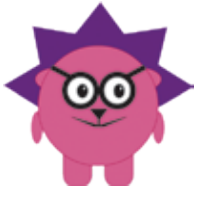


Правообладатель Адукацыя і выхаванне

Для этого создайте элемент рисунка, используя инструмент **Прямоугольники и квадраты**. Примените к созданным элементам операции, указанные в таблице.

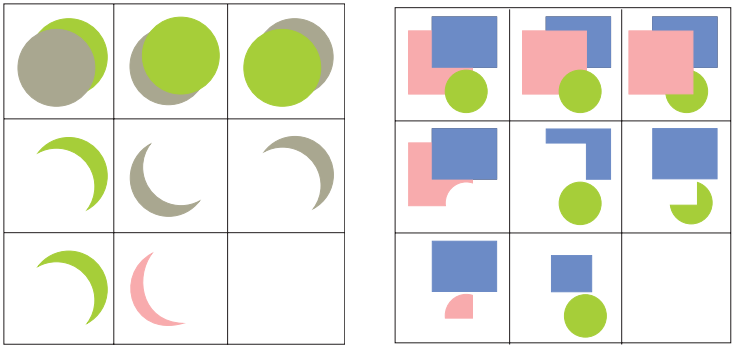
Операции	Результат
Дублирование + перемещение	
Выравнивание + изменение цвета	
Нарисовать белый круг, выровнять по центру и создать три его дубля	
Выделяя попарно один квадрат и круг, применить операцию пересечения	

4 Нарисуйте одно из изображений.






5 Откройте файл и дополните изображения недостающими фигурами.



6* Используя изображения из задания 4, придумайте и нарисуйте сюжетный рисунок.

§ 24. Работа с текстом

Для создания текста используется инструмент **Текст** A. В векторных редакторах предлагается два варианта работы с текстом — простой текст и художественный текст.

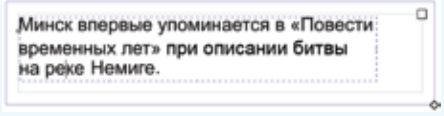
Простой текст используется для отображения на рисунках больших текстовых фрагментов и может разделяться на абзацы. Простой текст можно форматировать точно так же, как и в обычном текстовом редакторе.

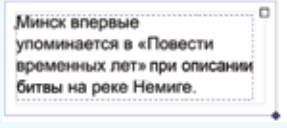
Для создания простого текста необходимо:

1) выбрать инструмент A;

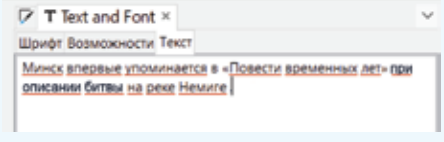
2) установить параметры текста на контекстной панели управления;

Пример 24.1. Создание простого текста.





Вводить текст удобно в текстовом редакторе. Его вызов выполняется по команде **Текст** → → **Текст и шрифт**, вкладка **Текст**:



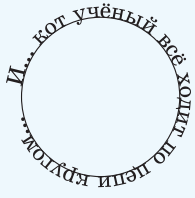
Правообладатель Адукацыя і выхаванне

Пример 24.1. *Продолжение.*

Текст можно вставить через буфер обмена из другого текстового редактора (Блокнот и др.).

Простой текст всегда можно преобразовать в художественный: **Текст → Преобразовать в текст**. Перед преобразованием текст должен быть выделен. Если после выполнения команды выделить текст, то рамка с узлами будет отсутствовать. Это уже художественный текст.

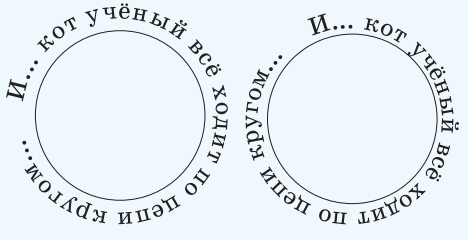
Пример 24.2. Расположение текста[1] по окружности.



После связывания текста и контура можно редактировать каждый из этих объектов, не нарушая связи между ними:

1) приподнять текст над контуром (изменить размер рамки, которой ограничен текст);

2) повернуть контур (при этом текст поворачивается вместе с контуром).



3) обрисовать с помощью мыши текстовую рамку и ввести в неё текст.

Текстовая рамка представляет собой сплошную прямоугольную рамку. Текст в ней ограничивается пунктирной рамкой. При помощи маркера в правом нижнем углу можно менять размеры текстовой рамки. При этом текст подгоняется под размеры рамки (пример 24.1).

Художественный текст используется для создания художественных надписей. Созданные надписи являются графическими объектами. К ним можно применять различные эффекты, например изменять форму букв, создавать тень, располагать текст по нужной траектории. Для создания художественного текста нужно щёлкнуть левой кнопкой мыши в любой точке холста. В месте щелчка начнёт мигать курсор, приглашая к вводу текста. Переход на новую строку происходит по нажатию клавиши Enter.

При любом способе создания объекта с текстом этот объект можно перемещать, поворачивать и т. д. с помощью инструмента ↖.

[1] Пушкин А. С. Руслан и Людмила. М., 1996. С. 5.

Для редактирования объекта с текстом необходимо выбрать инструмент A и щёлкнуть по объекту.

Много интересных возможностей работы с художественным текстом предлагает меню **Текст.** Рассмотрим некоторые из них.

1. Разместить по контуру (пример 24.2 на с. 172). Перед применением этой команды нужно выделить текст и контур.

2. Текст в рамку (пример 24.3). Порядок выполнения действий при этом такой же, как и при размещении текста по контуру.

Текст можно раскрасить в разные цвета (целиком или частично). К тексту можно применить градиент (пример 24.4). При этом в дальнейшем текст можно редактировать обычным образом. Буквы текста можно превратить в контур (меню **Контур** → → **Оконтурить объект**). После этого текст уже нельзя редактировать, но к нему можно применять разнообразные эффекты, используя команды меню **Фильтры**. В примере 24.5 применён эффект **Искажение** → **Огрубление.**

Пример 24.2. *Продолжение.*

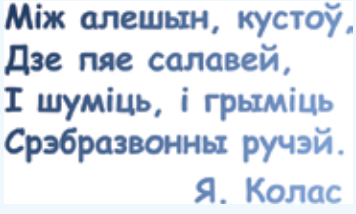
3. Убрать контур (сделать прозрачным).



Пример 24.3. Расположение текста внутри контура.



Пример 24.4. Применение градиента к тексту[1].



Пример 24.5. Применение эффекта к тексту.



[1] Колас Я. Ручэй // Собрание сочинений. Том I. Минск, 1928. С. 21.

Правообладатель Адукацыя і выхаванне

1. Какие возможности работы с текстом существуют в редакторе Inkscape?
2. Какие действия можно выполнять с текстовым объектом?
3. Можно ли редактировать художественный текст?
4. Каким образом можно превратить буквы текста в контуры?
5. Можно ли редактировать текст после превращения его букв в контуры?
6. Какие действия необходимо выполнить перед применением к тексту эффектов?

Упражнения

1 На рисунок трактора из упражнения 5 § 22 добавьте надпись «Belarus».

2 Нарисуйте логотип белорусского предприятия.



3 Нарисуйте стикер к учебному предмету «Химия». Текст разместите по контуру кривой линии.



4* Придумайте и нарисуйте стикер к одному из учебных предметов (на выбор).

5 Нарисуйте открытку или плакат. Примените различные виды заливок и вставьте текст.





6* Придумайте и нарисуйте плакат на одну из тем.

Национальные символы Республики Беларусь	Мы в ответе за тех, кого приручили
Мойте руки перед едой	Берегите электроэнергию
Моё счастливое детство	Правила поведения в компьютерном классе

Правообладатель Адукацыя і выхаванне

Глава 5
ИНФОРМАЦИЯ И ИНФОРМАЦИОННЫЕ ПРОЦЕССЫ
§ 25. Информация в жизни человека

Понятие «информация» имеет множество определений. Смысл, вкладываемый в понятие информации, зависит от области применения. Не существует универсального определения, которое объединяло бы концепции информации из разных сфер деятельности.



Знаменитый советский учёный в области общей механики и прикладной математики академик Никита Николаевич Моисеев (1917–2000 гг.) считал, что «в силу широты этого понятия нет и не может быть строгого и достаточно универсального определения информации».

В Законе Республики Беларусь «Об информации, информатизации и защите информации» даётся следующее определение: «информация — сведения о лицах, предметах, фактах, событиях, явлениях и процессах независимо от формы их представления».

25.1. Виды информации

Каждый из нас не раз слышал слово «информация». Информацию мы получаем из книг, газет, Интернета, от людей, с которыми общаемся. А что же означает данное понятие?

Информация — сведения о предметах, событиях, явлениях и процессах окружающего мира.

Большая часть сведений, на основании которых формируется представление человека об окружающем мире, поступает к нему благодаря органам чувств. Наличие зрения, слуха, осязания, вкуса и обоняния позволяет нам получать знания об окружающей действительности. В зависимости от того, с помощью каких органов чувств информация поступила к человеку, её классифицируют по способу восприятия (пример 25.1).

• **Визуальная информация** воспринимается органами зрения, различающими форму, объём, цвет, перемещение и изменение объектов.

• Органы слуха воспринимают **звуковую информацию.** С их помощью можно распознавать звуки, различать их тембр, высоту, ритм, эмоциональную окраску.

• **Тактильная информация** связана с органами осязания, позволяющими на ощупь определить характер поверхности, её температуру, почувствовать прикосновение.

• С помощью органов вкуса человек получает **вкусовую информацию.** Вкус может быть горьким, сладким, кислым, солёным и др.

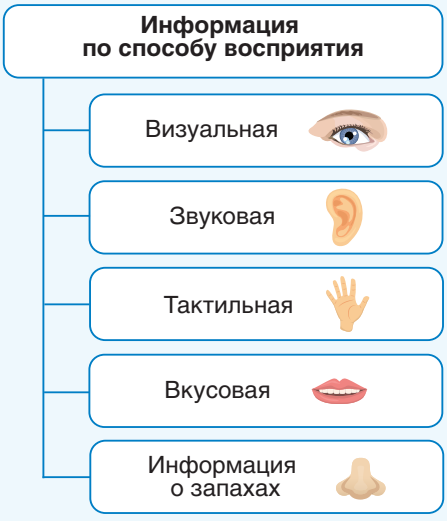
• Орган обоняния осуществляет восприятие и распознавание **информации о запахах.**

Существует также классификация информации по форме представления (пример 25.2).

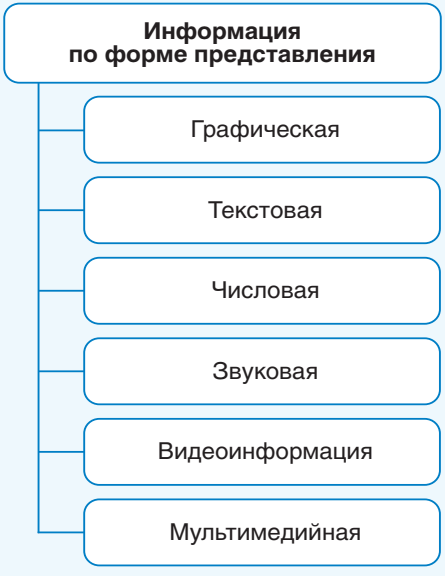
• **Графическая информация** — сведения в виде рисунков, схем, чертежей, картин, фотографий.

• **Текстовая информация** — сведения в виде специальных символов (буквы различных алфавитов, иероглифы, с помощью

Пример 25.1. Классификация информации по способу восприятия.



Пример 25.2. Классификация информации по форме представления.



Правообладатель Адукацыя і выхаванне

Помимо классификации информации по способу восприятия и по форме представления, существуют также и другие классификации.

По **сфере возникновения:**

- *механическая* информация отражает процессы и явления неодушевлённой природы;
- *биологическая* информация связана с процессами животного и растительного мира;
- *социальная* информация отражает процессы человеческого общества.

По **назначению:**

- *массовая* информация понятна и доступна большей части социума;
- *специальная* информация используется в узкой социальной и профессиональной группе людей;
- *секретная* информация доступна узкому кругу лиц и передаётся по закрытым (защищённым) каналам;
- *личная (приватная)* — набор сведений о какой-либо личности.

По **достоверности:**

- *истинная* информация получена из надёжных проверенных источников; обоснованная информация;
- *ложная* информация содержит заведомый обман или неподтверждённые сведения.

которых записывают отдельные слоги или слова).

- **Числовая (цифровая) информация** — сведения, отражающие количественную меру объектов и их свойств с помощью чисел и цифр.
- **Звуковая информация** — сведения, представленные в виде звуков.

Существуют и комбинированные виды информации, такие как **видеоинформация** и **мультимедийная** информация.

Одна и та же информация может поступать и храниться в различных формах (например, музыкальное произведение в виде аудиофайла или нотной записи). Для преобразования информации из одной формы представления в другую используются различные алгоритмы и устройства.

25.2. Носители информации

Для записи, хранения и считывания информации используются носители информации. В древности человек хранил важные сведения лишь в собственной памяти, т. е. память человека является естественным носителем информации. С появлением письменности появились и первые

носители информации: папирус, пергамент, восковые и глиняные таблички. На белорусских землях и территории Древней Руси в X–XIII вв. носителями информации были берестяные грамоты. Важная информация записывалась и хранилась на бересте (коре берёзы) (пример 25.3).

Потребность запоминать и хранить всё возрастающие объёмы информации привела к созданию и использованию различных материалов и устройств. К ним относятся: бумага, книги, газеты, фотографии, магнитные и оптические диски, винчестер, флеш-память и др. (пример 25.4). В современном мире для обработки, хранения и распространения больших объёмов информации строят специальные здания — дата-центры.

25.3. Информационные процессы

В повседневной жизни мы записываем, запоминаем и считываем полученную информацию. Человек может поделиться известной ему информацией с другими людьми. Кроме того, на основе обработки имеющейся информации можно создать новую.

Пример 25.3. Древние носители информации.



Восковые таблички



Пергамент



Глиняные таблички



Витебская берестяная грамота (ориентировочно XIII в.) — первая берестяная грамота, найденная на территории БССР (1959 г.)

Пример 25.4. Компьютерные носители информации.



Карта памяти



Внешний жёсткий диск



Флешка



Оптический диск



Перфокарта. Носитель информации для ЭВМ «Минск-22» (1965–1970 гг.)

Правообладатель Адукацыя і выхаванне

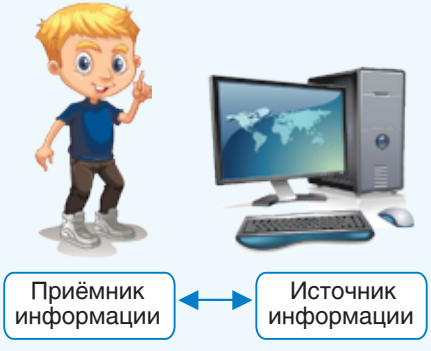
Пример 25.5. Виды информационных процессов



Пример 25.6. Хранение информации.



Пример 25.7. Передача информации.



Любая деятельность человека, связанная с информацией, является **информационным процессом.** Различают следующие информационные процессы: **хранение, передача, обработка, поиск** информации (пример 25.5).

Хранение информации — это её запись на различных носителях для последующего использования. Люди хранят информацию либо в собственной памяти, либо на каких-то внешних носителях (пример 25.6). Информация в человеческой памяти доступна только ему, человек может забыть информацию. Информация на внешних носителях может храниться долго и быть доступна разным людям.

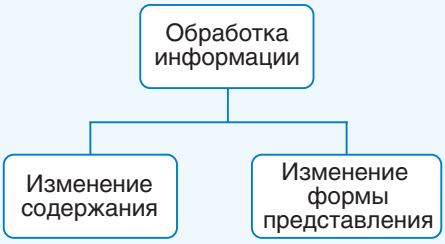
Передача информации происходит, к примеру, при разговоре двух людей, при чтении книги или журнала, при просмотре страниц в сети Интернет и др. В процессе передачи информации всегда участвуют два объекта: **источник** и **приёмник.** Передача информации происходит через **канал связи:** звуковые волны при непосредственном разговоре, услуги почтового сервиса при переписке, сотовая связь при разговоре по мобильному телефону (пример 25.7).

В результате **обработки** информации происходит изменение содержания или формы представления (пример 25.8).

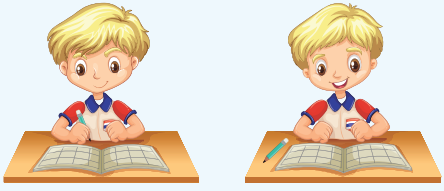
Содержание информации может измениться в результате вычислительных действий, например при решении уравнения (пример 25.9). Процесс рассуждений человека также может приводить к появлению новой информации, которая заменяет старые знания. Новая информация возникает при исследовании различных явлений природы, физических процессов и др.

Форма представления информации изменяется при рисовании картин по текстовому описанию или при пересказе сюжета видеофильма в виде статьи в журнале, при переводе текста с одного языка на другой (пример 25.10) и т. п. Сегодня с такой обработкой информации успешно справляется искусственный интеллект. ИИ-помощник, встроенный в Яндекс Браузер, может помочь с ответами на вопросы, переводом текста и решением различных задач, связанных с обработкой и анализом естественного языка (пример 25.11).

Пример 25.8. Операции обработки информации.



Пример 25.9. Изменение содержания информации.



$2x + 3 = 7$

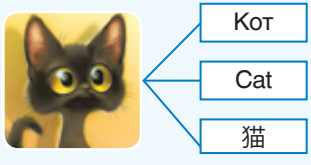
$2x = 7 - 3$

$2x = 4$

$x = \frac{4}{2}$

$x = 2$

Пример 25.10. Изменение формы представления информации.



Пример 25.11. ИИ-помощник в Яндекс Браузере.



Правообладатель Адукацыя і выхаванне

Пример 25.12. Нейросеть «Нейрохолст» для генерации изображений по описанию на русском языке.



Пример 25.13. Поиск информации.



Большое количество изображений сегодня получают с использованием искусственного интеллекта, который может создавать рисунки по текстовому описанию (пример 25.12). Также ИИ может создавать видео на основе текстового описания или изображения, добавлять новые элементы в готовые видео, изменять стили существующих видео и др.

Если человеку необходимо найти интересующие его сведения, то он осуществляет **поиск** информации. Для поиска нужной информации используются разнообразные способы: чтение энциклопедий, словарей, книг и журналов, просмотр видеофильмов и телевизионных передач, поиск в сети Интернет (пример 25.13) и т. д.

1. Что понимают под информацией?
2. Как можно классифицировать информацию по способу её восприятия человеком?
3. На какие виды можно разделить информацию по форме представления?
4. Что такое носитель информации?
5. Какие информационные процессы выполняет человек?
6. Какими возможностями обладает искусственный интеллект в области обработки естественного языка?

Упражнения

1 Назовите, какие информационные процессы выполняет семиклассник при следующих видах деятельности:

1. Запись в конспект материалов урока.
2. Устный ответ у доски.
3. Перевод текста с русского языка на английский.
4. Подбор материалов для реферата по истории.

2 Приведите примеры профессий, в которых основная деятельность связана с информацией.

3 Опишите ситуации, в которых вы можете играть роль источника информации, приёмника информации. Каким каналом связи вы пользуетесь при этом?

4 Обратитесь к ИИ-помощнику «Пересказ» в Яндекс Браузере для составления краткого конспекта параграфа.

5 Подготовьте сообщение на одну из следующих тем:

1. Древние носители информации.
2. Электронные носители информации.
3. История звукозаписывающих устройств.
4*. Дополненная реальность как форма представления информации.
5. Искусственный интеллект как инструмент для создания изображений.

§ 26. Представление информации в компьютере

26.1. Кодирование информации

Для общения друг с другом люди используют естественный язык, например белорусский или русский. В основе естественного языка лежит алфавит — система графических знаков для передачи звуков устной речи. Алфавит естественного языка является универсальным кодом любой письменной культуры. При разговоре

Кодирование информации использовалось с глубокой древности для хранения и передачи информации.

Примером кодирования информации является шумерская клинопись.



Правообладатель Адукацыя і выхаванне

этот код передаётся звуками, при письме — буквами.

Кроме естественных языков человек часто использует формальные языки со своими особыми кодами: язык математических или химических формул, ноты и др.

Код — это совокупность условных знаков, каждому из которых присваивается определённое значение (примеры 26.1–26.3).

Процесс записи или преобразования информации в соответствии с правилами, заданными некоторым кодом, называют **кодированием**. Процесс, обратный кодированию, называют **декодированием**.

Кодировать и передавать информацию можно различными способами: устно, письменно, жестами и др. Компьютер может обрабатывать числовую, текстовую, графическую, звуковую и видеоинформацию только в цифровом формате. Для получения цифрового формата информация с помощью специальных программ преобразуется в двоичный код, понятный компьютеру.

Пример 26.1. Широко известным является шифр Юлия Цезаря — кодирование, которое применялось для записи и передачи зашифрованных сообщений. Каждый символ в тексте заменяется символом, находящимся на некотором постоянном расстоянии левее или правее него в алфавите. В шифре со сдвигом вправо на 3 позиции буква «А» была бы заменена на «Г», «Б» станет «Д» и т. д.



Пример 26.2. Сегодня широко применяются штрих-коды на различных товарах. Товары, произведённые в Беларуси, имеют номер, начинающийся с цифр 481.



Штрих-код зефира в шоколаде ОАО «Красный пищевик», г. Бобруйск

Двоичный код — это способ кодирования, при котором каждый **разряд кода** принимает одно из двух возможных значений, обычно обозначаемых цифрами 0 и 1. **Разряд** в этом случае называется **двоичным разрядом.**

Такой способ кодирования позволяет создавать технические устройства, обладающие двумя устойчивыми состояниями: включено/выключено, соединено/разъединено и др.

Для кодирования числовой информации в компьютере вместо известной всем десятичной системы счисления используется двоичная система счисления. В ней для записи чисел используют две цифры: 0 и 1.

Кодирование текстовой информации в компьютере выполняется с помощью специальных кодовых таблиц, в которых каждому символу ставится в соответствие определённая последовательность из нулей и единиц (пример 26.4).

На разных компьютерах могут использоваться разные кодовые таблицы. В этом случае текст, набранный на одном компьютере, может неправильно читаться на другом.

Пример 26.3. С появлением смартфонов стали распространяться QR-коды. Они позволяют моментально заносить в свой телефон текстовую информацию, добавлять контакты в адресную книгу, переходить по веб-ссылкам, отправлять СМС-сообщения и т. д. Следующий QR-код позволяет перейти на сайт «ЮНИ-центр-XXI» факультета прикладной математики и информатики БГУ, посвящённый дополнительному образованию учащихся по математике и информатике.



Пример 26.4. Кодирование некоторых букв русского алфавита на компьютере.

Символ	Двоичный код	Десятичный код
А	11000000	192
Б	11000001	193
В	11000010	194
Г	11000011	195
Д	11000100	196
Е	11000101	197
Ж	11000110	198
З	11000111	199

Правообладатель Адукацыя і выхаванне

Пример 26.5. Инструменты для различных измерений — времени, расстояния, температуры, массы.



Пример 26.6. Соотношение между битом и байтом.



Слово «бит» произошло от двух английских слов binary (двоичный) и digit (знак).

Байт не всегда равнялся 8 битам. Термин «байт» впервые был использован в 1956 г. при проектировании первого суперкомпьютера IBM 7030 Stretch для обозначения последовательности битов числом от 1 до 6.

В советских ЭВМ БЭСМ-6, М-220 байт равнялся 6 битам, а в ЭВМ Минск-32 использовался 7-битный байт.

26.2. Единицы измерения объёма информации

В своей деятельности человек постоянно применяет различные единицы измерения. Например, единицы измерения времени — секунды, минуты, часы; расстояния — метры, километры; температуры — градусы и т. д. Измерения производят с помощью измерительных приборов (пример 26.5).

Для определения количества информации также существуют свои единицы измерения. Минимальное количество информации, для кодирования которой достаточно одного двоичного разряда, называют **битом** (bit). Бит является минимальной единицей, выражающей количество информации. При этом он может принимать только одно из двух значений: 0 или 1. Более крупная единица измерения информации — байт (пример 26.6).

Байт — единица измерения количества информации, состоящая из восьми последовательных битов.

1 байт = 2^3 бит = 8 бит.

Для обозначения большего объёма информации используются другие единицы измерения:

1 Кбайт (килобайт) = 1024 байт = $= 2^{10}$ байт;

1 Мбайт (мегабайт) = 2^{10} Кбайт = $= 2^{20}$ байт;

1 Гбайт (гигабайт) = 2^{10} Мбайт = $= 2^{30}$ байт;

1 Тбайт (терабайт) = 2^{10} Гбайт = $= 2^{40}$ байт;

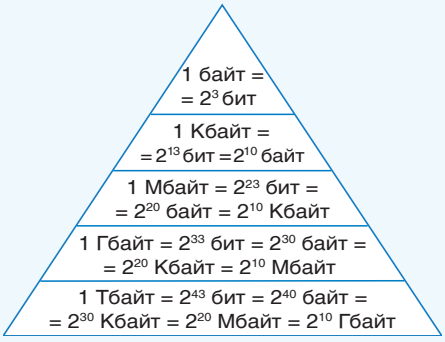
1 Пбайт (петабайт) = 2^{10} Тбайт = $= 2^{50}$ байт;

1 Эбайт (эксабайт) = 2^{10} Пбайт = $= 2^{60}$ байт.

Значения данных единиц взаимосвязаны между собой различными степенями числа 2 (пример 26.7). Они позволяют измерять количество (объём) оперативной или внешней памяти компьютера, размеры файлов. В этом случае в качестве измерительного прибора используется компьютер.

В примере 26.8 показано, как можно переводить одни единицы измерения в другие: мегабайты в килобайты и в гигабайты.

Пример 26.7. Соотношение единиц измерения информации.



Приставки кило-, мега-, гига-, тера-, которые используются в названиях единиц измерения информации, являются общепринятыми. Однако в международной системе единиц (СИ) этим приставкам соответствуют другие числовые значения (10^3, 10^6, 10^9, 10^{12}, 10^{15} и 10^{18}). Для единиц измерения информации применяются другие приставки: киби-, меби-, гиби-, теби-, пеби-, эксби-.

Пример 26.8. Перевести 2368 мегабайт в килобайты и гигабайты:

2368 Мбайт = $(2368 \cdot 2^{10})$ Кбайт = $= 2\ 424\ 832$ Кбайт.

2368 Мбайт = $(2368 / 2^{10})$ Гбайт $\approx$ $\approx 2{,}3$ Гбайт

1. Что такое код?

2. Какой процесс называют кодированием информации?

3. Какой код используют для кодирования информации в компьютере?

4. Какие единицы измерения информации вы знаете?

Правообладатель Адукацыя і выхаванне

Упражнения

1 Выполните перевод единиц измерения информации.

1. 174 байта в биты.
2. 342,3 Кбайт в байты.
3. 45 638 Мбайт в гигабайты.

2 Закодируйте фразу «Кто владеет информацией, тот владеет миром», используя шифр Юлия Цезаря со сдвигом на 3 позиции вправо.

3 В азбуке Морзе буквы заменяются последовательностями из коротких и длинных сигналов — точек и тире.

А	Б	В	Г	Д	Е	Ж	З	И	К	Л	М	Н	О
.-	-...	.--	--.	-..	.	...-	--..	..	-.-	.-..	--	-.	---
П	**Р**	**С**	**Т**	**У**	**Ф**	**Х**	**Ц**	**Ч**	**Ш**	**Щ**	**Ъ, Ь**	**Ы**	**Э**
.--.	.-.	...	-	..-	..-.	....	-.-.	---.	----	--.-	-..-	-.--	..-..
Ю	**Я**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**0**		
..--	.-.-	.----	..---	...--	....-	.....	-....	--...	---..	----.	-----		

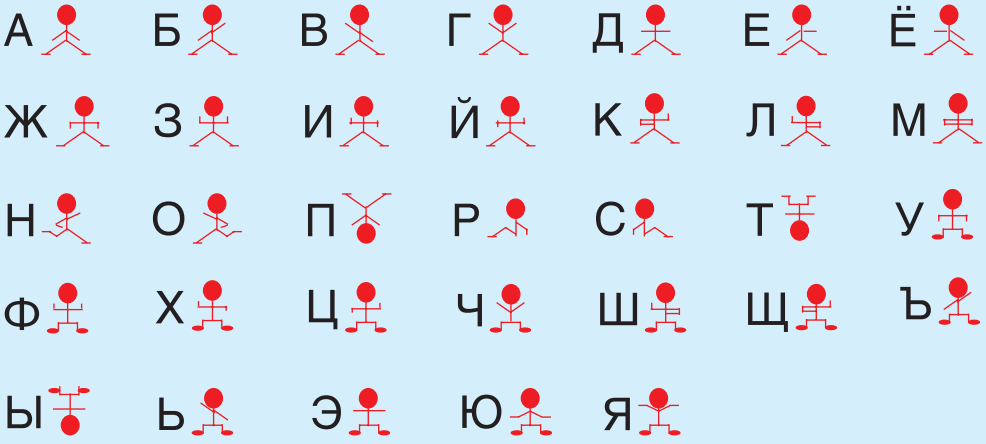
1. С помощью азбуки Морзе запишите: «Запас беды не чинит».
2. Расшифруйте информацию, представленную с помощью азбуки Морзе.

-.	.-		---	----	..	-...	-.-	.-	....		..-	---.	.-	-	...	.-.-

4* В одном из рассказов А. Конан Дойля великий сыщик Шерлок Холмс разгадывает шифр пляшущих человечков. Расшифруйте фразу.



Алфавит для кодирования информации



5* Найдите в сети Интернет ответы на следующие вопросы.

1. Какие шифры встречаются в литературных произведениях?
2. Как искусственный интеллект может расшифровывать старинные книги?

Для получения ответа перейдите в редактор Яндекс Браузера или в режим ИИ в Google Chrome.

Правообладатель Адукацыя і выхаванне

Учебное издание
Котов Владимир Михайлович
Лапо Анжелика Ивановна

ИНФОРМАТИКА

Учебное пособие для 7 класса
учреждений образования, реализующих образовательные
программы общего среднего образования,
с русским языком обучения и воспитания

2-е издание, пересмотренное
и дополненное

Зав. редакцией *Г. А. Бабаева*. Редакторы *А. М. Гаврилова, С. Г. Девдера*. Художественный редактор *О. Н. Карпович*. Художники *В. А. Якубовская, И. И. Дубровская*. Техническое редактирование и компьютерная вёрстка *И. И. Дубровской*. Корректор *С. Г. Девдера*.

Подписано в печать 06.11.2025. Формат $70\times90\,{}^{1}/_{16}$. Бумага офсетная. Печать офсетная. Усл. печ. л. 14,04. Уч.-изд. л. 8,0. Тираж 116 500 экз. Заказ .

Республиканское унитарное предприятие «Издательство “Адукацыя і выхаванне”». Свидетельство о государственной регистрации издателя, изготовителя, распространителя печатных изданий № 1/19 от 02.08.2013. Ул. Будённого, 21, 220070, Минск, Республика Беларусь.

Открытое акционерное общество «Полиграфкомбинат им. Я. Коласа». Свидетельство о государственной регистрации издателя, изготовителя, распространителя печатных изданий № 2/3 от 04.10.2013. Ул. Корженевского, 20, 220024, Минск, Республика Беларусь.

(Название учреждения образования)

Учебный год	Имя и фамилия учащегося	Состояние учебного пособия при получении	Оценка учащемуся за пользование учебным пособием
20 /			
20 /			
20 /			
20 /			
20 /			
20 /			

Правообладатель Адукацыя і выхаванне

Котов, В. М.

К73 Информатика : учебное пособие для 7-го класса учреждений образования, реализующих образовательные программы общего среднего образования, с русским языком обучения и воспитания / В. М. Котов, А. И. Лапо. — 2-е издание, пересмотренное и дополненное. — Минск : Адукацыя і выхаванне, 2025. — 192 с.: ил.

ISBN 978-985-34-0336-7.

Первое издание учебного пособия вышло в 2017 г. в издательстве «Народная асвета».

УДК 004(075.3=161.1)
ББК 32.81я721